Rethinking Memory Management across Modern Computing Stacks: CPU, I/O, and GPU Memory

by

Siyuan Chai

Dissertation

Submitted in partial fulfillment of the requirements
for the degree of Doctor of Philosophy in Computer Science
in the Graduate College of the
University of Illinois Urbana-Champaign, 2026

Urbana, Illinois

Doctoral Committee:

Associate Professor Tianyin Xu, Chair
Professor Josep Torrellas
Professor Darko Marinov
Assistant Professor Saugata Ghose
Assistant Professor Saksham Agarwal
Assistant Professor Weiwei Jia, University of Rhode Island
Associate Professor Rachit Agarwal, Cornell University

Abstract

Modern computing stacks are complex and heterogeneous: datacenters are fabrics of CPUs, GPUs, accelerators, and high-bandwidth interconnects, running workloads—LLM training, in-memory analytics, network-intensive applications—that push the hardware to its limits. Memory is the essential resource of this stack; every workload, on every device, ultimately reads and writes memory, and it is the memory management systems—the OS kernel, the IOMMU subsystem, device drivers, and AI frameworks—that tie the stack together. Yet these memory management systems have not kept pace with the hardware and workloads they now serve, and the cost of the mismatch is paid in performance, security, and extensibility.

This dissertation identifies three concrete manifestations of the gap. First, commodity OS kernels remain hardwired to radix-tree page tables, blocking experimentation with new translation architectures that promise to recover much of the 20–50% address-translation overhead on terabyte-scale servers. Second, strict I/O memory protection in virtualized environments incurs 83–95% throughput degradation from per-DMA VM exits, forcing production deployments to disable it entirely and accept the security risk. Third, CUDA Unified Virtual Memory (UVM) is over 50× slower than PyTorch's native `cudaMalloc`-based allocator when GPU memory is not oversubscribed, and over 5× slower than application-level offloading under oversubscription, forcing each AI framework to implement its own offloading logic.

We address each with a targeted system. *Extensible Memory Translation (EMT)* gives Linux an architecture-neutral interface for memory translation, enabling OS support for diverse hardware schemes—hash-table-based, flat, and hybrid—without modifying the kernel for each. *Virtually Free* redesigns the Linux IOMMU invalidation protocol around one-for-many invalidations and deferred free page, achieving performance within 1–8% of

an unprotected system—74–147× better than the state of the art—while preserving strict security guarantees. *PyTorch-UVM* integrates a UVM-aware caching allocator and framework-guided eviction into PyTorch, enabling practical GPU memory oversubscription for LLM inference without modifying application code.

Together, these systems demonstrate that bottlenecks in modern memory management arise not from hardware limitations but from memory management systems designed without deep understanding of the hardware capabilities and workload characteristics they now serve, and that targeted, hardware- and workload-aware changes suffice to eliminate these bottlenecks without compromising security or extensibility.

Acknowledgements

First and foremost, I would like to thank my advisor, Professor Tianyin Xu, for his boundless energy, rigorous pursuit of academic excellence, and unwavering support throughout my PhD journey. In 2021, I arrived at UIUC with little experience in real-world systems and limited ability to carry out independent research. Our first project on memory translation took far longer than expected and encountered many more challenges than we had planned for, but Tianyin's patience and guidance helped me overcome them and gain confidence in my research abilities. Together we went through countless rounds of research ideas, manuscript edits, and presentation details over many late nights. I am deeply grateful for his passion and his dedication to both his research and his students. As he often said, "Research never dies," and this will continue to inspire me throughout my research journey. Through my time with Tianyin, I can say with confidence that I have become a more professional researcher and a better person.

I would also like to thank my committee members, Professor Josep Torrellas, Professor Darko Marinov, Professor Saugata Ghose, Professor Saksham Agarwal, Professor Weiwei Jia, and Professor Rachit Agarwal, for their valuable feedback and guidance on my research. I met Josep on the first day of my PhD journey, and his strong passion for research and deep insights into computer architecture greatly influenced my EMT project. I am very fortunate to have met Darko, who gave me strong support for my research from a software engineering perspective. I am also grateful to Saugata, with whom I had the good fortune to work on the memory colocality project. The project gave me another concrete experience of hacking the Linux kernel.

I met Saksham after I finished the EMT project, when I was looking for a new project beyond classic CPU memory. Saksham's experience with IOMMU and networking introduced me to the challenges and adventures of building a secure and performant IOMMU subsystem. I am very grateful, as this later became a concrete project for my

PhD dissertation.

I worked with Weiwei on the EMT and DMT projects. Weiwei's strong background in computer systems and virtualization brought tremendous help to both projects. I am especially grateful for his help during the evaluation process, which helped me articulate our arguments and make our results more convincing.

I would like to thank Rachit for his help with the vFree project. We had so many interesting discussions on the security and performance of vFree. Rachit was always encouraging and supportive of our ideas and plans, and he helped us prioritize tasks during the deadline crunch.

I wish to give special thanks to my undergraduate advisor, Professor Peter Dinda from Northwestern University. He introduced me to the field of computer systems and inspired me to pursue a PhD. My undergraduate research experience with him was hands-on, and full of fun. Peter showed me so many fascinating systems tricks and research ideas that I became determined to make this field my own.

The EMT project was my first project in my PhD, and it spanned almost four years. I would like to give special thanks to Jiyuan Zhang, who offered countless interesting research ideas and much practical advice on how to hack the Linux kernel faster. The paper would not have been possible without his help. I want to thank Jongyul Kim, who helped me articulate ideas and pushed me to think more deeply about the research questions. I am very lucky to have come across Alan Wang, who helped me test EMT on different architectures, and Fan Chung, who helped me with radix tree support for EMT and its QEMU implementation. ECPT is one of the most important architectures supported by EMT; it was developed by Professor Dimitrios Skarlatos and pushed further by Jovan Stojkovic. I wish to express my gratitude to them for sharing their knowledge and expertise from the architecture perspective. Finally, I want to thank Rubin Du and Shanbo Zhang, who helped me with the EMT artifact evaluation and won all three badges.

I shifted my focus to IOMMU during the vFree project. I would like to give special thanks to Leshna Balara, who helped with the design, measurement, debugging, security analysis, and writing of the paper. She went through almost every detail with me, and we would not have been able to make the paper ready for submission in time without her help. I am very grateful to Ishvik Singh for his help with setting up various evaluation environments and testing the vFree prototype. I am very grateful to Owen Cochell for his effort to implement vFree in modern drivers and test them on different platforms. I also wish to thank Jiyuan Zhang again for his research ideas on this project.

I was very fortunate to join NVIDIA as an intern in the summer of 2025 and work on the PyTorch-UVM project. My mentor, Ram Tummala, was very supportive and flexible with my research plan, and his guidance and technical insights led to a successful internship. During my internship, I was very fortunate to work with a great team of engineers and researchers, including Balbir Singh, Alistair Popple, Guilherme Cox, Mitko Haralanov, Ziyi Zhao, Vishnuswaroop Ramesh, and Vivek Kini, who provided strong technical support in GPU/CUDA driver implementation and testing. My manager, Rahul Anand, was especially supportive of my funding and research ideas.

Industry internships were a critical part of my PhD journey. I interned at Google in the summer of 2022 and at Meta in the summer of 2024. I wish to thank my mentors at Google, Alex Tran, Michael Wong, and Havard Skinnemoen. This was my first internship experience in the US, and it gave me first-hand experience working with large-scale systems. At Meta, I worked with Jaewon Lee on chunked prefill. I am grateful for his mentorship, which introduced me to the world of real-world AI systems.

I am extremely fortunate to have met groups of close friends during my PhD journey. First, I want to thank my xlab peers. Xudong Sun is the most senior PhD student in our group. We met even before I started my PhD, and he supported me throughout the journey. Jinghao Jia also worked on the OS kernel, and we spent countless nights together hacking code, talking about research ideas, and debugging kernel bugs. Tyler

Gu, Yinfang Chen, Shuai Wang, and Xuhao Luo started their PhD in the same year as me. We spent countless days and nights together talking about research or simply sharing our life experiences. Jiyuan Zhang collaborated with me on almost every project in my PhD. Beyond working together, we often had interesting discussions about the different corner cases of systems and of the world. Wentao Zhang became the person who stayed with me late at night during the last few years. Wentao was especially helpful, rebooting my failed machines several times during deadline runs. In my later years of PhD, I also met Cathy Cai, Jackson Clark, Yiming Su, Ayush Bansal, and Owen Cochell, with whom I shared many conversations about research and life.

Beyond my lab, I want to thank my friends who supported me throughout my PhD journey. Thank you to Peiye Guan, Zhuoran Han, Yongzhou Chen, Youning Chen, Junzhe Kang, Boxin Wang, Mingye Xiong, Nanjie Yu, Xiaoqin Cai, Yunqi Li, Xuan Zhang, Yinbo Tang, Nicko Wei, Bingyan Liu, Chuyang Xu, Xiying Zhao, Hou Xie, Dylan Zhang, Lan Li, Lan Jiang, Zikun Liu, Bill Tao, Dingyuan Cao, Lilia Tang, Chirag Shetty, Gangmuk Lim, Wanyu Zhao, Wenjie Ma, Jiaqi Lou, Jiaxin Wan, Shuang Song, Xinyi Liu, Jiaxi Li, Billy Li, Mingyuan Wu, Kaizhuo Yan, Sizheng Zhang, Yifan Zhao, Zixin Huang, Yan Sun, Gabriella Cai, Benjamin Reidys, Daixuan Li, Shaobo Li, Ke Zhong, Ziyi Zhao, and Kaiyang Zhao. Special thanks to Jinghan Sun, Ying Jing, Yibo Zhang, Bowen Jin, Yucheng Ma, Bowen Chen, Shenjiang Liu, Hongbo Zheng, and Hanqi Mao for the wonderful soccer games. Special thanks to Peiye Guan, Tingkai Liu, Jiakun Yan, Youning Chen, Yongzhou Chen, Shenjiang Liu, and William Zheng for the competitive badminton games. Special thanks to Jiakun Yan, Danyang Duan, Jing Li, Yuqing Jian, and Daniel He for the excellent frisbee games.

Finally, I want to thank my family, who supported me throughout my PhD journey. They gave me unconditional love and encouragement, and I am forever grateful.

To the journey of discovery and the pursuit of knowledge.

Contents

Chapter 1 Introduction

Modern workloads run on top of heterogeneous hardware. A datacenter is a fabric of CPUs, GPUs, accelerators, smart NICs, and disaggregated memory pools, knitted together by hundred-gigabit interconnects and shared by millions of tenants: server DRAM reaches terabytes per socket; GPUs are the dominant substrate for AI, linked by cache-coherent interconnects such as NVLink-C2C; and DMA-capable devices—NICs, NVMe drives, accelerators—move data at hundreds of gigabits per second without ever involving the CPU. The workloads on this hardware are equally demanding: in-memory databases generate working sets of hundreds of terabytes [1]; network-intensive applications in virtualized clouds issue DMA at rates close to local DRAM bandwidth [2]; and a single LLM training run marshals tens of thousands of GPUs across hundreds of hosts to fit a model whose parameter count has grown by orders of magnitude in five years [3].

Across this hardware and workload landscape, memory is the essential resource. Every workload, on every device, ultimately reads and writes memory: a CPU thread chases pointers through a terabyte heap; a NIC streams packets directly into guest physical memory; a GPU performs tensor computations by reading from and writing to device and host memory.

It is the memory management systems that tie this stack together. Data-intensive applications rely on the operating system kernel to set up page tables and translate their terabyte-scale address spaces. Network-intensive applications rely on the guest kernel's IOMMU subsystem and device driver—together with the host kernel and hypervisor—to mediate every DMA a device performs and protect memory from unauthorized access. AI workloads rely on the GPU driver and the AI frameworks built atop it to place, migrate, and evict tensors across device and host memory. Whatever the workload, its performance, security, and extensibility are ultimately determined by these memory management systems.

1.1 The Problems

Yet, the memory management systems have not kept pace with the hardware and workloads they now serve. The kernel's memory management subsystem, the IOMMU subsystem, and the device memory management system were each designed for a world that no longer exists: servers with limited memory capacity, device interconnects orders of magnitude slower than today's, and accelerators with dedicated memory large enough to hold the entire working set. Their core abstractions—the radix-tree page table, the synchronous IOTLB invalidation, the application-opaque unified virtual memory—cannot keep up with today's scale and speed, and have become the bottleneck for the workloads that depend on them.

This dissertation argues that existing memory management systems have become a critical **performance** bottleneck, and that the usual ways of relieving that bottleneck force an unacceptable compromise. To stay usable, production systems either sacrifice **security**—disabling expensive protection mechanisms—or forfeit **extensibility**—hardwiring rigid designs that block the deployment of new hardware ideas and slow the broader research and development cycle. Across the stack—from CPU virtual memory, to I/O memory protection, to GPU device memory—the same pattern recurs: as hardware grows faster and more heterogeneous and workloads grow more demanding, the memory management systems responsible for mapping and protecting memory increasingly become the limiting factor.

This dissertation studies these bottlenecks in three concrete settings, each chosen because it is both representative of the broader pattern and important in its own right: CPU virtual memory translation in the OS kernel, I/O memory protection in the OS kernel (specifically, the guest kernel's IOMMU subsystem, in concert with the host kernel and hypervisor), and the GPU memory management system. In each setting, we identify the specific invariant that the legacy abstraction enforces unnecessarily, redesign the memory management system around that insight, and show that targeted, hardware- and

workload-aware changes are sufficient to eliminate the bottlenecks—without refactoring the applications or abandoning the existing computing stacks.

1. **CPU virtual memory translation** has become a major bottleneck for memory-intensive workloads on terabyte-scale servers. Address translation can consume 20–50% of execution time on big-memory workloads [4], [5], [6], [7], and a substantial body of computer-architecture research has proposed faster translation schemes—hash-based [8], [9], [10], tree-based [11], or hybrid [12], [13]—that promise to recover much of this overhead. Yet commodity OS kernels remain hardwired to a single translation architecture, the radix-tree page table, and provide no extensible interface to experiment with alternative designs. The difficulty of building OS support has, in turn, discouraged disruptive architecture research. Hardware proposals are evaluated almost entirely in simulation, under the assumption that OS overhead is constant across translation architectures—an assumption we show to be false.

2. **I/O memory protection in virtualized environments** is prohibitively expensive. In modern clouds, virtual machines are routinely granted direct access to high-performance devices, but allowing a guest-controlled device direct access to physical memory poses severe intra-guest security risks. Existing IOMMU mechanisms offer a strong, "strict," safety guarantee that purges stale device translations immediately after each DMA; however, in virtualized environments, enforcing this guarantee requires at least one VM exit per DMA to invalidate translation entries in IOMMU caches. The resulting overhead is staggering: 83–95% application throughput degradation—a penalty so severe that production deployments universally disable strict protection, leaving them vulnerable to known attack classes. Cloud operators are thus forced into a choice between performance and security that the hardware itself does not require.

3. **GPU memory oversubscription for memory-intensive AI workloads** has become an existential issue for the AI software stack. Flagship model sizes grow faster than GPU memory capacity, and the dominant response has been for each AI framework to implement its own complex, application-specific offloading logic—hand-written code paths for KV-cache eviction, parameter swapping, and activation rematerialization, re-implemented across PyTorch [14], vLLM [15], and DeepSpeed [16]. The CUDA driver offers a transparent alternative—Unified Virtual Memory (UVM)—but it is widely considered impractical for production. Our measurements show that UVM is over 50× slower than PyTorch's native `cudaMalloc`-based allocator when memory is not oversubscribed, and over 5× slower than application-level offloading under oversubscription (consistent with prior work [17]).

These three problems span very different parts of the stack—a kernel data structure, a kernel-level invalidation protocol that spans guest and host, and a GPU memory management system—but they share a common shape. In each, the memory management system fails to adapt to the hardware's capabilities or the workload's characteristics, and the mismatch surfaces as a bottleneck—paid in performance, in security, or in the engineering effort required to work around the system.

1.2 Thesis Statement

Designing memory management systems with deep understanding of hardware capabilities and workload characteristics can eliminate performance bottlenecks without compromising security or extensibility.

1.3 Contributions

This dissertation supports the thesis with three systems, each addressing one of the manifestations described above. Each system retains the surrounding stack—the OS

kernel, the virtualization stack it participates in, or the GPU memory management system—but adds a thin layer of awareness that closes the gap between the abstraction and the underlying hardware or workload.

EMT: An OS Framework for New Memory Translation Architectures. The first problem—hardwired radix-tree translation in Linux—blocks researchers and hardware designers from experimenting with new translation architectures. New schemes such as hash-table-based ECPT [8] and flat FPT [11] have been shown to reduce translation overhead significantly in simulation, but almost no proposals have ever run on a commodity OS. Without OS experiments, hardware research relies on performance models that assume OS overhead is constant across translation architectures—an assumption we show to be false.

We present *Extensible Memory Translation (EMT)*, a pragmatic framework atop Linux to empower different hardware schemes of memory translation such as radix tree and hash table. EMT provides an architecture-neutral interface that 1) supports diverse memory translation architectures, 2) enables hardware-specific optimizations, 3) accommodates modern hardware and OS complexity, and 4) has negligible overhead over hardwired implementations. We port Linux's memory management onto EMT and show that EMT enables extensibility without sacrificing performance. We use EMT to implement OS support for ECPT and FPT, two recent experimental schemes for fast translation; EMT enables us to understand the OS perspective of these architectures and further optimize their designs.

Virtually Free: Making Strict IO Memory Protection Feasible for Virtualized Environments. The second problem concerns IO memory protection in virtualized environments. Device pass-through delivers near-native I/O performance to virtual machines, but allowing a guest-controlled device direct access to physical memory poses severe intra-guest security risks. Virtualized IOMMUs (vIOMMUs) address this by

exposing a virtual IOMMU to the guest, enabling fine-grained DMA protection. However, enforcing the highest level of protection—*strict* invalidation, which purges IOTLB entries immediately after each DMA—requires a synchronous VM exit per unmap in current hardware-assisted designs. A VM exit costs 10–20 μs; a bare-metal IOTLB invalidation costs 200–300 ns. The result is 83–95% application throughput degradation—a penalty so severe that production deployments universally disable strict protection.

We present vFree, a simple modification to existing I/O memory protection mechanisms that makes strict I/O memory protection feasible in virtualized environments. The key insight in vFree is that, in virtualized environments, it is possible to reduce the overheads of strict I/O memory protection to a bare minimum by *increasing* the cost of address translation. Intuitively, this is because each VM exit has much higher cost than the worst-case translation cost. The vFree design incorporates two ideas—*one-for-many invalidations* and *deferred free page*—that, when put together, result in higher address translation cost but significantly reduce the number of VM exits (and hence, I/O memory protection overheads) while maintaining strict I/O memory protection. We implement vFree on Linux, and evaluate it across multiple real-world applications. Our evaluation suggests that, at the cost of one dedicated core, vFree consistently achieves performance within 1–8% of the unprotected system—74–147× higher than state-of-the-art I/O memory protection mechanisms.

PyTorch-UVM: Practical Unified Virtual Memory for Large Language Model Inference. The third problem is GPU memory management for LLM inference. Flagship model sizes grow faster than GPU memory capacity, yet the transparent alternative provided by the CUDA driver—Unified Virtual Memory (UVM)—has been considered impractical for production: prior work reports UVM is more than 5× slower than application-level offloading. The gap has two sources. First, ML frameworks like PyTorch bypass UVM with their own caching allocators, so managed allocations miss

pooling and reuse optimizations. Second, the driver has no visibility into model execution order, so page faults and migrations are triggered late and often evict data that is about to be reused.

We present PyTorch-UVM, an application-aware UVM substrate for PyTorch that closes this gap without changing model code. PyTorch-UVM contributes (1) a *UVM-aware caching allocator* that proactively returns cached memory to the driver under oversubscription, avoiding the OOM kills that naive UVM caching suffers, and (2) *framework-guided eviction*, which uses the framework's knowledge of execution order and tensor reuse to mark reclaimable blocks as discardable, freeing them without copying data back to host. We evaluate on Hugging Face OPT and Qwen inference workloads across A100-PCIe, H100-PCIe, and GH200 NVLink-C2C. When memory is not oversubscribed, PyTorch-UVM matches PyTorch's native `cudaMalloc`-based allocator within 20%. Under oversubscription, where the native allocator crashes with OOM, PyTorch-UVM is 45×–58× faster than default UVM and 27×–35× faster than HF KV-cache offloading. At high oversubscription, performance narrows to 1.6×–1.7× over default UVM.

Summary of Contributions This dissertation makes the following contributions:

- **EMT** (*EMT: An OS Framework for New Memory Translation Architectures*): A framework that gives Linux an extensible, architecture-neutral interface for memory translation, enabling OS support for radically different page table structures without forking the kernel; plus an open platform for evaluating OS kernels on new translation hardware.

- **Virtually Free** (*Virtually Free: Making Strict IO Memory Protection Feasible for Virtualized Environments*): A redesign of the Linux IOMMU subsystem that makes strict I/O memory protection feasible in virtualized environments for the first time without sacrificing throughput.

- **PyTorch-UVM** (*PyTorch-UVM: Practical Unified Virtual Memory for Large Language Model Inference*): An application-aware UVM integration for PyTorch that enables practical GPU memory oversubscription for LLM inference without changing application code.

- **A unifying perspective.** Together, these systems demonstrate that making system software layers hardware- and workload-aware—through targeted changes—eliminates performance bottlenecks without compromising security or extensibility.

1.4 Dissertation Roadmap

The remainder of this dissertation is organized as follows. Chapter Chapter 2 presents EMT and its evaluation on new translation architectures. Chapter Chapter 3 presents the redesigned vIOMMU mechanism and its evaluation on 400 Gbps networking workloads. Chapter Chapter 4 presents PyTorch-UVM and its evaluation on LLM inference across PCIe-based and NVLink-C2C hardware. Chapter Chapter 5 concludes the dissertation.

Chapter 2 EMT: An OS Framework for New Memory Translation Architectures

"It so happens that a tree format is the only sane format..." [18] —Linus Torvalds, 2002

Virtual memory translation has become a major performance bottleneck of emerging memory-intensive computing [4], [5], [6], [7], [8], [11], [13].[1] With unprecedented growth of memory capacity, driven by terabyte-scale memory [20], [21] and memory expanders like CXL [22], [23], TLBs cannot scale in the same way as memory. Moreover, emerging workloads like machine learning, graph processing, and bioinformatics have irregular memory access patterns with weak locality, making TLBs and other MMU caches less efficient. As a result, TLB misses are inevitably increasing, resulting in expensive address translation across the memory system.

However, today's translation architectures were designed at a time of scarce memory, and optimize space-efficiency over performance. The *de facto* schemes organize translations into a multi-level radix tree [24], [25], [26]; upon a TLB miss, the MMU must *sequentially* walk the tree, resulting in multiple memory accesses. The x86-64 architecture uses a four-level tree, with a fifth level added in recent hardware [27], [28]. In virtualized environments, translation overhead is magnified by nested translation [29], [30] which takes a two-dimensional walk over the guest and host page tables, resulting in up to 24 sequential memory accesses on four-level page tables.

To address this pressing problem, many new hardware architectures for MMUs have been developed to realize fast translation for today's terabyte-scale, heterogeneous memory. For example, hashing-based translation schemes are revisited [8], [9], [10], [31], [32], [33], [34], as hashing is inherently more scalable than walking a tree [9]; a recent hashing scheme, based on Elastic Cuckoo Page Table (ECPT), is reported to reduce

[1] This chapter is based on work published as "EMT: An OS Framework for New Memory Translation Architectures" [19].

translation overhead significantly by enabling parallel lookups of page table entries [8], [10]. New translation schemes using flattened or linear page tables [11], [35] have also been developed. In addition, recent studies advocate for hybrid translation architectures that use different schemes collectively or selectively [12], [13], [36], [37], [38], [39].

Unfortunately, OS support for hardware innovations falls short; few aforementioned new hardware schemes were experimented with commodity OSes like Linux. Instead, evaluations of experimental architectures mostly use performance models to estimate OS overhead [7], [9], [35], [39], [40], [41], or trace-driven simulation with traces collected by running workloads on unmodified Linux [7], [8], [9], [41]. The assumption is that OS overhead on different translation architectures is constant. However, our paper shows that translation architectures could have significant impacts on OS performance.

In fact, the difficulties of OS support has affected hardware research—disruptive hardware designs are often considered "*undesirable*" and lose to incremental approaches (see [7]). Our discussions with hardware vendors tell us that the lack of commodity OS support and evaluation is a major barrier to assessing and adopting new hardware translation schemes.

The unsatisfactory OS effort is largely due to memory management systems in commodity OSes not having the extensibility for emerging translation architectures. For example, Linux assumes a radix-tree based page table structure and lacks extensibility to support hardware schemes that cannot fit in its tree definition. As a result, supporting a new architecture often requires heavy modifications of memory management code in architecture-independent modules. Essentially, Linux provides no extensible interface for memory translation, unlike its other subsystems (e.g., VFS for file systems [42]).

Contributions. We develop a pragmatic OS framework and toolchains atop Linux to embrace new hardware translation architectures for today's memory technologies. We term our framework Extensible Memory Translation (EMT). We target Linux as it is still the *de facto* commodity OS and is mostly assumed by architecture research on memory

systems.

EMT provides an architecture-neutral Linux interface that 1) supports diverse memory translation architectures, 2) enables hardware-specific optimizations, 3) accommodates modern hardware and OS complexity, and 4) has negligible overhead over Linux's hardwired implementation. By abstracting translation-related operations, EMT simplifies the effort to support and experiment with new translation architectures on Linux. With EMT, developers only need to implement an MMU driver as a hardware-specific implementation of translation logic, without modifying the architecture-independent memory management code. EMT also makes it easy to profile and analyze OS performance with regard to hardware translation schemes as it abstracts translation-related operations.

We implement EMT on Linux (v5.15), referred to EMT-Linux. We modularize architecture-independent code with EMT API, removing hardwired assumptions. EMT realizes negligible overhead through careful engineering and optimizations (with less than 0.5% overhead on average across key OS operations). Moreover, EMT-Linux realizes all existing features and hardware-specific optimizations in vanilla Linux.

We build on EMT-Linux to add OS support for ECPT [8] and FPT [11], two new translation schemes. Porting these new hardware schemes on Linux without a framework like EMT would require major rewriting of Linux's memory management module. With EMT, supporting them on Linux is modularized with manageable engineering efforts.

Evaluating new hardware schemes on EMT-Linux faces a common challenge of hardware-software codesign—the hardware is not yet available to run the OS. To address this problem, we assemble a toolchain that runs EMT-Linux on QEMU with an emulated MMU where we implement the hardware translation logic. Our toolchain enables us to understand the OS perspective of new translation architectures, such as its OS overhead over x86. The toolchain also supports cycle-accurate hardware simulation.

We share our experience of supporting new translation schemes on EMT-Linux, which

enables us to understand OS memory management challenges beyond hardware perspectives. We present our reflection on the ECPT design and address correctness challenges such as managing the kernel page table (kECPT) and the paradox involved in changing the translations of the kECPT itself and of the kernel code that manages kECPT, as well as performance challenges like efficient locking and memory scanning. Arguably, OS framework support is essential to encourage and embrace disruptive hardware innovations, and EMT is an important step forward.

Summary. This paper makes the following contributions:

- A discussion on empowering new, experimental hardware-assisted translation architectures on commodity OSes.
- EMT as an extensible framework for developing and evaluating OS memory management on new translation architectures, and its implementation on Linux.
- An experience of building ECPT and FPT on Linux using EMT and the reflection on hardware/OS designs.
- An open platform for developing, testing, and evaluating OS kernels on new memory translation architectures.
- Our artifacts: `https://github.com/xlab-uiuc/emt`.

2.1 Background

2.1.1 Memory Translation Hardware

Modern computer systems use hardware-assisted memory translation, where the translation schemes are defined by the hardware architecture. Upon a TLB miss, the MMU searches the translation structures (e.g., a page table) to obtain the translation—the virtual-to-physical address mapping.

x86 Translation Scheme. All x86 processors since Intel 80386 have used a radix tree, as depicted in Figure 1. The depth of this tree has increased from two levels in 80386 to

four levels in x86-64, with the fifth level upcoming [27], [28] and already supported in Linux [43].

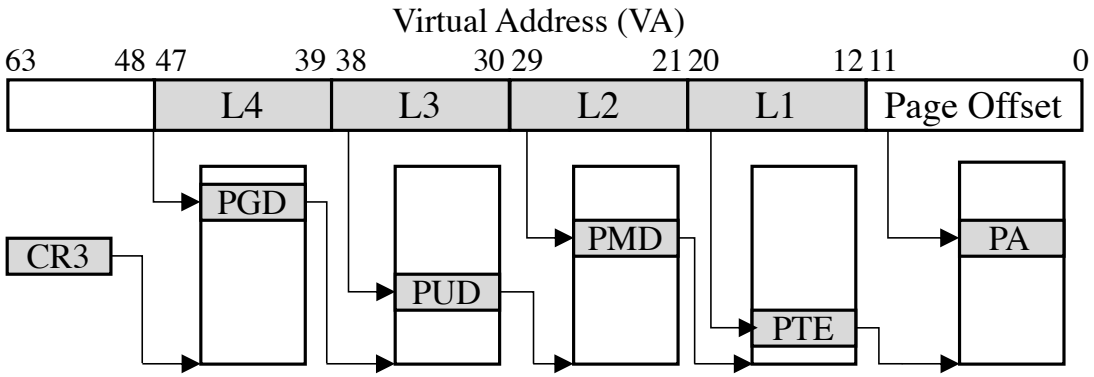


Figure 1: **Radix-tree based page table walk in x86-64 ISA.**

The main advantage of radix-tree based translation is space efficiency—tables at each level are created on demand—if at any level, no page is allocated within an address range, the sub-tree is not allocated. This yields significant memory savings, as the address space of typical applications is sparse.

However, tree-based translation suffers from a major performance drawback—it must *sequentially* walk the tree with multiple memory accesses. Page walk caches [11], [44], [45] and huge pages [6], [46] are used to reduce the length of the walks, but struggle to address emerging workloads with large memory footprints and weak-locality access patterns. The overhead is further magnified in *nested translation* for virtualized environments. The MMU performs a two-dimensional walk over the guest and the host page tables. A nested translation takes up to 24 sequential memory accesses with four-level page tables, and up to 35 with five-level tables. It is reported that nested translation can take more than 50% of the execution time of memory-intensive workloads [7], [12], [47].

Flattened Page Table (FPT). Recently, extensive efforts are being made to optimize memory translation architectures. A common principle is to shorten page table walks of the x86-64 scheme [7], [11], [35], [36], [39], [40], [47]. A recent proposal from Arm, known as Flattened Page Table or FPT [11], [48], flattens the x86 page tables by dynamically

merging intermediate tree levels to reduce indirections and prioritize caching of page table entries. Specifically, FPT tries to merge L4 and L3, as well as L2 and L1, in Figure 1, shortening the walk by half.

Hashing-based Translation and ECPT. Hashing is being actively revisited for translation [8], [9], [10], [13], [31], [32], [33], [34]. We focus on ECPT (Elastic Cuckoo Page Table) [8], a new hashing-based scheme which effectively speeds up translation with fully parallel lookups of page table entries.

Figure 2 depicts ECPT-based memory translation. Different from conventional hashed page tables [9], [49], [50], [51], [52], ECPT uses process-private hashed page tables that are dynamically resized based on occupancy. ECPT uses Cuckoo Hashing [53] and maintains multiple tables (called *ways*) to proactively resolve hash collisions by moving entries across ways. During a page table walks, the MMU computes hashes of the Virtual Page Number (VPN) to look up the ways in parallel. It eliminates the need for sequential tree walks. ECPT supports multiple page sizes (e.g., 1GB, 2MB, and 4KB pages) by maintaining a set of hash page tables for each page size.

To reduce parallel lookups, ECPT uses an in-memory data structure named Cuckoo Walk Table (CWT); each CWT maintains metadata (size and way) of ECPT translation entries. CWTs are cached in special MMU caches named Cuckoo Walk Caches (CWCs). During translation, if a requested page hits a CWC, the MMU can directly look up the specific page table (with the size) or the specific way.

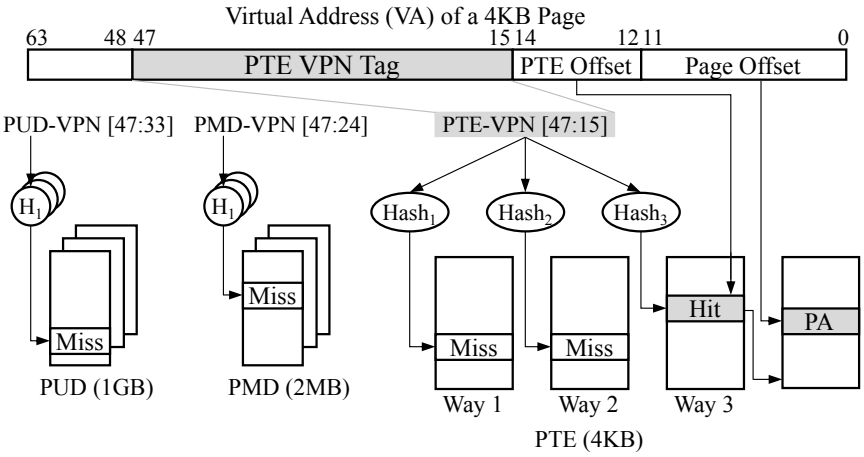


Figure 2: **Parallel page-table lookup in an ECPT-based architecture [8] using three-way cuckoo hashing.**

2.1.2 OS Memory Management

The OS manages in-memory translation structures like page tables and other auxiliary structures (e.g., CWT in ECPT), defined by the hardware architecture. The OS is also responsible for managing translation data (virtual-to-physical address mappings and metadata like protection and dirty bits).

Translation information is consumed by almost all OS memory management operations. Hence, translation architectures have strong implications on OS performance. For example, recent work [54] shows that the majority of page migration cost is from OS memory management on translation-related operations like unmapping/remapping pages and demoting huge pages, instead of actual page copies.

Modern OSes separate *architecture-independent* OS memory management and *architecture-dependent* hardware support. Mach [55] designed a machine-independent memory manager where its architecture-independent code makes few assumptions about MMUs [56]; the design is inherited by BSD kernels [57]. EMT is inspired by Mach/BSD, but focuses on empowering new, experimental translation schemes.

Linux layers machine-independent/dependent code differently. It maintains a

multi-level tree-based page table in the architecture-independent module. This design achieves high-performance memory management: 1) it enables optimizations that need to directly manipulate page tables and translation entries, and 2) it avoids overhead due to indirections. However, it lacks extensibility to different translation architectures especially those that do not fit its tree definition such as ECPT or even the conventional hashed page tables.[2]

2.2 The Need for a New OS Framework

```
1  void vunmap_pmd_range(pmd, addr, end) {
2    pmd_t *pmd = pmd_offset(pud, addr);
3    do { ...
4      // try remove huge page entry
5      int cleared = pmd_clear_huge(pmd);
6      ...
7      if (pmd_none_or_clear_bad(pmd))
8        continue;
9
10     // try remove underlying PTEs
11     vunmap_pte_range(pmd, addr, next, mask);
12     ...
13   }
14   while (pmd++, addr = next, addr != end);
15 } /* mm/vmalloc.c */
```

Figure 3: **Examples of Linux memory management code that is hardwired to radix-tree based translation.**

Emerging memory technologies and research on MMU architectures pose a strong need of developing and evaluating OS memory management on new translation schemes. The current practice of evaluating new translation schemes mostly relies on hardware simulation, either using performance models to estimate OS overhead [7], [9], [35], [39], [40], [41], or replaying traces collected by running workloads on vanilla Linux [7], [8], [9], [41]. Such approaches can hardly capture complex OS-architecture interactions [60]. We

[2]Hashed Page Tables (HPTs) was provided by architectures like IA-64 and POWER [58], [59], but Linux does not have native support for HPTs [18]. Linux maintains a software tree-based page table to store full virtual memory mapping and treats the HPTs as soft-managed extended TLBs. Such an approach leads to duplication of translation data and redundant maintenance.

argue that the lack of OS effort is largely due to memory management systems in commodity OSes not providing an extensible interface for new, different translation architectures.

Hardwiring Translation Schemes Is Untenable. Linux's memory management code is currently hardwired to a five-level tree. Without an extensible framework, it can hardly embrace new translation architectures that use different lookup structures such as hash tables [8], [9], [32], flattened or range tables [11], [35], [36], and hybrid schemes [12], [13].

Figure 3 shows a representative example in Linux, where a memory operation for unmapping a virtual address range at PMD level (`vunmap_pmd_range` in `vmalloc.c`) makes the following assumptions specific to Linux's tree definition:

- A translation entry of a 2MB virtual address range (PMD) can be found from the entry of a 1GB address range.
- An entry of a 2MB address range (PMD) either points to a 2MB huge page or a directory of 4KB pages.
- The translation entry of the next 2MB address range can be iterated by increasing the pointer.

Such implementation patterns are commonplace in Linux's current architecture-independent memory management module. Hence, supporting a different translation architecture would require heavy rewriting of existing Linux code.

In fact, it is also nontrivial for Linux to support translation schemes that use a different tree. Evidently, adding the fifth level took 715 lines of code changes across 23 files [61], all in the architecture-independent modules of Linux. Similarly, though FPT's design strives to limit OS changes [11], it still introduces nontrivial changes of Linux's architecture-independent code to fold the intermediate levels of the tree.

Furthermore, without a well-defined interface, memory management code becomes hard to maintain due to implicit assumptions and semantic overloads with continuous

optimizations and fixes. In Figure 3, `pmd_none_or_clear_bad()` overloads three semantics, making the code hard to maintain.

```
void walk_pte_range(pmd, addr, end, walk) {
  spinlock_t *ptl;
  pte_t *pte = pte_offset_map_lock(mm, pmd, addr, &ptl);
  ...
  for (;;) {
    err = ops->pte_entry(pte, addr, addr + PAGE_SIZE, walk);
    ...
    addr += PAGE_SIZE;
    pte++;
  }
  ...
  pte_unmap_unlock(pte, ptl);
} /* mm/pagewalk.c */
```

Figure 4: **Hardware-specific optimizations in Linux.**

Hardware-Specific Optimizations Are Desired. Inspired by Mach/BSD [56], [57], [62] that have clean separation of machine-independent and dependent code, we started from a pmap-like interface for Linux. In Mach/BSD, architecture-indepedent memory management code uses pmap interface to manage virtual memory mappings. pmap [56] represents a virtual-to-physical address map, which can point to a VAX linear page table [63] or segment registers in IBM RT PC. With pmap, different translation schemes can be supported by writing pmap modules. Typical pmap operations include creation, deletion, and update of address mappings [56], [62].

Unfortunately, we find that a map interface like pmap is not sufficiently expressive to write hardware-specific optimizations, especially in the Linux context. For example, pmap routines for inserting or finding a translation, only reference a virtual address without exposing translation entries. Figure 4 shows three common patterns of optimizations in existing Linux code: 1) batching with fine-grained locks for range operations, 2) in-place translation entry updates with no data copy, and 3) direct fetching of translation entries by offsets. None of them are easy to write using a map, as

they need to directly manipulate translation entries. As optimizations of memory management are critical to Linux performance—27.8% of Linux kernel patches to memory management are for performance optimizations [64]—a simple map interface makes it hard to fully empower translation architectures.

Existing Frameworks Do Not Target Translation. Driven by new memory technologies like tiered memory, several new memory management frameworks [65], [66], [67] have been developed to improve the extensibility of Linux's memory manager. However, few of them concern hardware translation architectures, but focus on application-specific page prefetching and replacement policies. FBMM [65] proposes to reuse VFS [42] and write memory managers as file systems. It exposes the `page_fault` VFS callback for managing page table entries. However, FBMM relies on Linux's existing code that is hardwired to the multi-level radix tree structure, and thus cannot support different hardware translation schemes. Fundamentally, file system interfaces can hardly serve as an effective framework for memory translation.

2.3 EMT Design

Extensible Memory Translation (EMT) is an OS framework built atop Linux with the goal of empowering new, experimental memory translation architectures. Figure 5 gives a high-level overview of EMT. EMT currently focuses on supporting new memory translation architectures in the OS kernel, instead of exposing them to userspace [68]. The design and implementation of EMT achieves the following goals:

- **EMT is an architecture-neutral framework.** EMT is not hardwired to specific translation structures such as multi-level tree in Linux. Supporting a new memory translation scheme should not change architecture-independent code.
- **EMT enables hardware-specific optimizations.** EMT allows architecture-dependent code to customize routines for hardware-specific optimizations. The ability to customize differs EMT from high-level interfaces like pmap in

Mach/BSD [56] and hat in SunOS [69].

- **EMT is modularized and improves maintainability.** EMT provides well-defined translation semantics and organizes them with an object-oriented design. The interface eliminates architecture-specific or overloaded semantics. EMT also makes it easy to profile and analyze OS performance with regard to translation architectures.
- **EMT has negligible overhead.** EMT is carefully implemented with compiler optimization, cache efficiency, and the ability to inline functions. EMT-Linux serves as a performance baseline for new architectures or OS modules.
- **EMT accommodates modern hardware and OS complexity.** EMT aims to develop insights into complex OS-architecture interactions. EMT supports *all* Linux's memory management features related to CPU MMU translation such as huge pages, swapping, DAX memory, etc.

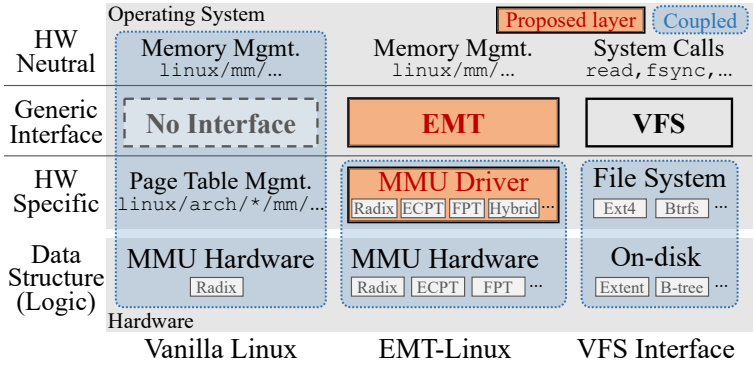


Figure 5: **Overview of the EMT framework, compared with vanilla Linux and analogized to VFS.**

2.3.1 EMT API

EMT exposes an object-oriented API that organizes translation related kernel routines around three architecture-neutral primitives: 1) *translation object* that maintains all the information of a virtual-to-physical address translation, 2) *translation database* that stores translation objects of an address space, and 3) *translation service* that manages the MMU states.

In EMT, all architecture-dependent code is implemented in drivers for MMU architectures, referred to as *MMU drivers*. To support a new translation scheme on EMT-Linux, one needs to write the MMU driver that implements EMT API without the need to modify architecture-independent code.

A key design principle of the EMT API is to abstract the *functions* of translation for OS memory management from hardware-defined *shapes*. Despite many different translation architectures (§2.1), the essential function of memory translation is to output the virtual-to-physical address mapping and all associated metadata, which is abstracted as a *translation object* in EMT. EMT avoids assuming shapes of translation data like translation entry schema or translation data structures. How to encode translation information of hardware-defined shapes into the EMT API is the job of MMU drivers.

The EMT API is organized as 1) *basic functions* that must be implemented by each MMU driver for its supporting translation architecture, and 2) *customizable functions* that have a default implementation that is architecture-independent; the default implementation only uses basic functions and other common code. These routines can be customized by MMU drivers for hardware-specific optimizations.

The basic functions represent the minimal set of functions to support a new translation architecture. Developers can start with basic functions to achieve a functional kernel and then iteratively add optimizations via customizable functions. Basic functions free developers from sophisticated optimizations in Linux; it leads to a more friendly development and test cycle.

```
//// Translation Object
// Read the attribute of an object based on the attribute key
tobj_read_attr(tobj, attr_key) -> (error, attr_value)
// Update the value of a given attribute in an object.
tobj_write_attr(tobj, attr_key, new_val) -> (error, old_val)
...
//// Translation Database
// Find a translation object in a translation database
tdb_find_tobj(tdb, vaddr) -> (error, tobj)
// Update a translation object matching tobj's va
// by copying attributes of tobj.
tdb_update_tobj(tdb, tobj) -> (error, old_tobj)
// Remove the given translation object from the database.
tdb_remove_tobj(tdb, tobj) -> (error, old_tobj)
...
//// Translation Service
// Switch the translation database of the current address space
tsvc_switch_tdb(tdb) -> (error, out_tdb)
// Read the translation database of a given address space
tsvc_read_tdb(cpu) -> (error, out_tdb)
...
```

Figure 6: **Examples of basic functions in the EMT API.** EMT exposes 15 basic functions in total.

Basic Functions Basic functions are expected to be architecture-dependent and are required to be implemented by each MMU driver. Figure 6 gives examples of basic function APIs. The API design avoids assuming specific hardware schemes like the five-level radix tree (§2.1) or page table entry schema. It also prevents overloaded semantics like `pmd_none_or_clear_bad` (Figure 3) due to software evolution.

Translation Object. A translation object encodes a virtual-to-physical address mapping and all its associated metadata. For paged architectures, it encodes translation information stored in page table entries. For tree-based architectures, a translation object encodes information in page table entries that point to the physical page (see §2.4). A translation object can also represent segments or variable-length memory regions. Translation metadata are encoded as attributes such as size, protection, presence, swap, etc., which are queried by general OS memory management. EMT lets

MMU drivers encode metadata into architecture-dependent bits. MMU drivers can also encode architecture-specific attributes and use them to implement architecture-specific features like protection keys and capabilities [24], [70], [71].

Translation Database. Translation objects of an address space are stored in a conceptual database. The database is commonly implemented by a page table (which can be of various shapes) in an MMU driver. It can also be implemented by multiple co-existing page tables (e.g., ECPT needs separate page tables for each userspace process and for a shared kernel space across processes; see §2.5), or by segments or VMA registers [13], [39]. EMT requires a translation database to return one and only one translation object for a virtual address, as the function of the database. EMT does not concern the shape of the database. Its basic functions abstract away architecture-specific structures.

The EMT API decouples translation objects from translation databases; the former does not concern how translation data are stored, while the latter does not interpret translation objects. The benefit is modularity, making it easy to reuse existing code (as shown by the FPT and ECPT MMU drivers).

Translation Service. A translation service abstracts the MMU of a system. It is the root of the EMT API, and manages the creation, destruction, and switching of address spaces. Upon a context switch, the translation service is called to switch the databases. Translation service decouples translation management from task management.

```
//// Translation Object Iterator
// Return the current translation object and advance the iterator
tobj_iter_next(iter) -> (error, tobj)
...
//// Huge Page
// Check if a given virtual address range can be a huge page
thp_eligible(tobj, pg_size, vma) -> eligible
...
//// Address Range
// Check if the given virtual address range has no mapping in it
addr_range_void(tdb, start, end) -> is_void
...
//// Lock
// Get a lock to protect all translation objects in the range
addr_range_get_lock(tdb, start, end) -> (error, tlock)
...
//// Swap
// Get the Linux swp_entry_t from a translation object
tobj_to_swap(tdb, tobj) -> (error, swp_entry)
...
```

Figure 7: **Example customizable functions in the EMT API.** EMT exposes 35 customizable functions in total in 7 groups.

Customizable Functions Customizable functions provide an interface for MMU drivers to implement hardware-specific optimizations. Each customizable function has a default architecture-neutral implementation using basic functions and other architecture-independent code. Figure 7 gives examples of customizable function APIs.

Customizable functions are exposed to MMU drivers via a combination of redefinable macros, following Linux's convention. The architecture-neutral version is wrapped in `#ifndef` so that it can be used if no architecture-specific implementation is available. If an MMU driver wants to implement a customizable function, it defines the customizable function name to its own implementation; otherwise, the `#ifndef` redirects the interface to the default implementation. Since customizable functions always have an architecture-neutral implementation, adding new customizable functions in EMT (if needed in the future) will not break existing MMU drivers.

In principle, customizable functions are those that can benefit from

```c
int tobj_iter_next(struct tobj_iter *iter, struct tobj *tobj)
{ ...
  int ret = tdb_find_tobj(iter->tdb, iter->va, tobj);
  ret = tobj_read_attr(tobj, TOBJ_ATTR_SIZE, &size);
  if (!ret) iter->va += size;
  return ret;
} /* mm/emt-generic.c */
```

(a) Default (architecture neutral)

```c
int tobj_iter_next(struct tobj_iter *iter, struct tobj *tobj)
{ ...
  // handling the most common case of iterating in a 2MB range
  if (iter->ptep) {
    tobj->va = iter->va;
    tobj->pte = iter->pte;
    if ((iter->va + PAGE_SIZE) & (~PMD_MASK)) {
      iter->va += PAGE_SIZE;
      iter->pte++; // Exploit radix's spatial locality
      return 0;
    }
    // Cross 2MB boundary, update ptep based on pmd
  }
  ... // handling other cases.
} /* arch/x86/mm/radix.c */
```

(b) x86-64 radix MMU driver

Figure 8: **A customizable function of the iterator.**

architecture-specific optimizations. The choice now is through high-level reasoning together with profiling that identifies performance-critical code. Currently, EMT exposes different groups of customizable functions, as exemplified by Figure 7. We discuss the iterator group as an example.

Translation-Object Iterator. Iterating over a large number of translation objects is a common management pattern when the OS scans a memory region (e.g., for page migration [72] and huge-page promotion [73]). The performance of such operations is critical, e.g., an optimized iterator can reduce page fault handling cost by 52.5% (§2.7.4). EMT provides a translation-object iterator with customizable functions. Figure 8a shows the default implementation of the iterator's `tobj_iter_next()` function, which is not

efficient—the OS needs to walk from the root to the leaves of the page-table tree in every iteration using `tdb_find_tobj`. Note that such OS walks cannot benefit from hardware caches like PWCs as they are done by software. Figure 8b shows the customized implementation of the x86-64 MMU driver that leverages spatial locality of the radix tree to directly increment the pointer to get the next object. We discuss the implementation of the ECPT MMU driver in §2.5.2.

EMT's iterator functions differ from Linux's page-table iterator [74]—it does not assume the radix tree structure. EMT's iterator does not return intermediate entries (which are specific to tree schemes), but returns translation objects.

2.3.2 Generality

It is hard to prove generality, but our effort on supporting x86-64 radix tree, FPT, and ECPT indicates that EMT can express different hardware schemes, and *all* related optimizations. These schemes represent tree- and hashing-based translation designs, which ground many emerging architectures. A few new architectures [5], [12], [13], [39] propose hybrid designs which often use a fast scheme for common patterns and a slow scheme for correctness [5], [13], [39], or expose multiple schemes to userspace [12], [68]. For pure hardware-based fast schemes [5], [39], the translation is still the traditional one from an OS perspective (e.g., Midgard [5] uses radix tree as its backend). For architectures that use multiple schemes for different address space segments, the MMU drivers can manage multiple page tables under the hood of a database.

From a metadata perspective, EMT can support address translation schemes with coupled or separate metadata management. For metadata that are separately stored from translation structures, the MMU driver encodes and updates the translation entry and its metadata separately in the translation object. For example, EMT supports protection keys like Intel MPK [71] and hardware capabilities [70] by encoding keys as attributes of translation objects. EMT naturally supports capability-based

translation [70], [75] which encodes permissions into capabilities (protected pointers).

EMT also supports mechanisms that manage cache modes like PAT [76] and MTRR [77]. EMT views the cache modes of a page as attributes of its translation object. When the cache mode of a page is changed, the MMU driver updates the attributes. For example, the x86 MMU driver will update the PAT, PCD, and PWT bits of its translation entry [78].

Limitations. EMT only concerns translation, and does not change operations on physical memory pages. EMT focuses CPU virtual-to-physical translation and does not target IOMMU translation for DMA requests. Linux code for IOMMU translation is also hardwired to the radix-tree scheme; the EMT approach can potentially apply. EMT assumes that programs run with virtual addresses; EMT currently does not support designs that make program use physical addresses directly [79], [80].

EMT currently only supports the native environment; virtualization support is on the roadmap. As the extended page table [24] (EPT) has a similar structure as the native page table, we plan to create a virtualization-specific MMU driver to manage EPT. The driver creates a translation database for each VM and handles tasks like changing EPTP registers upon VM switches and allocating EPT tables. KVM can call EMT APIs accordingly. We expect paravirtualization support (`pv_ops.mmu` [81], [82] in particular) to be encapsulated in the MMU driver: MMU drivers with paravirtualization support can call `pv_ops.mmu` for function patching.

2.4 EMT-Linux with x86-64 MMU Drivers

We develop EMT on Linux, referred to as EMT-Linux. Conceptually, it took four steps: 1) identifying memory management code in architecture-independent modules that are hardwired to x86-like, tree-based translation scheme; 2) rewriting them using the EMT API; 3) moving architecture-specific optimizations into the x86-64 MMU driver, and 4) writing default implementations of customizable functions.

```
void walk_tobj_range(start, end, walk) {
  struct tlock lock; struct tobj_iter iter;
  struct tobj tobj; ulong size;
  struct tdb *tdb = walk->mm->tdb;
  err = addr_range_get_lock(tdb, start, end, &lock);
  addr_range_write_lock(&lock);
  err = tobj_iter_init(tdb, start, end, &iter);
  ...
  while (tobj_iter_has_next(iter)) {
    err = tobj_iter_next(&iter, &tobj);
    err = tobj_read_attr(&tobj, TOBJ_ATTR_SIZE, &size);
    err = walk->ops->tobj(tobj, tobj->va, tobj->va + size, walk);
    ...
  }
  tobj_iter_end(&iter);
  addr_range_write_unlock(&lock);
  addr_range_put_lock(&lock);
} /* mm/pagewalk.c */
```

Figure 9: **Rewriting code in Figure 4 with EMT.**

The EMT implementation on Linux (v5.15) takes 9.5K lines of code changes (7.3K for interface refactoring and 2.2K for the x86-64 MMU driver) in 15 person-months. We changed 196 kernel functions in the `mm` directory of Linux—most memory management code interacts with the page table.

We use macros and inline functions to minimize interface overhead as per Linux's coding style [83]; `#ifndef`-based function redirections are used otherwise. Specifically, we turn functions no more than three lines into macro or inline functions. We ensure that EMT does not break performance characteristics, e.g., no increase of stack size or call stacks, and no decrease of instruction cache hit rates in most cases (§2.7.3).

Moreover, we preserve *all* of Linux's existing architecture-specific optimizations and realize them in the x86-64 MMU driver. Hence, EMT supports all virtual memory features of vanilla Linux and is *transparent* to user applications.

OS Memory Management with EMT. In EMT-Linux, OS memory management is no longer hardwired to specific translation schemes. Meanwhile, EMT-Linux reserves hardware-specific optimizations like in Linux—both basic and customizable functions can

be instantiated by different MMU drivers. Figure 9 shows the code in EMT-Linux that implements the hardwired Linux routine in Figure 4. It uses the translation-object iterator (see Figure 8) to scan all the pages in a memory region. This is a common pattern used in many OS memory management operations such as page promotion and migration, as well as page eviction (e.g., using LRU and MGLRU).

Figure 10 shows a skeleton of the page fault handler in EMT-Linux. In vanilla Linux, the page fault handler walks down the page-table tree level by level to determine the type of faults and dispatch them to corresponding subroutines (e.g., a leaf-level PMD entry implies a 2MB page). Differently, EMT-Linux's page fault handler decouples page sizes from the translation data structures (the radix tree). It looks at the translation object and the corresponding page size based on the faulting address, and decides the subroutine to invoke.

Swapping operates on translation objects that map pages to be swapped. Different architectures may encode swap-page information differently. x86-64 MMU driver stores swap information in a page table entry if the present bit is cleared; huge page swapping [84] is done by encoding PMD entries. ECPT and FPT use the same mechanism. EMT provides customizable functions if architectures need different encoding.

EMT encapsulates translation cache management in MMU drivers, as the cache structure is architecture-specific. For example, ECPT does not use x86-64 PWCs but has CWCs to cache page size and way information [8]. TLB-related code is handled with the same principle. The MMU driver performs TLB flushes and guarantees the consistency of cache state. Batch invalidation is supported by customizable functions.

x86-64 MMU Driver. For the x86-64 radix-tree translation scheme, our x86-64 MMU driver maintains the multi-level tree page table. The MMU driver encodes page table entries at all levels of the radix tree for a given virtual-to-physical address mapping in the translation object. This encoding enables the MMU driver to realize all the

```
static vm_fault_t __handle_mm_fault(vma, vaddr, flags)
{ ...
  struct tobj tobj;
  struct tdb *tdb = vma->vm_mm->tdb;
  err = tdb_find_tobj(tdb, vaddr, &tobj);
  err = tobj_read_attr(&tobj, TOBJ_ATTR_MAPPED, &mapped);
  ...
  if (mapped) {
    if (flags & FAULT_FLAG_WRITE) { // fix the write fault
      err = tobj_read_attr(&tobj, TOBJ_ATTR_WRITE, &write);
      if (!write)
        return do_wp_page(&vmf);
    }
    ... // many other fixes
  } else { // mapping does not exist;
    ...
    while (pg_size > BASE_PAGE_SIZE) { // try huge page
      if (thp_eligible(tobj, pg_size, vma)) {
        int ret = create_huge_page(&vmf, pg_size);
        ...
      }
      pg_size = dec_page_size(tdb, pg_size);
    }
    if(vma_is_anonymous(vma)) { // handle anon page fault
      struct page *page = alloc_page_vma(...);
      if (!page) goto oom;
      struct tobj old_tobj;
      err = tobj_update_attr(&tobj, TOBJ_ATTR_PA,
            page_to_pfn(page) << BASE_PAGE_SHIFT);
      ... // update other attributes like permissions
      err = addr_range_get_lock(tdb, vaddr,
            vaddr + BASE_PAGE_SIZE, &lock);
      addr_range_write_lock(&lock); // lock the range
      err = tdb_update_tobj(tdb, &tobj, &old_tobj);
      addr_range_write_unlock(&lock);
      addr_range_put_lock(&lock);
      return 0;
    }
    ... // handle other type of base page faults
  }
  ... // error handling (e.g., oom)
} /* mm/memory.c */
```

Figure 10: **Snippet of EMT-Linux's page fault handler.**

optimizations specialized for the tree-based x86-64 page table, while making architecture-neutral code agnostic. For example, the MMU driver can directly operate on the PMD or PUD entries to implement Linux's split page table lock [85] through the `addr_range_get_lock` customizable function (Figure 9). The MMU driver can also implement optimizations that leverage spatial locality of the tree structure, as exemplified by the iterator (Figure 8). Encoding attributes of x86 page table entries via translation object API is straightforward by reading/writing bits in the entries, and our x86-64 MMU driver supports hardware-specific features like Intel MPK.

FPT MMU Driver. We implemented an MMU driver for a new translation scheme based on Flattened Page Table (FPT) [11]. The FPT design aims to minimize OS changes [48] and is based on the x86-64 radix-tree page table. Without an interface like EMT, it needs nontrivial changes to Linux's architecture-independent code, including changing macros that define bits in page table entries and checking if a level needs to be folded. The Linux prototype in the FPT paper [11] supports flattening of L3+L2 only. We wrote the FPT MMU driver by reusing the x86-64 MMU driver code with 664 lines of C code. With the EMT API, no architecture-neutral OS code needs to be changed. Our FPT MMU driver supports all three types of flattening patterns of tree levels, and it co-exists with the x86-64 radix MMU driver.

2.5 ECPT on EMT-Linux

2.5.1 Emulator-based Toolchain

It is challenging to develop OSes for new translation architectures without manufactured MMU hardware. We did not find an available toolchain for developing and evaluating OS kernels with experimental architectures like ECPT. The original evaluation of ECPT [8] collects memory traces from simulation using Simics [86] on *vanilla Linux* and replays traces in SST [87] that simulated an ECPT MMU. Simics is closed-source and only supports existing ISAs.

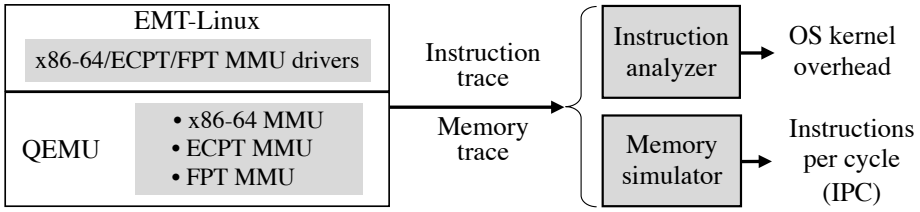


Figure 11: **Emulator toolchain for OS development and evaluation on experimental memory architectures.**

We develop an emulator-based toolchain using QEMU [88] (Figure 11). We use QEMU's software MMU mechanism [89] to develop an emulated ECPT MMU where we implement ECPT translation logic and hardware caches in 3.1K lines of C code. QEMU offers an emulated x86-64 MMU which serves as a baseline for performance evaluation.

Our toolchain can connect to trace-driven hardware simulators (QEMU provides no cycle-accurate simulation). We developed instruction and memory tracers using QEMU's TCG plugin [90]. The instruction trace helps analyze kernel and userspace behavior (§2.7.4), as well as hardware simulation.

2.5.2 ECPT MMU Driver

We implement an ECPT MMU driver on EMT based on the hardware architecture design [8]. We use a three-way ECPT for each page size (4KB/2MB/1GB) so the translation database manages nine control registers, one per way; registers are updated upon a process context switch. The driver also manages nine registers for the kernel page tables (§2.6.1). The ECPT MMU driver takes 7.4K lines of C code.

The ECPT structures are implemented in the ECPT MMU driver. For memory efficiency, we allocate ECPTs of a specific size on demand. We follow the ECPT design of clustering eight translation entries into a 64-byte cache line to improve locality. To do so, we construct the VPN tag (Figure 2) with bits from multiple translation entries, as shown in Figure 12. For the PTE VPN tag, each entry contributes five bits so the first seven entries together make up the VPN tag, and the last entry contributes five bits as the

count of valid entries in the cluster. The count is used to calculate occupancy (for table resizing) and for freeing the cluster. These metadata are managed by basic functions. PUD and PMD entries use the same design but with 24 bits and 15 bits for the VPN tag.

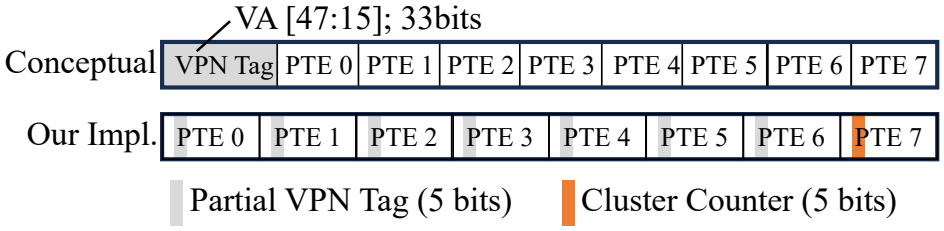


Figure 12: **Implementation of 64-byte PTE-ECPT entry clusters consists of eight 8-byte PTE entries.** Each entry contributes five bits for the VPN tag or a cluster count.

When a table's occupancy exceeds a threshold (0.6 as in [8]), a background kernel thread resizes the table and migrates entries from the old table to the new table gradually.

Except for the repurposed five bits for metadata, ECPT's translation entry by design follows x86-64 PTE format. We reuse the PTE encoding from the x86 radix MMU driver, enjoying the benefit of EMT's object-oriented design.

Cuckoo Walk Tables (CWTs) are implemented in the ECPT MMU driver, invisible to architecture-independent code. We follow the ECPT design [8] to use a 5-bit section header (a section is the address range translated by one ECPT translation entry) and cluster 64 headers into a 64-byte cache line. Each header is one byte and contributes three bits to encode the VPN tag and count of valid section headers.

Optimizations. We implemented a series of optimizations through the customizable functions, driven by benchmarking and profiling (§2.7.4). For example, we customize the iterator with architecture-specific optimizations to accelerate range operations. The idea is to exploit locality within a translation entry cluster: the iterator finds the next entry by pointer arithmetic, instead of hashing-based lookups, when the entry is not the last one in its cluster. One common optimization pattern is to minimize the cost of range operations—the default implementation (e.g., Figure 8a) often takes too many

fine-grained hashing-based operations.

2.6 Reflection on ECPT Design

We show that building OS components is essential to understanding architecture designs. The efforts on developing and evaluating ECPT with EMT-Linux reveals OS challenges of using a fast translation scheme, which were unexpected (undocumented in the original hardware design).

2.6.1 Managing Kernel Page Tables

In modern OSes like Linux, the kernel address space is shared across processes [91]. In tree-based translation, sharing is realized by having high-level page table entries pointing to the shared subtree of the kernel address space [92]. This design does not apply to hashing-based translation. One option is to maintain one ECPT for each process containing both user- and kernel-space addresses. However, such a design is memory inefficient and leads to high overhead, e.g., once a translation of a kernel-space address is updated, the OS must update the corresponding entries in ECPTs of all processes.

Our ECPT MMU driver maintains a global kernel-space ECPT (kECPT) shared among processes and an independent user-space ECPT (uECPT) for every process. The kECPT has the same page size and way configuration as each uECPT. The kECPT and uECPTs are managed independently. When KPTI (kernel page table isolation) [92] is enabled, two independent global kECPTs are managed (a complete kECPT and a minimal kECPT). This design requires ECPT MMUs to expose another nine control registers that point to kECPT(s).[3]

Self-Reference Paradox. Managing kECPTs is more challenging than uECPTs.

[3]The cost of additional architectural registers is negligible; they add to less than 0.01% of the die area. The area overhead was measured with the same methodology as in [8], [10] using CACTI [93] with 22nm technology. The new design also distinguishes whether the target address belongs to the upper or lower half of the address space and directs the lookups to uECPT or kECPT. CWC is shared and has no additional cost.

Different from uECPTs that are mandated by the kernel, the kECPT manages the kernel's own code and data—translations for kECPT management code and kECPT itself are stored in kECPT. This creates a challenge for ECPT which needs to move translation entries across ways for resolving hash collisions—in the moving window, if the entries map the kECPT itself or the kernel code for moving entries are missing, the kernel crashes due to missing translation of the kECPT or code, as shown in Figure 13. Frequent triggers are kernel's huge page promotion/demotion which remove entries and insert new entries in different tables.

We term the issue *self-reference paradox*. The root cause is a lack of hardware support for atomic updates on multiple memory locations. Radix-tree page tables do not face this paradox, because they do not move entries and the huge-page promotion/demotion is done by updating one PUD/PMD entry. In general, the self-reference paradox can happen in kernel page tables that need to move entries (many advanced index schemes need to move entries [94], [95]). Note that other kernel structures like forwarding table [96] do not face this paradox if they do not map kernel code/data.

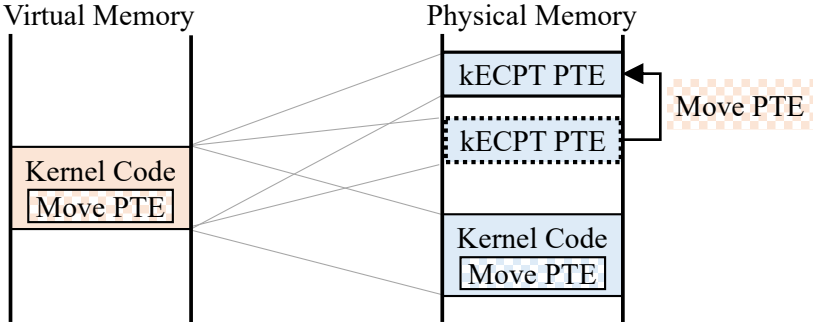


Figure 13: **An example of self-reference paradox caused by the need of moving kECPT entries that are needed to find kernel code for moving kECPT entries (§2.6.1).**

We address the paradox via a series of software-hardware endeavors. First, we always copy the entry to the new location *before* removing it at the old location, which may cause duplicated entries but no missing entries. We instruct the MMU to handle duplicate entries that have the same virtual-to-physical address mapping and the same

protection bits; the deduplication policy also works for huge-page promotion/demotion as it does not change the mapping or protection. We avoid location-changing read-update-write operations in OS code so that duplicated entries always have the same mapping and protection. For ECPT, any operation that may change the content of an entry can only be done after the entry is locked.

Atomic kECPT Switching. Another form of self-reference paradox manifests via KPTI [92]—when switching between the kernel space and the user space, the OS needs to switch between a full kECPT and a minimal kECPT. One potential implementation is to use multiple `mov` instructions to redirect control registers from the old kECPT ways to the new ways. However, this may lead to an inconsistent ECPT state during the switch window, where some registers point to new ways and the others point to old ways. This leads to issues when the translations of currently executing kernel code are stored in different ways across the new and old kECPTs. Note that simply keeping context switch code covered by both kECPT is not enough since the instructions can reside in different ways of the old and new tables. Instructions might not be successfully fetched when kECPT state is inconsistent.

Solving this problem needs a mechanism for atomic switching from all old kECPT ways to the new kECPT ways. Such an atomic switching mechanism can be realized with hardware support. Our solution is to add additional sets of kECPT control registers and the hardware switches between the two sets of control registers, in a similar vein as the `VMLAUNCH` and `VMRESUME` instructions of x86 on updating multiple registers together. The switch is a serializing instruction to ensure that all instructions after it will use the new kECPT.

2.6.2 Implications on OS Performance

Managing Sparse Address Space. A fundamental efficiency property of tree-based page tables is the ability to manage sparse address space, which is critical for OS memory

management which commonly needs to scan large, but sparse address ranges. In tree page tables, a high-level entry can encode properties of a large address range, e.g., a nonexistent PMD entry indicates no valid page allocated in the corresponding 2MB range. Hashed page tables have no such hierarchical relationship between entries; hence, the OS may go through all the possible entries in the large, sparse address range.

One optimization is to enable a similar property in ECPT by designing special entries that are not for translation, but for encoding states. The OS checks the entry to learn the states of the corresponding address range, e.g., the OS can query such an entry to check if the corresponding 2MB address range has any allocated page, instead of checking all possible PTE entries. The design would need to cooperate with the MMU.

Multicore Scalability. We find it nontrivial to implement efficient locking primitives similar to Linux's split page table lock [85] in ECPT, because the location of an ECPT entry can be moved due to insertion or rehashing. The movement of entries makes it prone to deadlocks, e.g., one thread t holding a lock l and inserting a new entry, thus consequently requires moving an existing entry e (see [53]) which is locked by another thread that waiting on l. One solution is to let t release all its acquired locks and retry later when deadlock. The complexity lies in recording and rewinding states changed by t. We are exploring an alternative design that introduces a separate lock table to provide the OS with the flexibility of implementing lock primitives, which has the semantic of locking both an address range and the related entries. Our implementation of the ECPT MMU driver has no correctness issue, as we use a coarse-grained lock.

2.7 Evaluation

2.7.1 Methodology

We validate the correctness of the EMT-Linux implementation using Linux tests. Since EMT supports all virtual memory features in Linux, EMT-Linux should pass any userspace tests.

We also measure the interface overhead of EMT by running the same set of performance benchmarks on EMT-Linux (§2.4) and vanilla Linux running upon the same hardware.

We then measure the performance of EMT-Linux on ECPT. We run EMT-Linux with the ECPT MMU driver on top of an emulated ECPT MMU using our emulation framework (§2.5.1). We compare the performance of EMT-Linux with the Radix MMU driver running on an emulated x86-64 MMU.

Benchmarks. We use LEBench [97] as a micro benchmark to measure the performance of core OS operations such as page fault handling, context switching, and system calls.

We use nine memory-intensive macro benchmarks that stress the TLB and need off-TLB translation. We use the same set of macro benchmarks from the ECPT paper [8], [10] except two where we failed to reproduce the working set. The macro benchmarks include seven applications from the GraphBIG benchmark suite [98]: Breadth First Search (BFS), Depth First Search (DFS), Degree Centrality (DC), Single Source Shortest Path (SSSP), Connected Components (CC), Triangle Count (TC), and PageRank (PR); they use the LDBC-1000K dataset [99], with working sets of about 8.5 GB. We also run the GUPS benchmark [100] which issues random memory updates, and a memory test from Sysbench [101]. Both GUPS and Sysbench have 64 GB working sets.

We also evaluate three memory-intensive applications: Redis, Memcached, and PostgreSQL. Table 1 shows the workloads. All these applications are multithreaded programs.

2.7.2 Functional Correctness

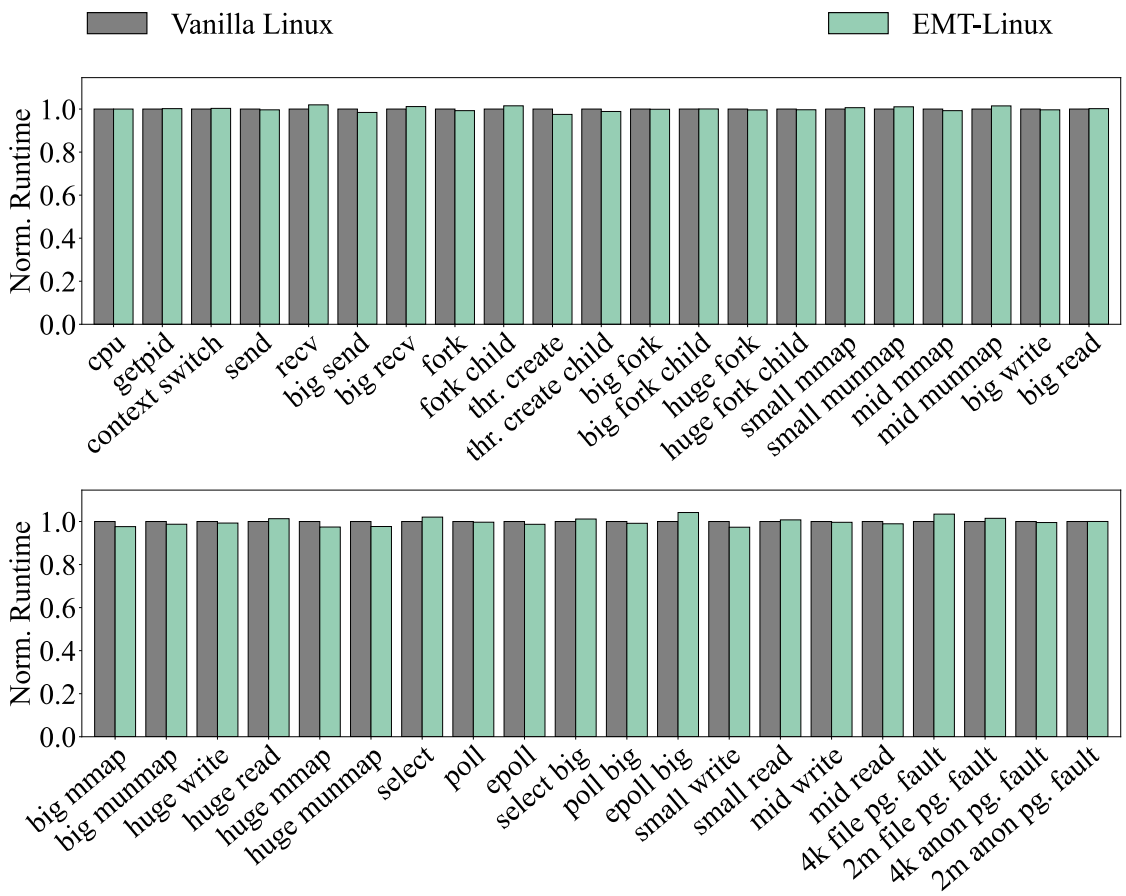
We show that EMT supports all memory management features of Linux by running Linux Test Project (LTP) [102] against EMT-Linux. We use the default kernel configuration for the generic kernel (5.15.0-125-generic) of Ubuntu 20.04. LTP includes 1,405 tests in total and 1,208 of them apply to the kernel configuration. EMT-Linux with Radix, ECPT, and

FPT MMU drivers both pass all 1,208 tests which covers 376 system calls. We also cross-validated program outputs of the micro and macro benchmarks across EMT-Linux and vanilla Linux. We use testing as a continuous effort throughout our development, rather than a one-time effort, which helped capture bugs in a timely manner.

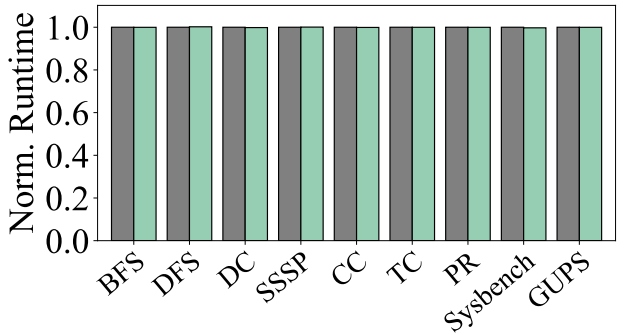
2.7.3 EMT Interface Overhead

EMT introduces negligible overhead. We run the benchmarks and application workloads on vanilla Linux and EMT-Linux running on the same hardware. The hardware is a dual-socket Intel Xeon Gold 6346 server at 3.10GHz with 16 cores and 256GB DDR4-3200 DRAM. We disable hyperthreading and fix core frequency to make the measurement stable. The overhead of EMT is calculated by normalizing the results of EMT-Linux to vanilla Linux. We experimented with various kernel configurations (e.g., enabling THP); the results are consistent.

Figure 14a shows the results of kernel micro benchmarks LEBench [97]. EMT-Linux exhibits an average normalized kernel-routine performance of 99.9% relative to vanilla Linux (with a standard deviation of 1.1%) across the 41 LEBench microbenchmarks. The largest overhead comes from the "epoll big" benchmark, where EMT-Linux slows the benchmark by 4.2%. It was caused by not inlining certain functions due to coding style restriction, which can be further optimized by restructuring the code or forcing inlining.



(a) Micro benchmark



(b) Macro benchmark

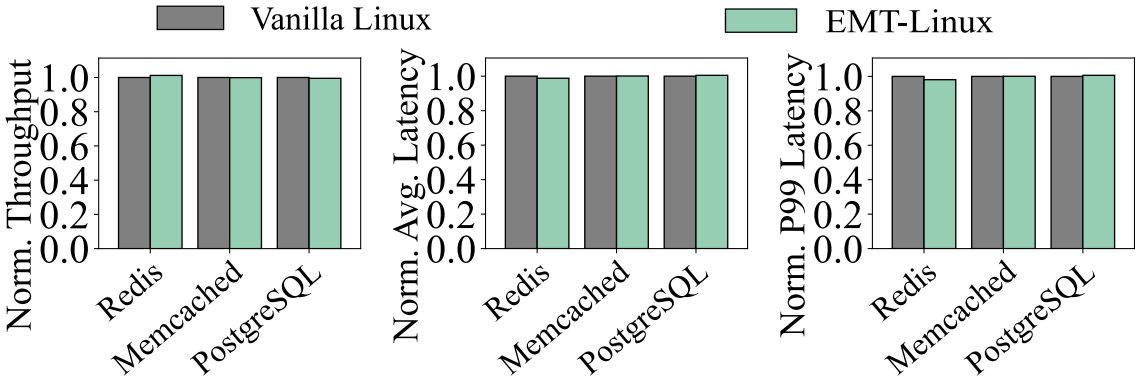
Figure 14: **Performance overhead of EMT-Linux over vanilla Linux (measured by the micro and macro benchmarks).**

Figure 14b shows that the overhead of EMT-Linux is less than 0.1% on the macro benchmarks. Figure 15 shows the normalized throughput, average latency, and tail

latency of three real-world applications (with workloads in Table 1) on EMT-Linux, normalized to their performance on vanilla Linux. The measured differences of the three metrics are within 0.1%, 0.1%, and 0.2%, respectively. The results of the real-world application show that the EMT interface does not affect system responsiveness.

Application	Working Set	# Records	Read:Write	# Requests
Redis	128 GB	536 M	50:50	60 M
Memcached	69 GB	56 M	80:20	10 M
PostgreSQL	64 GB	21 M	100:0	25 M

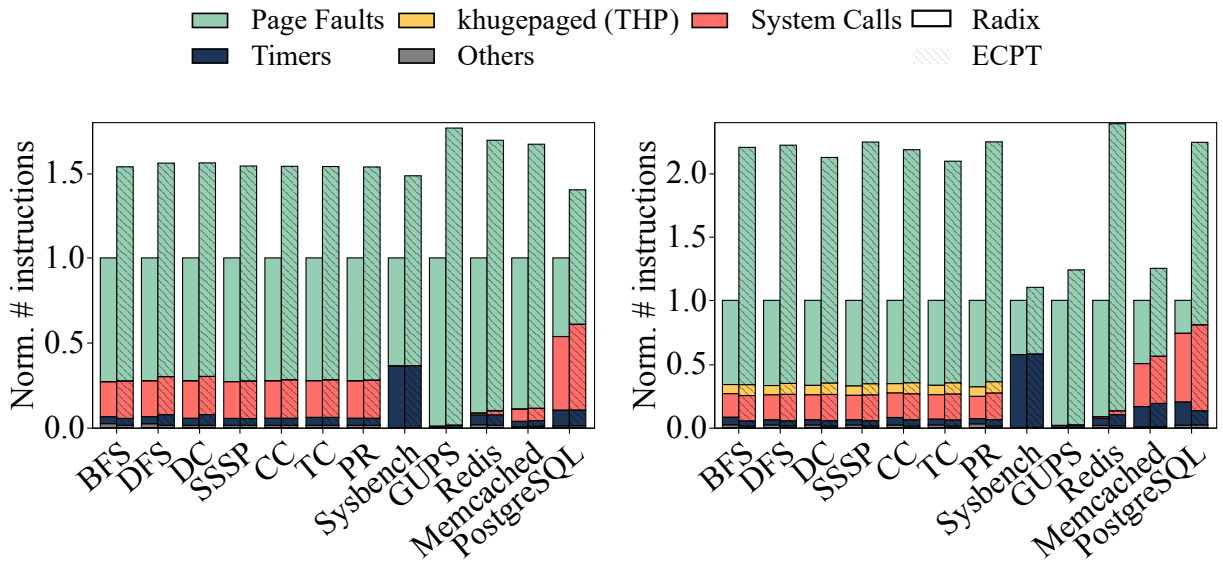
Table 1: **Application workloads used in the evaluation.**



(a) Throughput (b) Avg. Latency (c) P99 Latency

Figure 15: **Performance overhead of EMT-Linux over vanilla Linux on real-world applications**

The negligible overhead is attributed to two kinds of endeavors. First, we carefully engineered the EMT interface to preserve the performance characteristics—we minimize increased stack size or deepened call stacks, and maintain cache efficiency. Instruction cache hit rates on EMT-Linux differ from those on vanilla Linux by at most 0.8%. Second, EMT enables us to implement all the hardware-specific optimizations in the MMU drivers.



(a) 4KB (b) THP

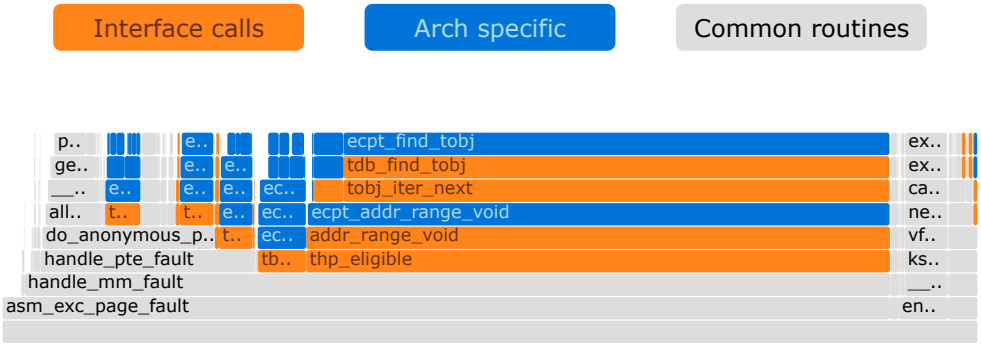
Figure 16: **Distribution of kernel instructions of EMT-Linux with the Radix and ECPT MMU drivers.**

2.7.4 OS Performance on ECPT

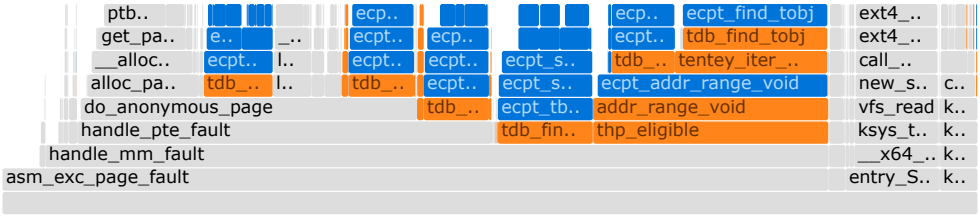
We show that EMT enables us to understand OS performance on new translation architectures using ECPT as an example. We evaluate ECPT by running EMT-Linux with the ECPT MMU driver on our emulation framework and comparing its performance with the x86-64 radix MMU driver. We run macro benchmarks and application workloads and record all kernel and user instructions (with both 4KB pages and THP).

Understanding OS Overhead of ECPT. Figure 16 shows the distribution of kernel instructions; most of them are for page fault handling. Compared with Radix, ECPT spends 1.74x more instructions on page fault handling on average for 4KB pages, and 2.59x more with THP enabled. When THP is enabled, ECPT leads to relatively more work than Radix because Linux's THP implementation uses a few expensive operations to check if there exist valid entries in a given address range. These checks are on the critical path of page fault handling when THP is enabled since the kernel needs to know if the 2MB range has any mappings before it can decide if a 2MB huge page mapping

should be built. For Radix, these checks are cheap (0.39% of total kernel work) because a PMD entry has information on PTE tables. If the entry is not valid or present, then neither 2MB nor 4KB mappings exist; the huge-page bit tells if the entry points to a huge page or a directory of 4KB pages. However, these checks are expensive in ECPT, as ECPT's entries are independent, the kernel may need to check all 512 4KB entries of pages in the 2MB range, which requires many expensive lookup operations. We discussed the potential solution in §2.6.2.



(a) ECPT MMU driver with no iterator optimization



(b) ECPT MMU driver with iterator optimization

Figure 17: **Flame graph of EMT-Linux kernel instructions when running Graph-Big BFS (THP enabled).**

Effectiveness of Optimizations. EMT exposes optimization opportunities for MMU drivers using customizable functions. Figure 17 uses flame graphs to show the effectiveness of iterator optimization running GraphBIG BFS with THP. With the iterator optimization (§2.5.2), ECPT can save 49.0% of total kernel work, and 52.5% of the page fault handling work. The optimization drastically saves the work of hashing and lookups of the architecture-neutral default implementation, by incrementing the pointer

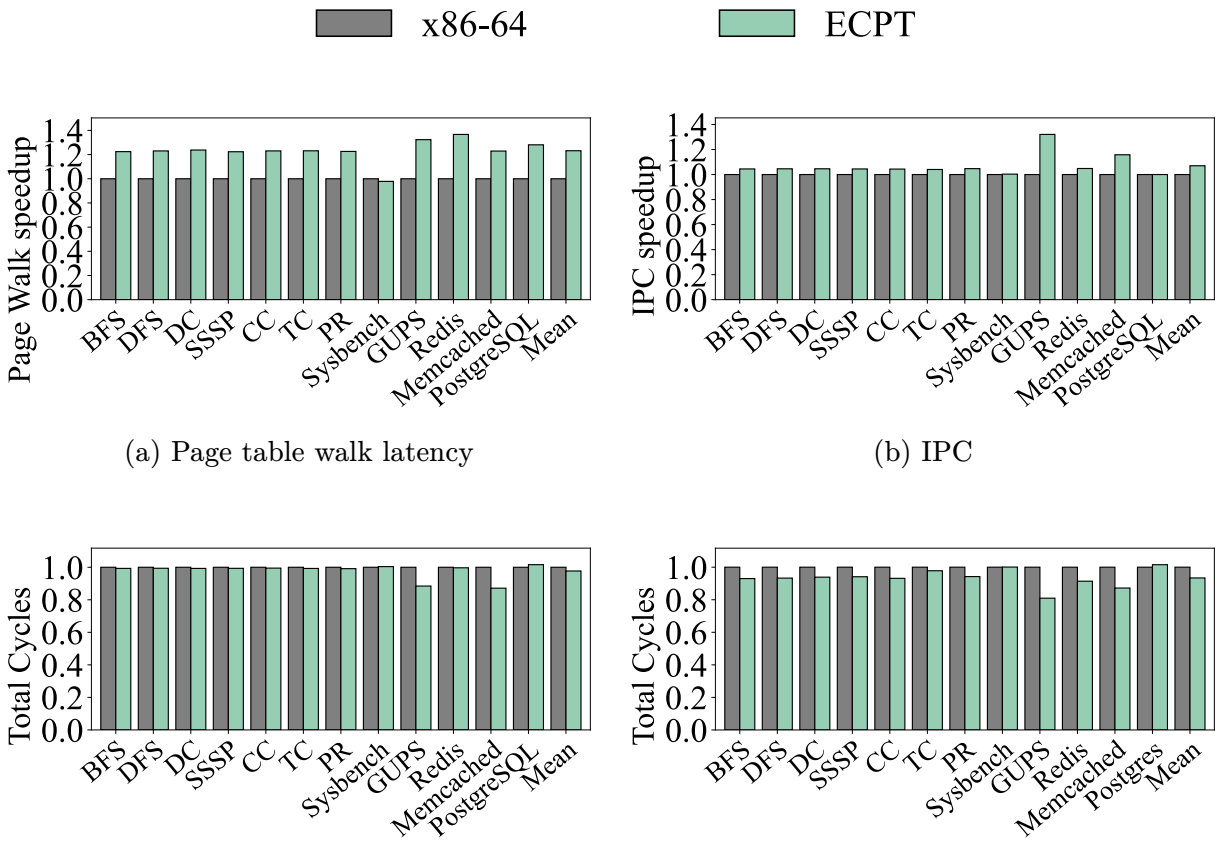
to an entry directly, as long as the entry is within an entry cluster (Figure 12). Note that such kernel work as software overhead cannot benefit from hardware caches such as TLBs or PWCs.

2.7.5 Hardware Simulation

ECPT EMT's emulator toolchain provides an open platform of hardware simulation for experimental architectures where silicon implementations are not available. To demonstrate this, we run hardware simulations for EMT-Linux with the x86-64 radix and ECPT MMU drivers using the DynamoRIO simulator [103]. We use the hardware configuration described in the ECPT paper [8]. For PWC configuration, we follow more recent works [7], [39] to use 3 levels of PWC with 2-4-32 entries per level. We use the macro benchmarks and the three application workloads described in §2.7.1.

Traditional Hardware Metrics. Our toolchain measures traditional hardware metrics, namely Page Table Walk Latency and Instructions Per Cycle (IPC), which are considered key performance metrics of hardware translation architectures (see [8], [11]). As shown in Figures 18a and 18b, ECPT has significant performance advantages over x86-64 (Radix). On average, ECPT speeds up page table walks by 23.1%, and increases IPC by 7.0%. For most benchmarks, ECPT shows nontrivial page table walk and IPC speedups, as it directly accesses the last-level PTEs. The only exception is Sysbench whose workload has a high PWC hit rate of 99.8% at the PMD level on x86-64, i.e., the x86-64 page table walks skip most intermediate steps and directly accesses last-level PTEs; meanwhile, ECPT pays extra cycles for hash computation in addition to CWC accesses, so it slightly slows down Sysbench's page table walk by 2.2%

New Metrics with OS Overhead. We find that IPC, which measures instruction throughput, does not fully reflect application runtime—though ECPT increases IPC, our ECPT MMU driver introduces more kernel instructions than the x86-64 MMU driver (§2.7.4). Hence, we measure the total cycles for running macro benchmarks and



(a) Page table walk latency

(b) IPC

(c) Total cycles

(d) Total cycles (running phase)

Figure 18: **ECPT hardware simulation results— Page Table Walk Latency, Instruction Per Cycle (IPC) and Total Cycles—with the ECPT system and the x86-64 system, running EMT-Linux.** All results are normalized to those of the x86-64 system as the baselines.

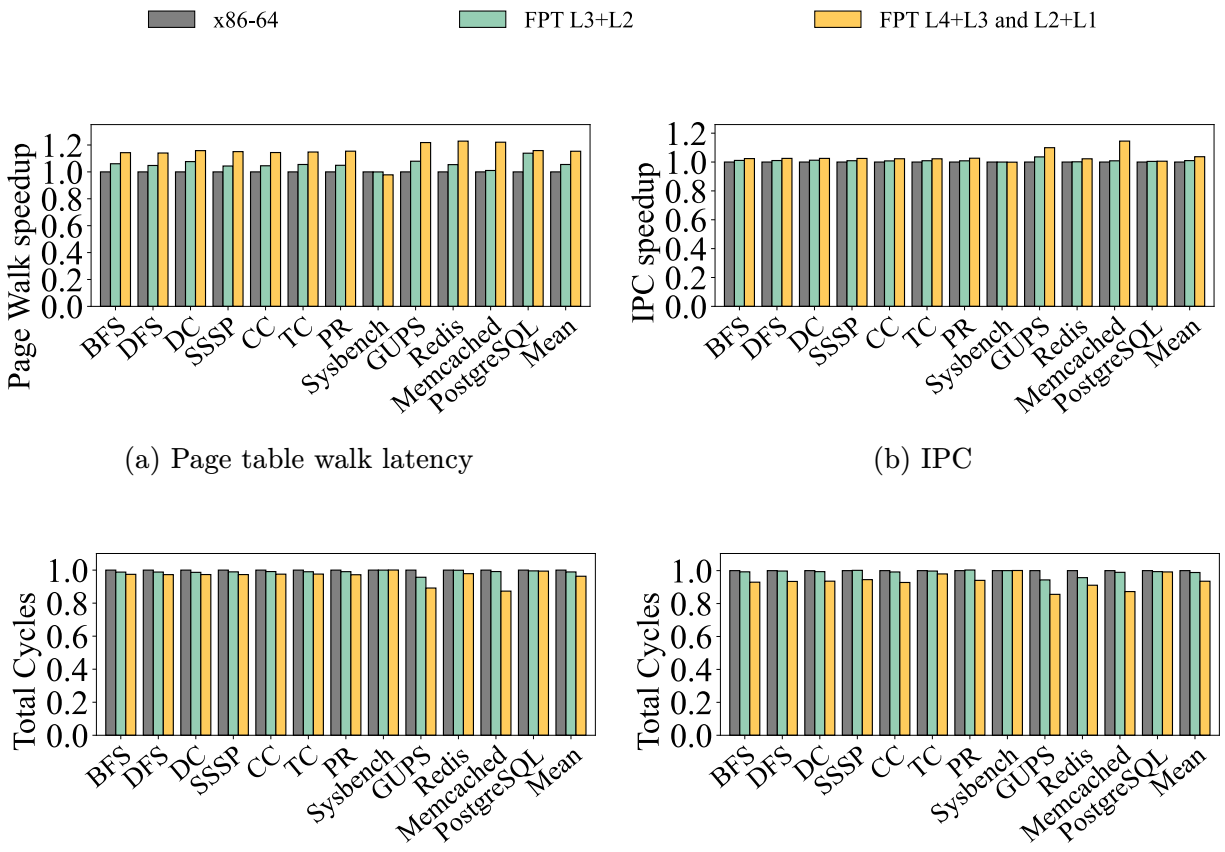
applications, including the OS overhead (mostly page-fault handling as shown in Figure 16). Figure 18c shows that, on average, the ECPT system reduces total cycles by 2.3% across the workloads. GUPS and Memcached show major benefits, where the ECPT system reduces the total cycles by 11.5% and 12.9%, because page table walks account for over 66% of their execution time.

The aforementioned analysis shows that the OS can play an important role in application performance, which could be overlooked with hardware metrics. Without building the OS memory management, it is hard to evaluate OS overhead accurately. Prior work assumed that OS overhead is constant under different memory-translation

architectures. In the initial modeling, the memory trace is collected from running benchmarks on vanilla Linux on x86-64; the trace is then replayed in a simulator of ECPT MMU. As shown in §2.7.4, memory translation architectures can have profound implications for OS performance. EMT aims to enable OS experience for new translation architectures. We believe that EMT effectively lowers the barrier to implement OS support for new memory-translation architectures by making the kernel developers focus on MMU driver implementations.

IPC as a metric aligns with application's execution time when OS overhead is minimal. All the evaluated workloads have two phases: a *loading phase* that loads data from the disk to memory, and a *running phase* that runs the computation or serves user requests. The running phase involves less kernel work with kernel instructions accounting for less than 1.3% of the total instructions. With only the running phase considered, ECPT improves IPC by 7.5% and reduces total cycles by 6.6% compared to the x86-64 system (Figure 18d).

The results could be different with different hardware simulators. We expect the impact of kernel instructions to become smaller when using SST [87] which models OoO execution more precisely. The SST evaluation remains future work.



(a) Page table walk latency

(b) IPC

(c) Total cycles

(d) Total cycles (running phase)

Figure 19: **Hardware simulation results of the FPT system and the x86-64 system, running EMT-Linux.** All results are normalized to those of the x86-64 system as the baselines.

FPT We start with a FPT configuration that flattens L3+L2 tables (see Figure 1) which was implemented by the original OS prototype [11]. Our evaluation shows that the benefit of flattening L3+L2 is limited: only L3 entries were saved, which were cached effectively in x86-64. As shown in Figures 19a, 19b, 19c, compared with Linux on x86-64 (baseline), on average, the FPT-based system speeds up page table walk latency by 5.5%, IPC by 1.0%, and the total cycles by 1.1%.

We then implemented the FPT configuration that flattens both L4+L3 and L2+L1. This configuration was not implemented in [11] due to its difficulties of supporting 2MB huge pages. We implemented a 4KB-only version and evaluated it with only 4KB base

pages (no huge page). With this configuration, compared with Linux on x86-64, the FPT-based system speeds up page table walk latency by 15.3%, IPC by 3.6%, and total cycles by 3.7% (Figures 19a, 19b, and 19c).

Flattening both L4+L3 and L2+L1 in FPT yields performance comparable to ECPT in the running phase. Compared with radix, ECPT and FPT reduce total cycles by 6.6% and 6.4%, respectively (Figures 18d and 19d). For hardware metrics (page table walk latency and IPC), the FPT-based system matched ECPT-based system's performance: ECPT is only faster than FPT by 2.4% in page table walk latency and 0.4% in IPC. On the other hand, ECPT inherently supports huge pages of varying sizes, unlike this FPT configuration.

2.8 Experience and Lessons Learned

Developing EMT-Linux and the MMU drivers together with the emulated MMU took extensive engineering efforts. Specifically, we had to develop interacting moving parts across the hardware-software boundary.

A key principle is to enable incremental, contiguous engineering practice. For example, when implementing ECPT, our first milestone is to write a basic per-process hashed page table (BHPT) using EMT, without features like collision resolution, elastic resizing, CWT/CWC, etc. We implement BHPT logic in both EMT-Linux and the MMU and have a running system that only supports a tiny microbenchmark. Despite being basic, the running system serves as a foundation for gradually adding features and eventually evolving into a full-fledged system. For features that need both hardware and software support, we start from the hardware, which typically has simpler logic and mostly reads translation data.

We continuously test and evaluate our system, not only for correctness (§2.7.2) but also for performance. Continuous profiling helped us quickly observe performance regression over vanilla Linux; when developing ECPT, profiling helped us identify major

inefficiencies compared to the radix-page-table-based system. We developed the instruction analysis with flame graphs [104] in our emulator toolchain. and use them extensively. We also added GDB support for the ECPT-based system (QEMU's GDB was hardcoded with radix).

We largely reuse the existing Linux compiler toolchain to keep the effort manageable. We leverage x86's model-specific registers (MSR) [24] as the control registers for ECPT, which is supported by GCC (x86's read/write MSR instruction allows access to arbitrary MSR with a 32-bit identifier). We use Clang's static analysis tools (e.g., clang-query [105]) to search for error-prone code patterns during refactoring.

One mistake we made was to start from Linux's boot-time kernel page table [106], [107], which is legacy code without debugging tools. Developing boot-time kECPT requires implementation in assembly and build mappings of kernel code/data from a bootstrap address space. We spent four months to understand the details of the boot-time kernel page table. Retrospectively, we should have started from runtime kECPTs (by switching the system to it after boot [108]) so we could have a running system quicker to parallelize our efforts.

2.9 Related Work

As memory translation has become a major bottleneck of emerging memory-hungry, irregular workloads such as generative models, graph analytics, and recommendation systems, translation architecture has been an immensely active research topic recently. Extensive efforts are made on new designs of TLB [109], [110], [111], [112] and page walk caches [44], [45], and more efficient huge page management [6], [46], [113], [114], [115].

To fundamentally resolve translation bottlenecks, recent work [5], [8], [9], [13], [31] is actively rethinking translation architectures; a common thread is to prioritize speed over space efficiency (with abundant memory) and use fast lookup structures to reduce tail latency (avoiding pointer chasing).

Memory translation architectures have profound implications on OS reliability and performance, but unfortunately have not been well explored in prior work. EMT is designed to enable development and evaluation of OS support for emerging translation architectures to formulate an OS perspective.

Note that memory management and its extensibility were well studied in OS research [56], [68], [116], [117], [118] and are continuously revisited, driven by the growth of memory capacity and the heterogeneity introduced by memory tiering [22], [65], [66], [67], [119], [120]. So far, extensibility refers to the ability for user space to customize memory management features, e.g., page fault handler. Few prior studies considered extensibility in terms of empowering memory translation architectures or studied their implications on OS performance.

We are inspired by machine-independent memory management designs in Mach [56] and SunOS [69]. The goal of EMT is to enable the OS development for emerging, experimental memory translation architectures and demonstrate the value with our experience. We attempted to simplify the EMT interface, e.g., to a map interface [56], [69]; however, we find it hard to balance architecture neutrality and low-level optimizations. One way is to add hints, but hints are not expressive for passing information to different MMU drivers.

Virtuoso [121] is a recent hardware simulation framework that considers OS overhead (mostly the overhead of page fault handling) using an imitated MimicOS in the user space. EMT and Virtuoso are fundamentally different but are complementary. Unlike Virtuoso which is designed for architecture-centric simulations, EMT is an OS interface for enabling OS developement and evaluation with regard to new translation architectures. EMT benefits architecture designs from an OS perspective by running real MMU drivers on Linux.

2.10 Concluding Remarks

EMT is an OS framework for developing and evaluating OS memory management for new memory translation architectures. Our work shows the importance of understanding hardware translation schemes from the OS perspective. Specifically, we show that fast translation schemes like ECPT can incur new challenges for the OS. We will release and make EMT and the emulator toolchains as an open platform and encourage new hardware architectures to experiment with modern OSes. With the significant diversity of emerging workloads and increasing heterogeneity of interconnected memory devices, it becomes harder to foresee a one-size-fits-all translation scheme. Hence, OS extensibility for different translation schemes is critical to enable specialized translation. We hope that EMT design starts a practical journey towards extensible OS kernels for translation of heterogeneous devices.

Chapter 3 Virtually Free: Making Strict IO Memory Protection Feasible for Virtualized Environments

Input/Output (IO) memory protection prevents buggy or malicious IO devices from executing errant data transfers into host memory. IO memory protection in modern servers is achieved using a combination of software and hardware mechanisms. The operating system (OS) assigns direct memory access (DMA) capable IO devices (*e.g.*, network interface cards and storage devices) with IO virtual addresses (IOVAs). IO devices initiate DMAs using these IOVAs; for each DMA, a host hardware—IO memory management unit (IOMMU)—uses an IO page table (IOPT) to translate IOVAs to host physical addresses that are used to execute the DMA; the address translation is sped up using special IOMMU caches. To achieve powerful (aka strict [122], [123]) IO memory protection, IOPT entries must be unmapped and IOMMU caches must be invalidated immediately after each DMA. Recent studies from production clusters have established that achieving strict IO memory protection, even with state-of-the-art mechanisms, has significant impact on application-level performance [124]. For the bare metal case, a now-long line of work [123], [125], [126], [127], [128], [129], [130], [131] has led to mechanisms that enable achieving strict IO memory protection with minimal overheads.

In virtualized environments, the above IOMMU-based design is sufficient to achieve strict IO memory protection *across* virtual machines (VMs): by isolating each VM's memory, the IOMMU-based design ensures that a device (or a virtual function) can only access memory regions explicitly authorized for the VM. Enabling strict intra-VM IO memory protection—fine-grained protection across devices and virtual functions *within a VM* [122], [131], [132]—is much harder. The state-of-the-art for (intra-VM) IO memory protection is the celebrated work from Amit et. al. on vIOMMU from over a decade ago [122]. vIOMMU exposes an emulated IOMMU to VMs; each VM operates on its emulated IOMMU, and the hypervisor intercepts and executes privileged operations to the physical IOMMU. *To maintain strict IO memory protection, this triggers at least one*

unavoidable VM exit. The vIOMMU evaluation establishes the hardness of simultaneously achieving strict IO memory protection and high performance: even with a variety of intriguing techniques, enabling strict IO memory protection with vIOMMU reduces application-level throughput to merely 3Gbps (30% of the 10Gbps network bandwidth at the time). This severely limits the feasibility of strict IO memory protection for IO-intensive applications: most of the host resources would be idle 70% of the time.

The server software and hardware ecosystem has evolved significantly over the past decade since the vIOMMU work. Advances in device passthrough [133], [134], [135], [136], [137] and IO memory management [130] mechanisms have been widely incorporated within modern software ecosystem, opening up the potential to reduce the overheads of IO memory protection. The number of cores have increased by 8–32×, enabling more resources to enable IO memory protection; and, advances in IOMMU hardware (*e.g.*, IOPT caches [123]) along with advances in processor architectures (*e.g.*, support for nested translations [133]) have the potential to further reduce IO memory protection overheads. Unfortunately, these software and hardware advances have not kept up with the 40× increase in server IO bandwidth over the past decade; for instance, we observe that enabling strict intra-VM IO memory protection—even with state-of-the-art device passthrough mechanisms, IO memory management, IOMMU hardware, and nested translation—results in as much as 83-95% degradation in application-level throughput on our testbed with 400Gbps network bandwidth! Thus, while exploiting IO transfers at memory access latency timescales (a page worth of data transfer at 400Gbps of IO bandwidth takes 80 nanoseconds) opens up exciting opportunities at all layers of the systems stack, such high IO bandwidths make it even more challenging to offer strict IO memory protection. Unsurprisingly, real-world deployments are forced to make the undesirable tradeoff: high performance or weaker (or even no) IO memory protection. This state of affairs is sad.

We present vFree, a simple modification to existing IO memory protection

mechanisms that makes strict IO memory protection feasible in virtualized environments. The key driving principle in vFree may be counter-intuitive: we demonstrate that, in virtualized environments, it is possible to minimize the overheads of strict IO memory protection by *increasing* the cost of IOVA-to-physical address translation. This is in sharp contrast to the bare metal context: recent work on F&S [123] achieves near-zero cost for strict IO memory protection in bare metal context by explicitly reducing the cost of address translation. This dichotomy is obvious in the hindsight: in the bare metal case, address translation can take as much as four sequential memory accesses during the IOPT walk (4× the data transfer time for a page worth of data with 400Gbps IO bandwidth) and is thus the primary performance bottleneck; in virtualized environments, the unavoidable VM exit becomes the primary performance bottleneck often taking 17-42× longer than the worst-case address translation time. vFree reduces the number of VM exits by explicitly increasing the cost of address translation.

vFree realizes the above principle using the key idea of *one-for-many invalidations*. Recall that, to maintain strict IO memory protection, two actions must be performed upon each DMA: (1) unmapping the guest IOPT entries (which can be done without hypervisor intervention); and (2) synchronously invalidating IOMMU cache entries corresponding to the IOVA used in that DMA. Each invalidation request invalidates exactly one entry in the IOMMU primary cache (referred to as IO translation lookaside buffer, or IOTLB) and may additionally invalidate one entry corresponding to each level of the IOPT other IOMMU caches (referred to as IO page walk caches, or IOPWC) [123]. This is to ensure that IOMMU cache entries for all other IOVAs are not impacted and can be used to speed up address translations for all other IOVAs. The one-for-many invalidations in vFree design, as we discuss below, forces all IOMMU caches (including both IOTLB and IOPWC) for a VM to be invalidated for each invalidation request; thus, the state of the caches after one invalidation request will be "worse" than what it would be if multiple invalidation requests would have been executed—subsequent address

translations may now require a full IOPT walk increasing translation overheads. However, the one-for-many invalidations in vFree design will opportunistically execute one invalidation request for many outstanding invalidation requests, resulting in much fewer invalidation requests (and thus, much fewer VM exits) than existing IO memory protection mechanisms.

Specifically, in existing systems, a guest kernel thread executes the above two operations synchronously: it unmaps IOPT entries, prepares an invalidation request, submits the invalidation request to the emulated IOMMU invalidation queue, and rings a "doorbell"—it writes to a certain register, which traps to the hypervisor; the hypervisor then executes the invalidation request on physical IOMMU and sends a completion signal to the guest. During this process, the guest kernel thread that initiated this invalidation request—and importantly, all other vCPUs that need to execute their own unmap and invalidation requests—must block on the completion signal. vFree enables a fast path[4] where all vCPUs within a VM can execute unmapping of their respective IOPT entries, prepare their respective invalidation requests, and then block (without submitting their invalidation requests). As a result, across vCPUs, we can now have multiple outstanding invalidation requests—as many as the number of vCPUs for the interesting case of high network load. A dedicated vCPU collects all invalidation requests from all vCPUs and submits *one of the invalidation requests along with a certain flag described below*, rings the doorbell, and then polls on the completion signal (and repeats the process upon receiving the completion signal). The flag indicates to the hypervisor that all IOMMU caches corresponding to the VM must be flushed during the invalidation process (rather than simply invalidating the address translation entry in the invalidation request). Thus, one-for-many invalidations executes only one invalidation request for all outstanding invalidation requests, opportunistically reducing the number of VM exits.

[4]The slow path still requires all vCPUs to block, *e.g.*, for the extremely rare case of devices being concurrently attached or detached from a so-called IO domain, a protection context that groups one or more IO devices sharing an IO virtual address space.; we discuss details in §3.3.

vFree incorporates a second idea of *deferred free page* that further reinforces one-for-many invalidations while maintaining strict protection. While one-for-many invalidations enable multiple vCPUs in the VM to concurrently create invalidation requests in the fast path, these vCPUs must still block on the completion signal after their invalidation request has been submitted. This is because, for strict IO memory protection, the IOVA and the physical page must be released for future use only after the invalidation has completed [122], [127], [129], [130], [131]. To that end, the deferred free page idea ensures that free IOVA and free page operations *for all outstanding invalidation requests* are executed only upon receiving the completion signal. To achieve this, vFree introduces a new callback-based interface in the IOMMU subsystem, allowing IO device drivers to pass a callback function that delays physical memory cleanup until invalidation completion signal is received. With deferred free page, the vCPUs can proceed immediately without waiting for the invalidation to complete while still preserving strict IO memory protection. This results in even more outstanding requests available to one-for-many invalidations, further reducing the number of VM exits and further reducing the overheads, while maintaining strict IO memory protection.

We implement vFree on Linux v6.12.9 with ∼1.5K lines of C code (350 in the device driver), and evaluate it on three widely used real-world applications—NGINX (a web server), Redis (an in-memory key-value store), and SPDK (a userspace storage stack that uses Linux TCP for remote storage). Our evaluation results show that vFree consistently achieves performance within 1–8% of the unprotected system—74–147× higher than state-of-the-art IO memory protection mechanisms. The overheads of achieving these benefits is one dedicated core; vFree thus expands the choices of real-world systems: high performance but with weak (or no) IO memory protection, abysmal performance with underutilized cores with strict IO memory protection, or, high performance with one dedicated core with strict IO memory protection.

3.1 Background: DMA Datapath in Virtualized Environments

This section gives an overview of the DMA datapath in virtualized environments with strict IO memory protection. Throughout the paper, we consider device passthrough which grants the guest direct access to the physical NIC—the guest can directly post IO descriptors to the NIC without host intervention. For brevity, we focus on the Linux network stack, Intel Emerald Rapids CPUs, and Nvidia CX-7 NICs.

IO memory protection is realized by providing NICs with IO virtual addresses (IOVAs). To execute a data transfer, the host hardware translates IOVAs to corresponding physical addresses using an IOMMU and an IO page table (IOPT) that stores IOVA-to-physical address mappings. The translation is sped up using hardware caches, including an IOTLB and IOPWCs for the final and intermediate translations [123].

Figure 20 illustrates the lifetime of a DMA operation under IO memory protection, breaking down the datapath in three phases: (1) Pre-DMA phase, involving creation of address mappings and supplying IOVAs to NIC; (2) DMA phase, involving performing address translation and DMA transfer; and (3) Post-DMA phase, involving destruction of address mappings and invalidating address translation caches in hardware. We describe these three phases in detail below.

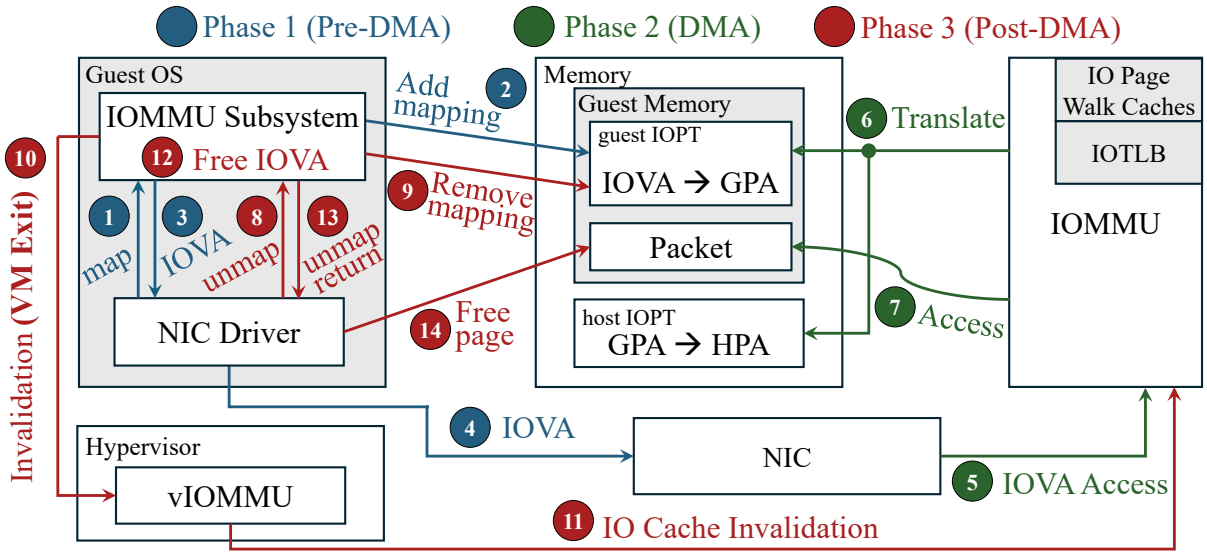


Figure 20: **Illustration of NIC-to-memory DMA datapath at the transmitter when memory protection is enabled.**

❶ **IOVA allocation and IOMMU mapping.** The guest uses the `map()` API to create an IOVA-to-GPA (guest physical address) mapping in the guest-local IOPT. The guest then creates a packet descriptor containing the IOVA, and passes the descriptor to the NIC.

With device passthrough, the NIC performs data transfers without involving the host. This is possible with IOMMUs in recent Intel CPU architectures (Sapphire Rapids and newer) using nested translation [133] that enables the IOMMU to translate IOVAs directly to corresponding host physical addresses (HPAs) rather than GPAs, without host intervention.[5]

❷ **DMA and address translation.** The NIC issues DMA requests using the IOVA in

[5]We focus on nested translation supported by modern server hardware. An alternative is to *shadow IOPT*, where the hypervisor maintains an IOPT that maps each IOVA directly to the HPA. Shadow IOPT requires every new mapping to trigger a VM exit for the hypervisor to update the shadow IOPT. Nested translation eliminates such VM exits. In our measurement, nested translation achieves strictly higher throughput than shadow IOPT, by 1.2× to 2.6× across core counts. While we focus on nested translations, some of our techniques may also be useful for shadow IOPT.

packet descriptors, which triggers IOVA-to-HPA address translation using the IOMMU; the final or intermediate translation could be cached in IOMMU's IOTLB and IO PWCs [123]. Upon successful translation, the DMA is executed to the corresponding HPA.

❸ **Completion and IOMMU cache invalidation.** To maintain strict IO memory protection, NIC's access to the target page(s) in the host memory must be revoked immediately after the DMA completes [122], [123]. This happens using two steps. First, the guest uses the `unmap()` API which removes the corresponding IOVA-to-GPA mapping from the guest IOPT.[6] Second, mappings in IOMMU caches are also invalidated—typically, these invalidations are executed synchronously within the `unmap()` call. The IOMMU provides an invalidation queue as an interface for the OS to submit *invalidation requests (IRs)*. This happens in two steps (see Figure 21).

1. **Preparing the IR descriptor (IR-PREP).** The guest kernel thread first acquires a per-IO-domain spinlock (termed `cache_lock` [138] in Linux). Upon acquiring the lock, the thread prepares and enqueues an IR descriptor in a per-IO-domain submission queue—referred to as *IO-domain queue* in the guest. Note that there can be multiple IO-domain queues in the guest (*e.g.*, for different devices).

2. **Submitting the IR to IOMMU hardware (IR-SUB):** To submit the IR descriptors in an IO-domain queue to the IOMMU hardware, the hypervisor, specifically the vIOMMU subsystem, exposes an *emulated IOMMU invalidation queue* to the guest. Before submitting the IR to the emulated invalidation queue, the guest needs to acquire another lock—a per-invalidation-queue spinlock (`q_lock` [139] in Linux). While holding `q_lock`, the kernel writes the IR descriptor

[6]GPA-to-HPA mapping are managed by the host. In KVM/QEMU, these mappings are established at VM creation time; for VFIO passthrough devices, guest memory is pinned, so GPA-to-HPA mappings remain static.

into the queue, then advances a tail register—via an MMIO write—to signal the hardware to process the IR.

Unavoidable VM exit for *every* IR. Submitting IRs from the guest to the IOMMU hardware necessarily requires hypervisor intervention. This is because the IOMMU invalidation queue—the interface provided to the OS for IOMMU cache invalidation—is a shared host resource with no per-VM partitioning [140]; allowing multiple guests to write concurrently would risk corruption. Thus, the MMIO write to the tail register triggers a VM exit, transferring control to the hypervisor. The hypervisor validates the IR, issues the corresponding IR to the physical IOMMU (by copying the IR to the IOMMU invalidation queue in the host), and returns control to the guest. The guest thread, in the meantime, busy-waits on the status field until the hardware signals that the invalidation is complete. So, the VM exit is performed for *every* IR, because an IR descriptor is written to the emulated queue protected by `cache_lock` (`q_lock` is acquired *inside* `cache_lock`).

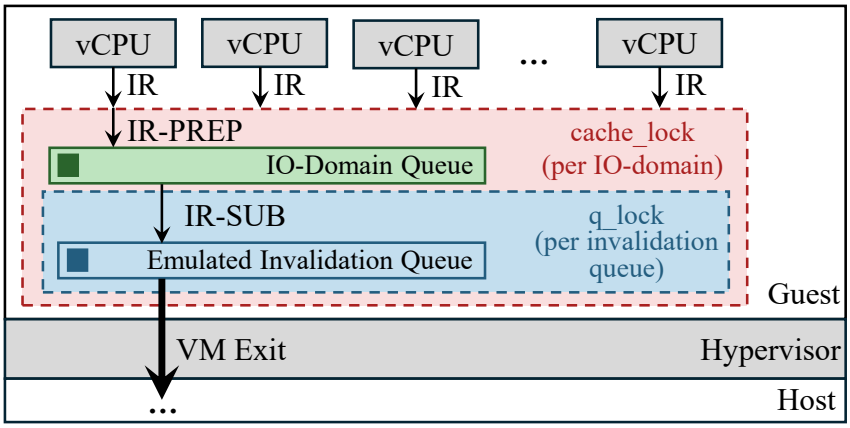


Figure 21: **IR-PREP and IR-SUB operations in the Post-DMA phase and the corresponding queues.**

Finally, upon successful IR processing, the IOVA is returned to the IOVA allocator and the corresponding physical page(s) used for DMA are made available for recycling. Both `cache_lock` and `q_lock` are released when IR-SUB finishes.

Optimizations for Rx Datapath in Linux. Recent versions of Linux network stack (>=6.11 [141]) implement an optimization in the Rx datapath to reduce `map()` and `unmap()` operations.[7] It uses a per-CPU cache to maintain a set of IOVAs permanently mapped to the IOMMU [130]. In the Rx datapath, the kernel uses the physical pages mapped to these IOVAs to allocated kernel-space buffers (*e.g.*, TCP socket buffers), used to DMA data from the NIC. Upon DMA, the data is copied from kernel-space buffers to the userspace buffers, before returning back the IOVA to the cache. Malicious NIC with access to the IOVA cannot access the userspace buffers (which are not mapped to the IOMMU). Using (and recycling) these IOVAs therefore do not require `map()`/`unmap()` calls. This optimization is helpful for workloads for which actively used IOVAs are consistently smaller than the cache size [123]. Upon exhausting this cache, whenever the kernel requires another IOVA, it uses the default IOVA allocation/deallocation datapath, using regular `map()`/`unmap()` operations.

3.2 Understanding Protection Overhead

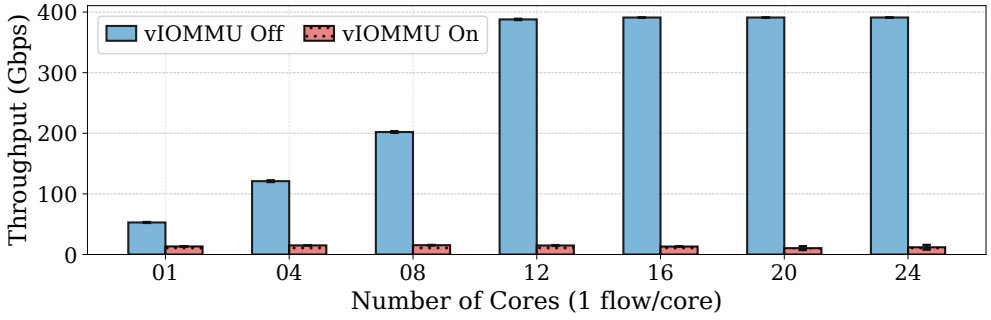
This section builds an in-depth understanding of inefficiencies in existing strict intra-VM IO memory protection mechanisms. We present more results, including those for real-world applications in §3.5.

Measurement Setup. We use two servers directly connected to each other via 400Gbps Nvidia ConnectX-7 NICs to ensure performance bottlenecks not in the network. The servers use Emerald Rapids processor architecture that supports nested translation, with Intel Xeon Platinum 8592V CPUs and Intel Xeon Gold 6538Y+ CPUs respectively. The former hosts our VMs, and the latter acts as client to send or receive traffic to the VM. The host server, guest VM, and client machines run with Ubuntu 22.04 and Linux v6.12.9. The VMs are hosted by QEMU v10.0 with nested translation [142]. We place

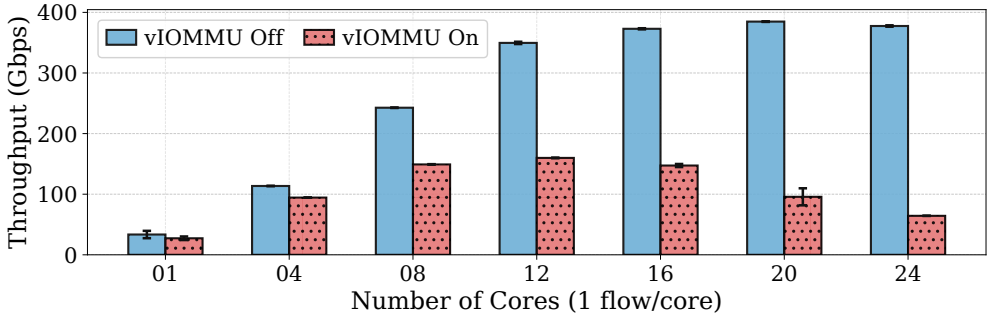
[7] For several pragmatic reasons, existing operating systems do not apply similar optimizations on the Tx datapath. Hence, every transmitted packet incurs at least one VM exit per `unmap()` call.

the VMs on the NIC-local NUMA node. Every vCPU of the VM is pinned to a separate physical core on the host to avoid interference. See §3.5 for more details.

We use Iperf3 [143] to generate network traffic with 4K MTU size, and measure throughput. We run each experiment five times, and present the average results and standard deviation. We use two setups to isolate the impact of IO memory protection on Rx and Tx datapaths. In the **Rx-server** setup, the client sends large payloads to the VM; the VM receives these payloads and transmits TCP ACKs back to the client. In the **Tx-server** setup, the VM sends large payloads to the client and receives TCP ACKs from the client.



(a) Tx-server (VM running as a transmitter)



(b) Rx-server (VM running as a receiver)

Figure 22: **State-of-the-art IO memory protection mechanisms suffer from significant performance degradation compared to the unprotected mode, and exhibit anti-scaling behavior with increasing cores.** Performance degradation is much more profound on the Tx-server setup.

[Figure 22] Performance degradation with IO memory protection in VMs.

Figure 22a shows throughput for the Tx-server. Without IO memory protection, 12 cores

are sufficient to saturate the network bandwidth; thus, the achievable throughput increases until 12 cores and plateaus thereafter. With IO memory protection enabled, the maximum achievable throughput, irrespective of number of cores used, is ∼15 Gbps—a ∼95+% degradation compared to no protection.

Figure 22b shows throughput for the Rx-server. Enabling IO memory protection again results in significant degradation in throughput: we observe a peak throughput of 160Gbps with 12 cores—∼54% degradation compared to the no protection case. Upon increasing the number of cores beyond 12, the throughput, rather surprisingly, starts to drop; we observed a maximum degradation of up to ∼83% when using 24 cores. Compared with the Tx-server case, the relatively higher Rx-server throughput is due to the page pool optimization that reduces map/unmap operations (§3.1); and TCP ACK coalescing, which increases the number of pages per `unmap` (by as much as ∼15×).

[**Figure 23**] **Root cause of performance degradation with IO memory protection: at least one VM exit per unmap/invalidation**. IO memory protection introduces two extra operations on the datapath: `map()` and `unmap()`. With nested translation, `unmap` is far more costly than `map`: besides updating IOPTs, `unmap` also performs IOMMU cache invalidations which require VM exits (§3.1). We observe that overheads of unmap operations are the primary contributor to the performance degradation due to IO memory protection.

We observe that processing IRs within the `unmap` operation accounts for over 95% of the total `unmap()` latency on average, across all vCPU counts in both the Rx-server and the Tx-server setup (not shown; measured via eBPF probes on the unmap path). Thus, we dug deeper into IR processing overheads. Figure 23 shows the average IR processing latency per `unmap` and its breakdown across different operations involved in IR-SUB datapath (§3.1). We observe that, as the number of cores increases, IR processing latencies increase. This increase in IR processing latency is rooted in two key characteristics of the existing IR processing path (phase ❸ in §3.1): (1) the vCPU

performing IR-PREP must acquire a per-IO-domain `cache_lock` *before* preparing the IR; and (2) IR-SUB is executed *inside* the critical section of the same `cache_lock`. Since IR-PREP and IR-SUB are tightly coupled under one lock, every invalidation request triggers its own VM exit (Figure 24). As a result, all concurrent vCPUs serialize on `cache_lock` for every `unmap`.

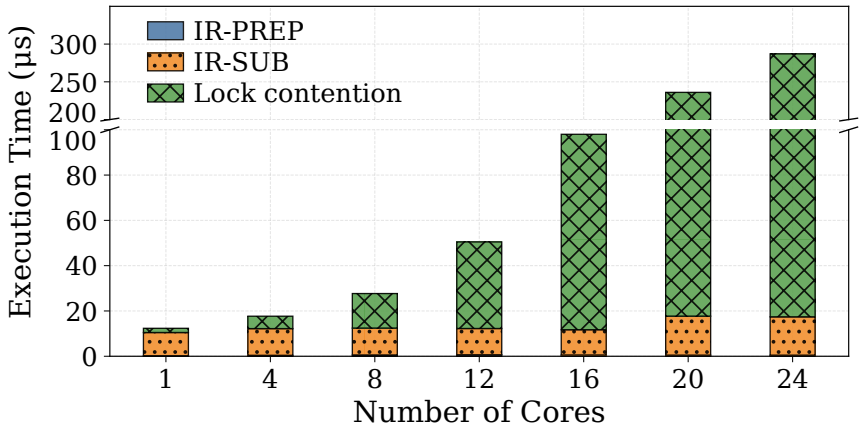


Figure 23: **Breakdown of IO memory protection overheads.** IR-PREP contributes minimally, independent of number of cores. For fewer than 8 cores, IR-SUB cost dominates; for 8 and more cores, lock contention overheads dominate. Moreover, with increasing number of cores, lock contention time increases superlinearly while the IR-SUB cost remains nearly constant.

Figure 23 shows the consequence: as the number of vCPUs increases, the time spent waiting to acquire `cache_lock` grows, while the time for IR-PREP and IR-SUB themselves remains nearly constant. In other words, the actual work per invalidation grows only modestly with scale—the dominant increase is wasted time spent serializing behind other vCPUs. Each vCPU blocks on its `unmap` call for the duration of this wait, so per-vCPU throughput drops as contention increases.

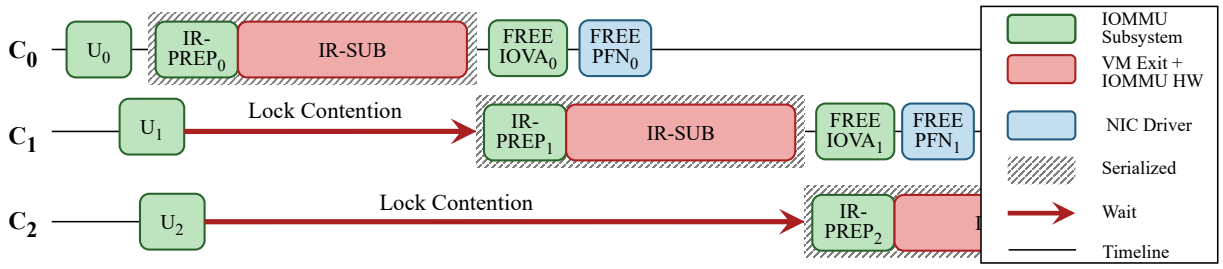


Figure 24: **Illustration of strict serialization of each IR-PREP and IR-SUB in existing strict IO memory protection mechanisms across CPU cores ($C_{0,1,2}$).**

[Figure 23] Root cause of throughput reduction at large core counts: contention-descheduling feedback loop in virtualized environments. We now explain the root causes behind the counter-intuitive observation of throughput reducing with number of cores increasing beyond 12.

We observe that two kinds of VM exits dominate. First, VM exits due to MMIO writes to the IOMMU invalidation tail register—"useful work" of the `unmap` path. Second, VM exits due to contention-induced vCPU descheduling (triggered by `HLT` event). In virtualized environments, an `HLT` halts a vCPU and triggers a VM exit that causes KVM to deschedule the vCPU till an interrupt or wake event arrives.

We observe that, at low core counts, nearly all per-`unmap` VM exit time (∼94%) is spent in handling writes to the MMIO invalidation queue tail register and the total per-`unmap` exit cost is ∼11–18 μs (IR-SUB cost in Figure 23). At 16 cores and above, contention-induced descheduling consumes 93–97% of per-`unmap` exit time, inflating the total cost to 141 μs at 16 cores and *514 μs* at 24 cores—a 46× increase over the low-contention baseline. The reason is as follows.

Contention-descheduling feedback loop. The transition from useful work to idle waiting follows a three-stage cascade process visible in the VM exit data:

1. *Spinning.* As vCPU count grows, more vCPUs contend for `cache_lock`. Waiters

spin in tight `PAUSE` loops, waiting for the lock holder to complete its invalidation. We find that, from 12 to 16 cores, `PAUSE`-related VM exits increase by 60×, triggered by the processor's Pause-Loop Exiting (PLE) mechanism, which detects sustained spinning and traps to the hypervisor once a threshold is exceeded [144].

2. *Descheduling.* Once a vCPU has exceeded its spin budget, an `HLT` event is triggered, which leads to another VM exit that causes the host (KVM) deschedules the vCPU. We confirmed that these `HLT`-related VM exits are lock-induced halts, and not due to idle vCPUs. The same phenomenon is observed for general kernel spinlocks: at a high contention level, most vCPUs simultaneously fail their optimistic spin and trap to the hypervisor [145].

3. *Expensive wakeup.* When the lock holder finishes and releases `cache_lock`, the next waiter must be woken via an IPI. This wakeup path, which is cheap on bare metal (∼8 μs), is an expensive event in virtualized environments, known as the *Blocked-Waiter Wakeup* (BWW) problem [146]: the IPI triggers a VM exit on the vCPU that triggers the wakeup event, invokes the KVM scheduler, and requires a VM entry to reschedule the target vCPU. The latency for the wakeup path is included in the `HLT` duration. In our measurements, average `HLT` duration per unmap grows from 2.5μs at 12 cores to 131 μs at 16 cores to 346μs at 20 cores to 498μs at 24 cores, far exceeding a single BWW event because the cost compounds: as more vCPUs are descheduled simultaneously, each must wait longer to be rescheduled, and the vCPU responsible for sending wakeup IPIs may be contending for the same lock.

This feedback loop results in *super-linear* increase in `HLT`-related VM exit time: for the core on which we profile, ∼157K invalidations are completed for 12 cores but only ∼35K invalidations are completed for 24 cores, a 4.5× reduction with merely 2× increase in number of cores.

This feedback loop is specific to virtualized environments. On bare metal, a spinning thread remains on its physical core and resumes immediately when the lock is released; the virtualization layer converts what would be a brief spin-wait into a cascade of VM exits whose cost compounds with every additional contending vCPU. The individual pathologies—PLE-triggered descheduling [144], mass halt under contention [145], and BWW [146], [147]—are well studied for general kernel spinlocks, but the IOMMU invalidation path is a uniquely severe instance: the serialized hardware queue is a fixed point on every packet's `unmap` that cannot be parallelized or avoided.

3.3 vFree Design

vFree is a modification to existing IO memory protection mechanisms for *strict* intra-VM protection in virtualized environments. vFree targets the same threat model as Linux strict mode (Appendix 3.8.1). vFree achieves this goal using *One-for-Many Invalidations* (§3.3.2) and *Deferred Free Page* (§3.3.3).

3.3.1 Enabling Concurrent Invalidation Requests

As discussed in §3.1, existing IO memory protection mechanisms enforce strictly one invalidation requests (IR) in flight: vCPUs acquire the `cache_lock` before executing IR-PREP, synchronously execute IR-SUB, and release the lock only after the IR completion signal is received; during this process, all other vCPUs remain blocked, waiting to prepare and submit their respective IRs. Since each IR requires a VM exit, existing IO memory protection mechanisms enforce at least one VM exit per IR to enable strict IO memory protection.

Fast path for concurrent IRs. vFree enables a fast path within IO memory protection mechanisms that allows keeping multiple in-flight IRs. To achieve this, vFree decouples the two operations involved in IR processing—preparing IR descriptors (IR-PREP) and submitting the descriptors to the hardware (IR-SUB)—across multiple

threads. Specifically, vFree uses a producer-consumer model, as shown in Figure 25. IR-PREP is executed by producer threads—which are the same threads performing unmap operations. Furthermore, vFree replaces the single IO-domain queue and `cache_lock` with per-CPU virtual IO-domain queues, each protected with per-CPU `cache_locks`. This allows multiple producer threads (running concurrently on multiple vCPUs) to concurrently execute IR-PREP: each producer acquires its per-CPU `cache_lock`, prepares an IR, enqueues it in per-CPU virtual IO-domain queue, releases the per-CPU `cache_lock`, and then waits to receive a completion signal for its IR. With this design, multiple IRs can be kept in flight.

The IR-SUB operation is executed by a consumer thread on a designated core. The consumer thread acquires each per-CPU `cache_lock` one at a time, copies pending IR descriptor(s) from the per-CPU IO-domain queue to a temporary buffer, and immediately releases the per-CPU `cache_lock`. When a producer observes that its IR has not completed, it attempts to activate the consumer by atomically test-and-setting a domain-wide BUSY bit. If the test-and-set succeeds, the consumer is inactive; thus, the producer schedules the consumer on the designated core via a workqueue. If the test-and-set fails, the consumer is already running; the producer spins on the completed counter until the consumer finishes. If the producer is running on the designated core (dispatching via the workqueue would deadlock), the producer executes the flush locally, bypassing the workqueue.

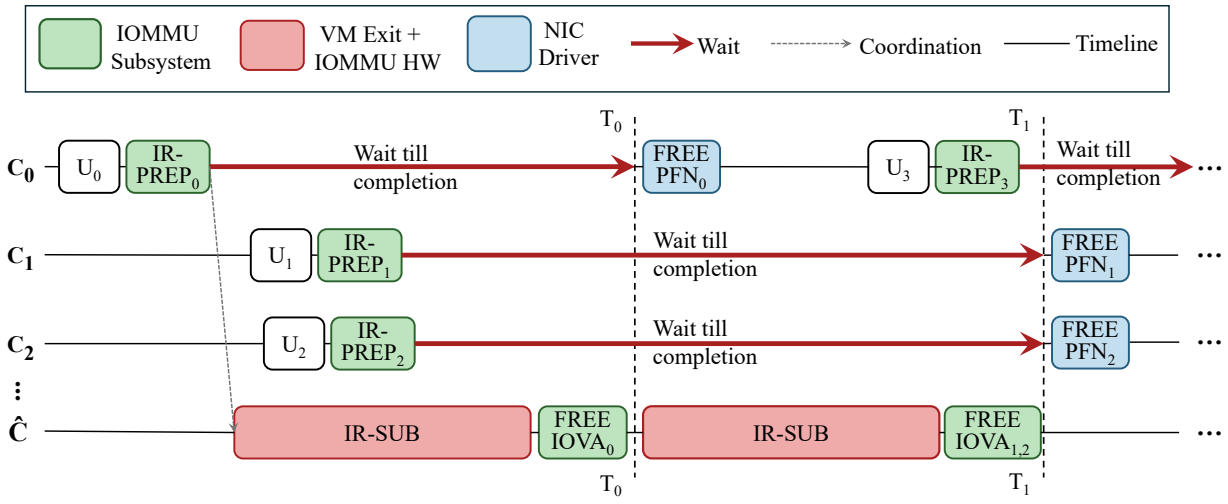


Figure 25: **One-for-many enables multiple IR-PREPs to run concurrently with an IR-SUB operation. It reduces the number of VM exits by using one IR-SUB operation for all outstanding IRs to flush all IOMMU caches.** In this figure, producers run on cores C_1, C_2, C_3, and a dedicated consumer thread runs on $\hat{C}$. Each producer performs an unmap operation (U_{1-3}) and busy-waits until its IR is invalidated by the consumer thread.

Slow path. As discussed in §3.2, IO memory protection mechanisms use `cache_lock` to protect cache tags for each IO-domain, as a domain can be attached/detached at any time, and the list of cache tags is recycled during invalidation. The domain attaching/detaching is a rare event, but must be accounted for to maintain strict IO memory protection. Thus, any domain attaching/detaching event triggers the slow path: *all* per-CPU `cache_locks` have to be acquired at attaching/detaching time to ensure safety.

3.3.2 One-for-Many Invalidations

Decoupling IR-PREP and IR-SUB processing across producer and consumer threads enables vFree to execute them concurrently. Recall that the majority time of IR-SUB is spent for the VM exit, which is executed within the `q_lock`; IR-PREP requires only acquiring the per-CPU `cache_lock`, and can therefore be executed concurrently. Executing multiple IR-PREPs concurrently during an ongoing IR-SUB enables generating multiple IR descriptors by the time this IR-SUB operation finishes. The

one-for-many invalidations technique in vFree exploits this opportunity to efficiently reduce the number of VM exits per unmap operation.

Flushing IOMMU caches to enable one-for-many invalidations. The (emulated) IOMMU invalidation queue supports submitting multiple IR descriptors—up to 256 entries in our setup—before updating the tail register. It is possible to use this interface to submit all outstanding IRs with a single tail register update (and hence a single VM exit); this will invalidate the IOMMU cache entries corresponding to the outstanding IRs. The invalidation queue in existing hardware only supports invalidating a contiguous IOVA range for any enqueued IR descriptor [148]. However, the group of IR descriptors to be invalidated can be arbitrarily discontiguous collectively[8]. Thus, when submitting multiple descriptors together to the invalidation queue, the total time to invalidate IOMMU caches will scale with the number of entries submitted. Specifically, IOTLB invalidation cost is known to be ∼600-700ns [126] Hence, the overhead of performing multiple invalidations, one for each outstanding IR, will at least be a few microseconds.

vFree executes only one IR for all outstanding IRs, but invalidates the *entire* IOVA address space for each IR-SUB operation, independent of the IR group size or IOVA ranges the IR group represents. This is feasible today via a special IR descriptor type that *flushes* all IOMMU cache entries associated with the guest[9]. Flushing the IOMMU caches maintains correctness guarantees—*i.e.*, the IOVAs represented by the group of IRs will be invalidated. Thus, the submitted IR acts as a *one-for-many* invalidations: the single submitted IR results in invalidating the IOMMU cache entries for all outstanding IRs. This reduces the number of VM exits by the same factor as the number of outstanding IRs. However, flushing the IOMMU caches also poses an inherent tradeoff: while VM exits per unmap and invalidation decrease, the address translation cost may increase (see phase 2 in §3.1). This is because after flushing IOMMU caches, subsequent

[8]Existing works [123] have studied reasons for discontiguity in IOVAs allocated even for successive maps.
[9]IOMMUs today provide per-guest isolation in caches by using separate PASIDs for each guest [149]

IOVA-to-HPA lookups may take as many as 4 memory accesses, instead of potentially 1–4 depending on cache hit rates [123]. This is a strictly good tradeoff to make: even in the extreme case where we incur three extra memory accesses for address translation, assuming a pessimistic memory access latency of 100ns, it will cost $\sim 3 \times 100$ns extra overhead. This is much smaller than a VM exit overhead, which can be several μs (10–12μs in our measurements). Moreover, larger sized IR groups being invalidated using a single flush (which will be the case in the high network load case) result in larger gains.

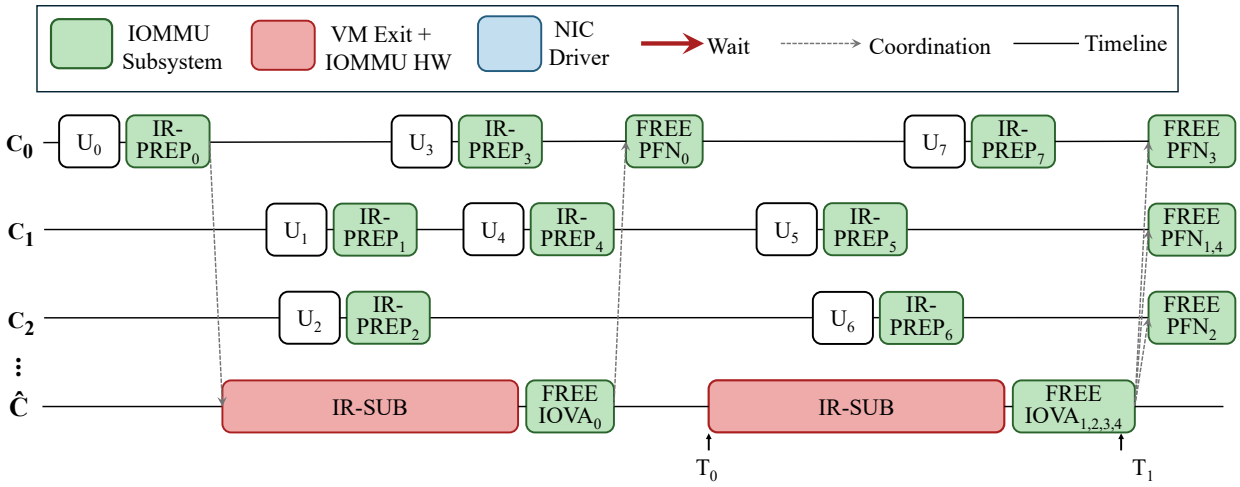


Figure 26: **With deferred free page, vCPUs can proceed immediately without waiting for the invalidation to complete.** In this figure, producers run on cores C_1, C_2, C_3 and dedicated consumer runs on $\hat{C}$. Each producer thread performs unmap (U_{1-3}) and IR-PREP operations and immediately proceed. It enforces strict protection by deferring the free page operation to after the IR-SUB operation completes.

Figure 25 illustrates the one-for-many invalidation design: each producer, upon any unmap, executes IR-PREP. Next, there are two cases. First, if the consumer thread is inactive (such as for $\texttt{IR\_PREP}_0$ in Figure 25), the producer thread schedules the consumer thread on a designated vCPU. The consumer copies IR descriptors from each of the per-CPU virtual IO-domain queues, acquires the `q_lock`, copies one of the IR descriptors with the flush-all flag, and then writes to the MMIO invalidation queues tail register (incurring a VM exit); the consumer then busy waits until it receives the invalidation completion signal; upon receiving the signal, it frees IOVAs contained in the IR

descriptors copied from the per-CPU virtual IO-domain queues. Or, if the consumer thread is active, running IR-SUB for previously prepared IRs (such as $\texttt{IR\_PREP}_1$ in Figure 25), the IR will be submitted by consumer's next round of operations. When any round of consumer operations finishes, the consumer starts executing another round if there are any descriptors present in the per-CPU virtual IO-domain queues. Otherwise, it terminates.

Importantly, each producer thread also *busy waits* until its IOVA has been successfully invalidated and freed by the consumer. For instance, producer thread $P0$ waits until time $T0$, and $P1, P2$ wait until time $T1$ in Figure 25. This is required to maintain strict IO memory protection with existing network drivers. The drivers today free a physical page after every unmap call (shown in blue color in Figure 25). To ensure strict protection, we need to invalidate the IOMMU caches before the physical page is freed and recycled; or else it could result in known integrity and confidentiality attacks [131].

Preserving strict IO memory protection. One-for-many invalidation preserves the invariant required for strict IO memory protection—each core still blocks until its own invalidation request (IR) completes before freeing the page. What changes is *when* the invalidation descriptor is submitted to the IOMMU hardware, *not* the order between IR completion and page free. The only relaxation is *between* cores: one core's IR need not complete before another core enqueues its own. This cross-core concurrency does not violate the invariant, which holds per-page: a page is freed only after the invalidation that covers it has completed.

3.3.3 Deferred Free Page

The vFree design of one-for-many invalidations significantly improves upon existing IO memory protection mechanisms; however, there is still a source of inefficiency: in order to guarantee safety, each producer thread must busy wait until the consumer invalidates and frees the IOVA. For example, in Figure 25, producer threads $P1$ and $P2$ wait until

time $T1$. This wait time can range tens of thousands of clock cycles ($\sim$10–20μs in our measurements) due to the necessary VM exit performed during each IR-SUB operation.

The second key idea in vFree design—deferred free pages—addresses the above inefficiency. The key insight is that strict IO memory protection can be achieved even *without* producer threads busy waiting: for any unmap request for an IOVA I and the corresponding mapped physical page(s) P, if we defer freeing P to after I is invalidated, we maintain strict IO memory protection because the driver cannot recycle the page before it is invalidated from IOMMU caches.

Using deferred free page in vFree. Figure 26 illustrates the vFree design with deferred free page. Each producer thread, upon any unmap, executes IR-PREP and then immediately proceeds—independent of the time taken by the consumer thread to complete invalidation. The key invariant is that free-page is deferred to *after* IR-SUB completes; the consumer thread strictly enforces this ordering.

To achieve this, vFree transfers control of the free-page operation from the device driver to the IOMMU subsystem. This is necessary because, in the design so far (Figure 25), the NIC driver controls the timing of the free-page operation, which may execute before the IOVA is actually invalidated. Specifically, we extend the IOMMU subsystem with a new callback-based `unmap` interface: the NIC driver calls this interface with a callback that performs the free-page operation and associated cleanups (*e.g.*, freeing network buffers). The consumer thread invokes the callback only after IR-SUB completes. Our callback-based interface follows Linux's convention for deferred memory reclamation [150], [151], [152], [153].

A straightforward approach is for the consumer thread to execute all accumulated free-page callbacks, such as freeing socket buffers. However, executing them sequentially on the single consumer core can create a CPU bottleneck when there are many producers: the consumer cannot free socket buffers fast enough, exhausting the finite socket-buffer pool and stalling all producers. vFree therefore dispatches each producer's

free-page operations back to its originating core, enabling parallel execution and offloading the bulk of free-page processing from the consumer's critical path.

Incorporating deferred free page in vFree enables each producer thread to execute more IR-PREPs per round of IR-SUB, creating even larger IR groups and further reducing the number of VM exits per unmap. For example, in Figure 26, at time T_0 the consumer thread gathers all outstanding IRs (IR-PREP$_{2\text{–}4}$)—including multiple IRs from the same producer (IR$_{1,4}$ from the producer on C_1)—into a single IR-SUB operation. After IR-SUB and free-IOVA complete at time T_1, the consumer dispatches free-page callbacks to each originating core, parallelizing the execution of FREE PFN$_{2\text{–}4}$.

Preserving strict IO memory protection. Deferred free page decouples the CPU from IR completion. Rather than blocking for hardware acknowledgment, the CPU enqueues the IR and registers the page-free operation as a callback that fires only when the IOMMU confirms completion. The CPU is free to return to the driver immediately after enqueue. The invariant is preserved by construction: the callback mechanism guarantees that the page is not returned to the allocator until the hardware has confirmed the invalidation.

3.4 Implementation

We implement vFree within Linux v6.12.9 in 1.9K lines of C code. The IOMMU subsystem, the DMA mapping layer, and the NIC driver (mlx5 driver) are modified. vFree requires no modification to the hypervisor (*e.g.*, the QEMU vIOMMU). We present the most interesting implementation details.

3.4.1 One-for-many Invalidation

Tracking IR completion with sequence numbers. A producer must not return until its IRs have been handled by the consumer. vFree uses a sequence-based mechanism to ensure this invariant, avoiding expensive IPIs. Each IR is assigned a *sequence number*

from a monotonically increasing atomic counter per IO domain. A separate IO-domain-wide *completed counter* records the highest number whose IR has been processed. After the consumer finishes IR-SUB and IOVA free operations, it advances the completed counter to the highest sequence number processed. For a producer to ensure its IR to be flushed, it spins on the completed counter until it reaches or exceeds the IR's sequence number.

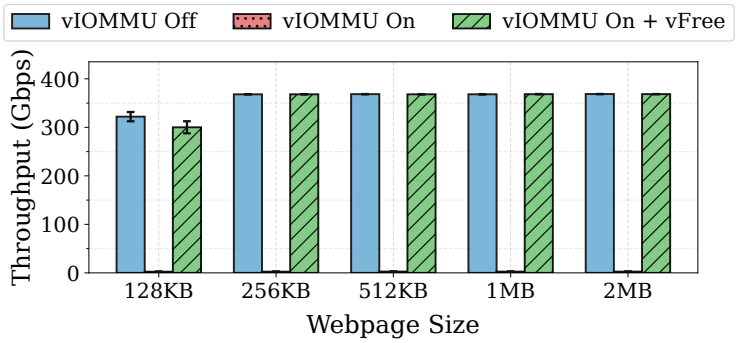
Zero copy with buffer swapping. The per-CPU queue is further optimized with buffer swapping to avoid copy overhead. With deferred free page, the consumer may need to copy up to 256 IRs (∼10 KB) from the per-CPU queues, which is non-trivial under the per-CPU lock. We implement buffer swapping by pairing each *active* per-CPU queue with a *shadow* queue. In this way, we replace the copy operation with a simple pointer swap, minimizing the time holding the per-CPU lock and thus the time blocking active producers.

3.4.2 Deferred Free Page

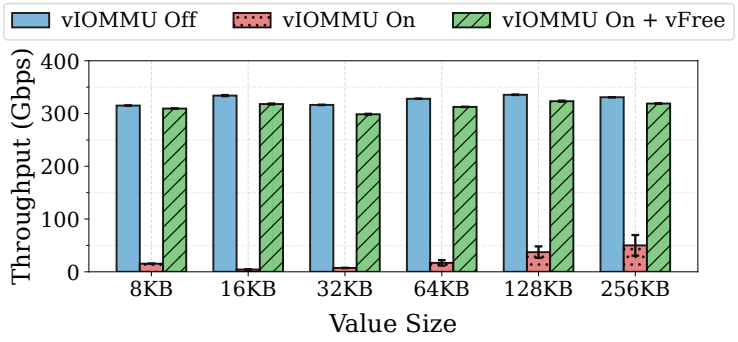
Callback-based `unmap()` interface. For deferred free page, we introduce a callback-based interface in the IOMMU subsystem that allows drivers to defer page free until invalidation completes. We modify the mlx5 NIC driver's Tx data path to use this interface in 350 LOC. The new interface follows Linux's established convention for deferred memory reclamation [150], [151], [152], [153]. The primary interface is `dma_unmap_sg_call`, which unmaps a scatterlist of physically discontinuous pages. The callback function is attached to the invalidation descriptor so it is invoked after the IR-SUB operation completes. Besides the scatterlist version, we also provide a single-region variant that enables DFP for single DMA region unmaps.

NIC driver. We restructure the mlx5 driver to aggregate all post-unmap cleanups—freeing socket buffers, reporting hardware timestamps, and releasing queue resources—into a callback function. The callback is passed to `dma_unmap_sg_call`, so that

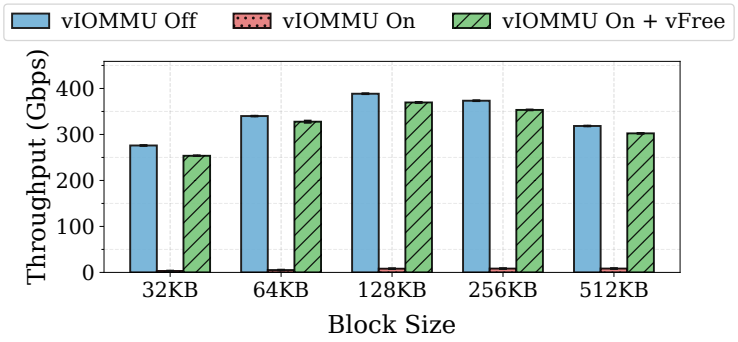
cleanup executes only after invalidation completes. The IOMMU subsystem returns immediately after enqueuing the invalidation descriptor, allowing the producer to resume packet processing without blocking. If the context allocation fails, the driver falls back to the synchronous path.



(a) NGINX



(b) Redis



(c) SPDK

Figure 27: **vFree enables high-performance networked applications to achieve close-to-vIOMMU-off performance with strict protection, consistently delivering low overhead across all evaluated applications.**

3.4.3 Code Optimizations

Reducing critical sections. With nested translation, invalidation is only required for unmap operations, not map operations [133]. However, vanilla Linux acquires the per-IO-domain lock at map time to check if the domain uses nested translation and if a write-buffer flush is needed for legacy devices. The implementation is inefficient as both can be determined at domain creation time. vFree precomputes these attributes and skips the lock acquisition at map time.

Leveraging IOVA contiguity. Inspired by prior work [123], we leverage IOVA contiguity to reduce the number of `map` and `unmap()` calls on the mlx5 Tx path. Each socket buffer may contain multiple fragments of physically discontinuous pages. In the vanilla driver, each fragment is mapped and unmapped individually. We instead use the scatterlist DMA API to map all fragments into a single contiguous IOVA range and unmap them in a single call, reducing the number of invalidation requests generated from `unmap()` operations.

3.5 Evaluation

This section presents vFree evaluation. We first discuss vFree benefits for three real-world applications (§3.5.1), and then dive deeper into understanding vFree benefits using microbenchmarks (§3.5.2). Next, we perform an ablation study of vFree design ideas (§3.5.3), discuss vFree scalability with increasing number of VMs (§3.5.4), and finally also discuss any associated costs of using vFree (§3.5.5). Unless stated otherwise, the system setup is the same as described in §3.2.

Measurement setups. As mentioned in §3.2, we use two servers with Emerald Rapids processor architecture (which supports nested translation) directly connected to each other via 400Gbps Nvidia ConnectX-7 NICs; this is to ensure that performance bottlenecks are not in the network. These servers are 4-socket servers; each socket has 32

physical cores, 252GB of memory and 150GB/s of memory bandwidth. The host server, guest VM, and client machines run with Ubuntu 22.04 and Linux v6.12.9. The VMs are hosted by QEMU v10.0 with nested translation [142] and unmodified host OS (Linux 6.12.9). We place the VMs on the NIC-local NUMA node to avoid any interference/degradation due to cross-NUMA traffic. Every vCPU of the VM is pinned to a separate physical core on the host to avoid interference. We disable hyperthreading, CPU frequency scaling (we fix cores at their base frequency), and NUMA balancing for stable performance. Unless otherwise specified, we run all VMs with 32 vCPUs.

We use Iperf3 [143] to generate network traffic and measure throughput. We measure CPU utilization with sar [154]. We use eBPF probes to record the execution time and invocation counts of relevant kernel functions. We measure VM exit counts, reasons, and time with perf [155]. We enable TCP Segmentation Offload (TSO), Generic Receive Offload (GRO), and adaptive Receive Flow Steering (aRFS) to reflect high-performance networks. We run each experiment five times, and present the average results and standard deviation.

3.5.1 vFree Benefits for Real-world Applications

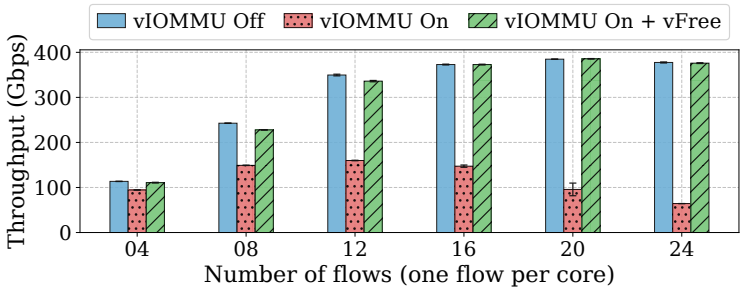
We show the benefits of using vFree for three widely used high-performance networked applications: NGINX (a web server), Redis (a distributed key-value store), and SPDK (a userspace storage stack that uses Linux TCP stack for remote storage). We use the same setup as earlier along with 9KB MTU size. This helps maximize the application performance and avoid CPU becoming the performance bottleneck, when running with vIOMMU off. This allows us to focus mainly on IO memory protection-induced overheads when enabling vIOMMU and/or vFree. We use similar workloads for these applications as in prior work [123]; we describe them below inline with each application. The NGINX workload most closely resembles the Tx-server setup, Redis to Rx-server setup, and SPDK has characteristics of both.

NGINX. We run an NGINX instance inside a VM (which auto-scales to increasing load by spawning more worker threads). On the other host, we launch 24 clients, each pinned to a separate core. The clients use the `wrk` benchmark [156] to issue HTTP `GET` requests to the server. We vary sizes of the requested webpage from 128 KB to 2 MB and report the aggregate throughput for each webpage size.

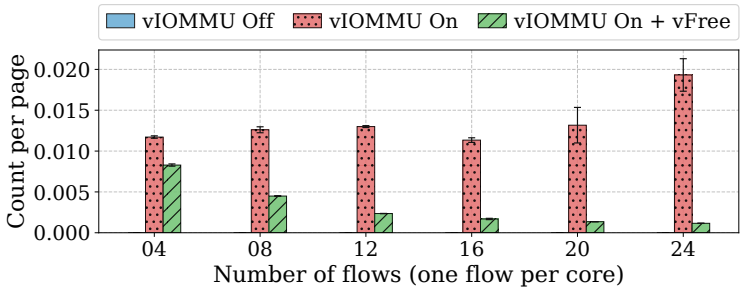
Figure 27a shows over 99% throughput degradation with vIOMMU-on, consistently across all evaluated webpage sizes. vFree improves throughput by up to 147× over vIOMMU-on, and achieves close-to-vIOMMU-off performance for webpage sizes of 256 KB and above, and within 7% for 128 KB.

Redis. We run 24 Redis server instances inside a VM, with one instance per core. We use redis-benchmark [157] to launch an equal number of client instances on the other host, each sending `SET` requests to a corresponding server instance. We vary the `SET` value size from 8 KB to 256 KB, and for each value size, we follow prior work [123] to tune the number of parallel connections, client threads, and request batch size and report the optimal aggregated throughput.

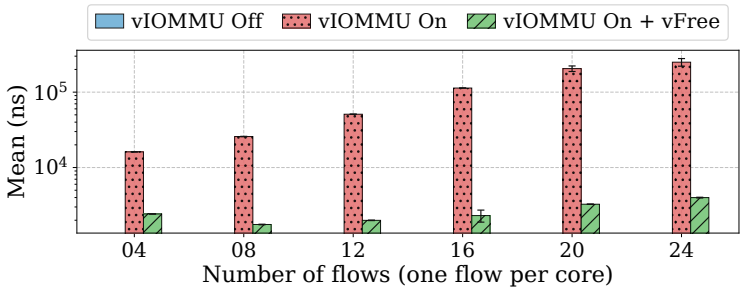
Figure 27b shows that vIOMMU-on can cause up to 98% throughput degradation with higher degradation for smaller `SET` value sizes (<= 64KB). vFree improves throughput by up to 74× over vIOMMU-on and achieves close-to-vIOMMU-off performance, with at most 6% gap.



(a) Throughput

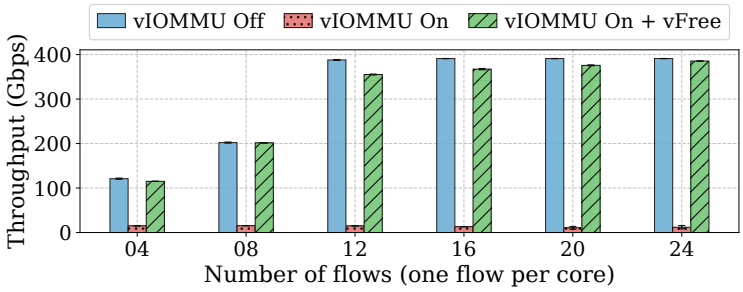


(b) IR-SUB count per page

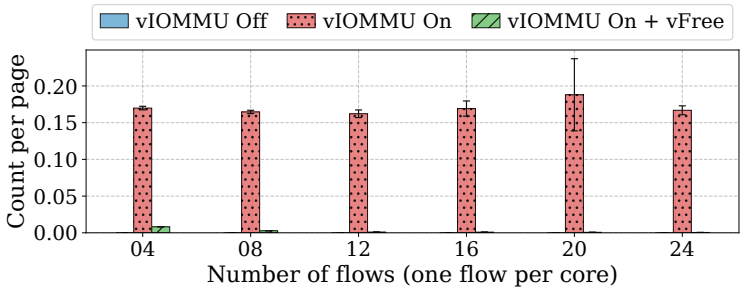


(c) IR processing latency per unmap

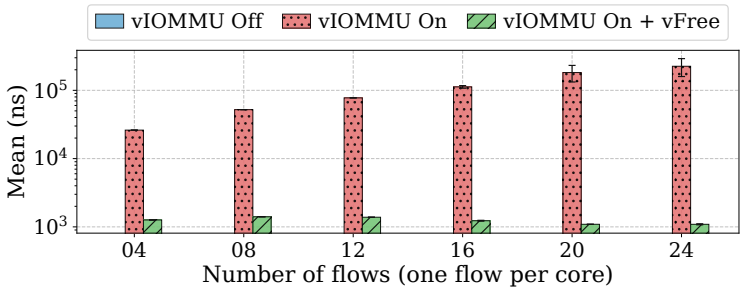
Figure 28: **Rx-server.** vFree achieves up to 4.9× Rx throughput improvement over vIOMMU-on.



(a) Throughput



(b) IR-SUB count per page



(c) IR processing latency per unmap

Figure 29: **Tx-server.** vFree achieves up to 36× Tx throughput improvement over vIOMMU-on.

SPDK. We create a RAM-backed block device and run 24 SPDK server instances inside a VM, one per core. On the other host, we run an equal number of `fio` [158] clients, each generating a workload with 60% reads and 40% writes and an IO depth of 8[10]. We vary the block size from 32 KB to 512 KB and report the aggregate throughput.

Figure 27c shows over 97% throughput degradation with vIOMMU-on across all block

[10]We opt for 60:40 read-write ratio and IO depth 8 to avoid memory bandwidth bottleneck. In general, writes consume twice the memory bandwidth of reads, as each write involves a memory read followed by a memory write.

sizes. vFree improves throughput by up to 82× over vIOMMU-on and achieves close-to-vIOMMU-Off performance, with at most an 8% gap.

3.5.2 Understanding vFree Benefits

To better understand the reasons for vFree performance benefits, we also evaluate vFree using microbenchmarks. We use Iperf3 [143] to generate network traffic and measure throughput. We use similar setup as in §3.2, running same sets of controlled experiments, with the VM acting as the Tx-server and Rx-server, respectively. This enables us to analyze the individual impact of these setups on vFree performance.

Rx-server. As shown in Figure 28a, vFree improves throughput by up to 4.9× in comparison to vIOMMU-on, matching closely to vIOMMU-off performance (with a maximum gap of <6%). The key contributor to this throughput improvement is the reduction of IR-SUB operations per page transferred over network. As shown in Figure 28b, vFree reduces the IR-SUB count per page by up to 82%. Recall from §3.1, each IR-SUB operation requires an unavoidable VM exit, hence, reduction in IR-SUB count translates to a similar reduction in VM exits per page. vFree achieves this reduction using two design ideas. One-for-many invalidations allow using a single IR-SUB to invalidate concurrently prepared IRs across multiple cores—hence, we see lower IR-SUB counts per page with increasing numbers of cores in Figure 28b. Furthermore, deferred free page allows each core to prepare multiple IRs that can be concurrently processed by a single IR-SUB operation. This is achieved by avoiding busy waiting by each core until successful invalidation for each prepared IR, and hence reducing the time spent on IR processing by the core after `unmap`. Figure 28c shows the average time spent by application core on IR processing: while vIOMMU-on incurs large IR processing time due to incurring all—`cache_lock` contention, IR-PREP and IR-SUB overheads, vFree avoids lock contention due to its per-CPU locks, and avoids IR-SUB due to deferred free page, and observes up to 99.9% reduction in IR processing time.

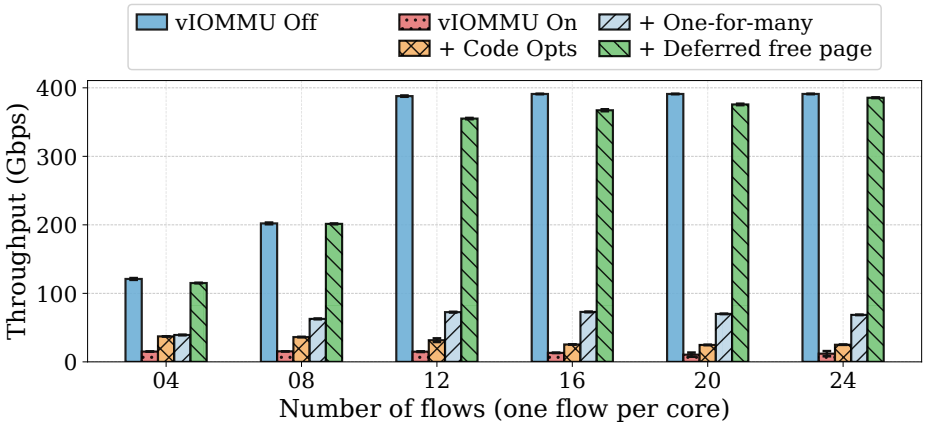
Tx-server. Figure 29a shows that vFree achieves up to 36× throughput improvement over vIOMMU-on, closely matching vIOMMU-off (with the largest gap of <9%). The root cause of vFree benefits are similar to those discussed earlier for the Rx-server case. We briefly discuss the reason behind the relatively larger throughput gains in the Tx-server case. This is because of larger number of unmaps per page performed in Tx datapath (recall that Linux's page pool optimization does not apply to Tx datapath; §3.1). Due to much larger number of unmaps per page, vFree is able to enqueue much larger number of IR descriptors into the IO-domain queue, which can be submitted using a single IR-SUB—hence further reducing the IR-SUB count per page (as evident from Figure 29b). This leads to relatively larger reductions in VM exits per page for Tx-server case, improving the relative performance gains.

3.5.3 vFree Ablation Study

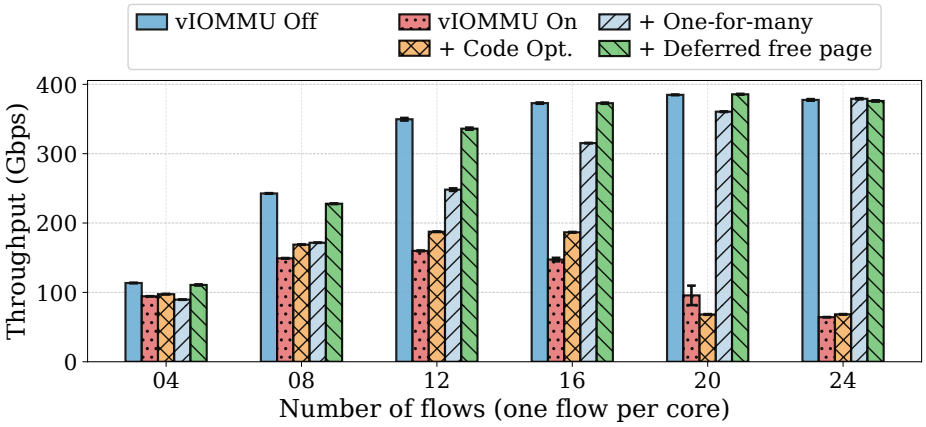
We quantify the contributions of one-for-many invalidation, deferred free page, and other vFree code optimizations (§3.4.3) to overall throughput for both Rx- and Tx-server setups. In our analysis, we vary the number of cores from 4 to 24, each running one flow, and report the corresponding throughput.

Tx-server. Figure 30a shows that code optimizations provide a modest contribution in overall gains. One-for-many invalidations improve the throughput further by 1.1-2.9×, with increasing gains with larger core count. This is expected since more cores enable vFree to invalidate more concurrent IRs via single IR-SUB. Deferred free page, however, provides most significant contribution to throughput gains—2.9 to 5.6×. Without deferred free page, each core stays blocked after IR-PREP until successful invalidation completion for the IR. When no longer constrained to busy wait with deferred free page, each core can continue execution of the future unmaps. And since in Tx-server setup, as discussed earlier, there are a large number of unmaps generated per core, this unblocking of cores results in significant throughput gains.

Rx-server. Unlike Tx-server, Figure 30b shows that for Rx-server, one-for-many invalidations provides more significant contribution to overall gains. This is due to relatively lower numbers of unmaps per page observed in the Rx-datapath: there is relatively smaller performance impact of unblocking the cores using deferred free page due to larger inter-arrival times between unmap requests within each core.



(a) Tx-server



(b) Rx-server

Figure 30: **Necessity of both design ideas—one-for-many invalidations and deferred free page—to achieve vFree benefits.**

3.5.4 vFree Scalability with Increasing VM count

Cloud deployments commonly deploy more than one VM per server. We evaluate how vFree performance fares with an increasing number of VMs. We fix the total core budget

at 32 (the maximum per socket on our setup) and vary the number of VMs from 1 to 16, dividing the cores equally (*e.g.*, assigning $32/N$ vCPUs) and using a dedicated SR-IOV Virtual Function [159] for each VM. We report aggregate throughput across all VMs. We focus on Tx-server setup since it shows the most severe degradation with vIOMMU-on.

Figure 31 shows that vFree maintains close-to-vIOMMU-off performance for each evaluated VM count, while vIOMMU-on results in 80–96% throughput degradation. As expected, with vIOMMU-on, aggregate throughput increases from 16.4 Gbps with 1 VM to 75.8 Gbps with 16 VMs. This is likely due to each VM independently being able to perform IR-SUB concurrently—increasing the arrival rate of IRs to the IOMMU invalidation queue.

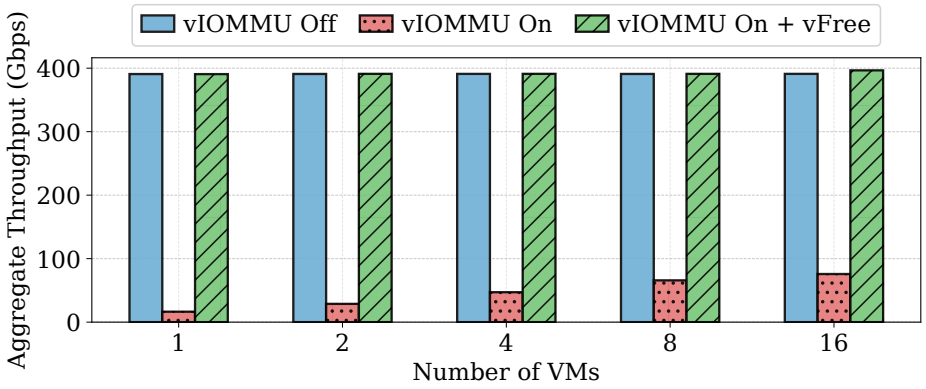


Figure 31: **vFree maintains its benefits with increasing number of VMs per host. Discussion in §3.5.4**

3.5.5 No Free Lunch: The Cost of vFree Benefits

vFree benefits come with two costs: it requires a dedicated core, and it requires extra memory due to deferred free page. We believe the former is not fundamental: it should be possible to opportunistically distribute the IR-SUB processing to producer threads (*e.g.*, any lightly-loaded producer can become a consumer, and execute IR-SUB on its local core). Nevertheless, even with one dedicated core, vFree expands the choices of real-world systems: high performance but with weak (or no) IO memory protection, abysmal performance with *significantly underutilized* cores with strict IO memory protection, or,

high performance with one dedicated core with strict IO memory protection. For the extra memory, our evaluation results presented in the Appendix-B suggest that the extra memory requirements are less than 4% across all our evaluated workloads.

3.6 Related Work

Bare-metal IO memory protection. There is a large and active body of work on enabling IO memory protection with near-zero overheads for the bare metal case [123], [125], [126], [127], [128], [129], [130], [131]. There is also work on rearchitecting IOMMU (*e.g.*, replacing page tables with circular tables [126] and using hardware support for security checks [131]). Collectively, these works have led to mechanisms that enable achieving strict IO memory protection with minimal overheads.

vFree builds upon these works for the bare metal case. For example, persistent mapping for receive buffers has been adopted by Linux (Rx datapath optimizations in §3.1); similarly, as discussed in §3.4, we use the techniques developed in [123]. As shown in Figure 30, these mechanisms are helpful but not sufficient in virtualized environments—the key focus of our work. Indeed, we show in §3.2 that the performance bottlenecks are quite different in virtualized environments and in fact may require design choices that make very different tradeoffs (*e.g.*, vFree reduces the number of VM exits by increasing the address translation cost).

vIOMMU emulation. Amit et al. presented an vIOMMU emulation for unmodified guests [122]. Prior work also optimized other virtualization overhead including memory pinning [160] and interrupts [161], [162]. All the above work was conducted at 10–100 Gbps. vFree extends these works in two ways. First, it presents new insights related to performance bottleneck at 400 Gbps (§3.2); and second, vFree addresses these bottlenecks with software techniques that require no paravirtualization or hardware modifications.

virtIO IOMMU. vFree targets hardware-assisted vIOMMU, not virtio-IOMMU; despite that, the vFree ideas may apply. vIOMMU offers VMs with native hardware interfaces, ensuring compatibility with nested translation and modern drivers. It is supported by mainstream systems such as QEMU [163]. On our platform (§3.2), virtio-IOMMU can only hit 51% of the vanilla vIOMMU throughput with nested translation.

3.7 Concluding Remarks

We show that it is feasible to achieve both strict IO memory protection and high-performance DMA by carefully analyzing performance bottlenecks and reconstructing the software stack. vFree makes a practical solution which can be readily applied to commodity OSes and hypervisors. We hope our work to greatly improve the safety and security of real-world systems with emerging high-speed devices.

3.8 Appendix

3.8.1 Threat Model

Modern OSes typically offer two modes of IO memory protection. The strongest guarantee is that of so-called strict mode [122], [164] which enforces the free-after-invalidation invariant to ensure *no page can be freed with a stale entry that permits access to the page in hardware caches (e.g., IOTLB)*. This invariant is key to data integrity [132] and confidentiality [131]. The other mode—referred to as the lazy or deferred mode [131], [132]—provides weaker protection because it violates this invariant by deferring invalidation after the DMA buffer is unmapped, creating a window of vulnerabilities to integrity and confidentiality attacks.

vFree follows the threat model of strict protection mode of Linux IOMMU [164] and prior work [122], [130], [131]. Our trusted computing base includes the guest OS (including the device driver), the host OS, hypervisor (including the vIOMMU subsystem), and the underlying hardware (CPU, memory system, and physical IOMMU).

We assume that the guest OS correctly configures and manages DMA mappings and that the host OS and physical IOMMU faithfully enforces these mappings. vFree defends against attacks that control malicious or compromised IO devices. The focus is protection from unauthorized DMAs in the intra-VM context.

Boot-time DMA attacks are assumed to be mitigated independently and thus out of the scope of IOMMU protection [130]. Data integrity attacks performed while the device *legitimately* holds a mapping (*e.g.*, a malicious NIC modifying packet payload before the DMA is complete) is equivalent to corruption on disk or a man-in-the-middle attack; this is also outside the IOMMU's scope of protection.

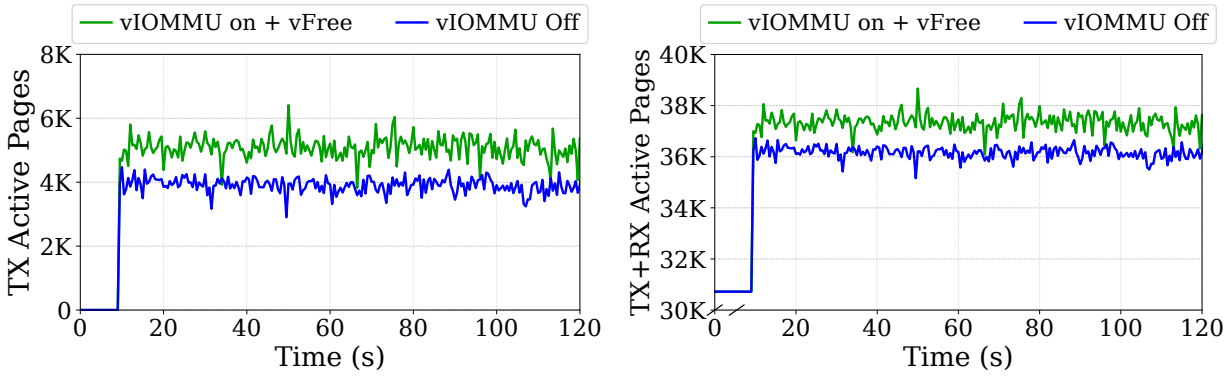
3.8.2 Memory Overhead of vFree

Using deferred protection in vFree provides higher performance at the cost of potentially larger memory utilization. This is because we delay the time when a page is freed. Therefore, for achieving any given throughput, using deferred free page would result in larger number of *active pages* at any point of time, where a page is considered active from the moment it is mapped to an IOVA, until it is freed.

We show that the memory overhead of vFree for our evaluation setup is pretty small. We run 24 Iperf flows for 120 seconds, sampling the number of active pages every 0.5 seconds for vFree and vIOMMU-off. vFree's throughput is within 1% of vIOMMU-off in these experiments.

Tx-server. We measure active physical pages for Tx held by the mlx5 driver. The Tx datapath shows an increase by 30.5%, an inherent cost of deferring free page (Figure 32a). However, Figure 32b shows that when accounting for both Tx and Rx-datapath active pages, the total page count increases by only 3.28% over vIOMMU-off, because the page pool on the Rx side retains pages in its cache rather than actively releasing them to the kernel allocator.

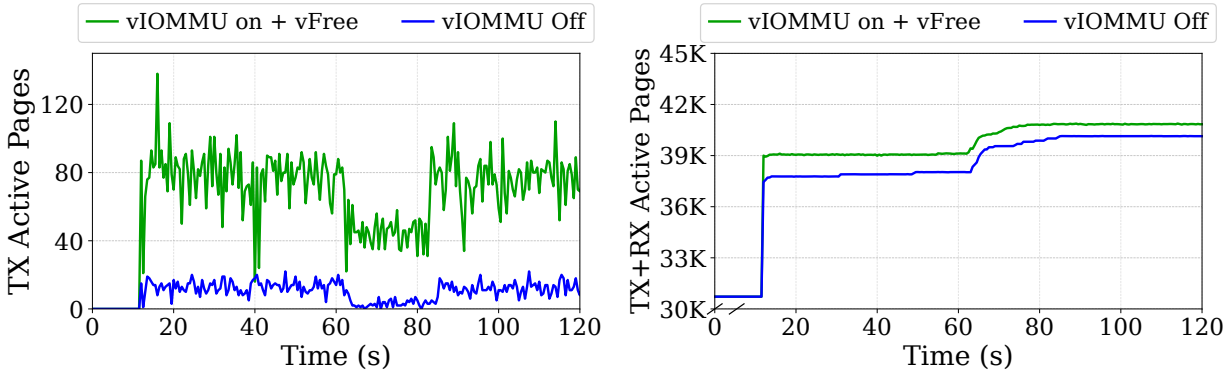
Rx-server. As shown in Figure 33b vFree requires only 2.23% more total pages over



(a) Active pages required by the Tx datapath over time.

(b) Active pages required by the Rx and Tx datapath over time.

Figure 32: For Tx-server setup, vFree incurs less than 3.3% memory overhead over vIOMMU-off.



(a) Active pages required by the Tx datapath over time.

(b) Active pages required by the Rx and Tx datapath over time.

Figure 33: For Rx-server setup, vFree incurs less than 2.3% additional memory overhead over vIOMMU-off.

vIOMMU-off. The active page count for Tx datapath only (Figure 33a) shows a larger relative difference, but memory consumption is dominated by the Rx datapath, so the overall memory overhead is minimal.

Chapter 4 PyTorch-UVM: Practical Unified Virtual Memory for Large Language Model Inference

Modern Large Language Model (LLM) inference workloads are increasingly bottlenecked by GPU memory capacity: flagship model sizes grow roughly 410× every two years, while GPU memory only doubles in the same period [165]. A single high-end accelerator can no longer hold even a moderately sized model together with its KV caches. To overcome the GPU memory wall, modern applications rely on application-driven **manual memory management** that stages weights and activations in host CPU memory [15], [17], [166], [167]. Each such framework re-implements its own memory manager and dictates when and where to move data.

Manual memory management poses two major challenges. First, offload paths are not overhead-free: they demand careful profiling so that copy cost stays hidden behind computation. Second, GPU kernels and allocator logic must be rewritten for *every* application that adopts offloading, and tuned carefully to avoid fatal GPU out-of-memory (OOM) errors. That refactoring is costly: over 7K lines of code in DLRM alone [168].

In contrast, driver-managed automatic memory such as CUDA Unified Virtual Memory (UVM) offers another way past the GPU memory wall. UVM lets the GPU transparently access CPU and peer-GPU memory, enabling oversubscription without application-specific offload code—in principle, it could retire much of the application-specific offloading code that requires manual intervention.

However, UVM has limited adoption in practice since it has been considered too slow for production: PyTorch's own documentation advises against `cudaMallocManaged`, the default UVM allocator. Prior work reports that UVM is over 5× slower than application-level offloading on LLM workloads [17]. This slowdown stems from two root causes. First, frameworks such as PyTorch and vLLM offer no first-class UVM integration, so managed allocations bypass their caching and pooling optimizations and incur memory traffic that an application-aware allocator could elide. Second, the driver

has no visibility into high-level access patterns, so page faults and migrations are triggered late and often evict data that the application is about to reuse.

We present PyTorch-UVM, a framework-aware UVM substrate for PyTorch that closes this performance gap and turns UVM into a practical tool for LLM inference. PyTorch-UVM is built around two pieces that propagate application-level intent down to the UVM driver without changing model code. First, a *custom UVM-aware caching allocator* retains UVM allocations in PyTorch's user-space memory pool and reuses them for future allocations, hiding the per-allocation driver overhead that makes default UVM slow. Its active garbage collection returns cached memory to the driver under oversubscription, avoiding the host OOM kills that a naive caching layer on UVM would suffer. Second, a *framework-guided eviction* strategy marks partially-free cached blocks as evictable so the driver can reclaim them without copying their data back to host.

Even with these two mechanisms, page-fault migration still degenerates into thrashing once the working set substantially exceeds GPU memory: every fault-in evicts a page that is itself about to be reused, and because the driver has no view into the application, it often evicts exactly the page the GPU will touch next. Framework-guided eviction softens this—the driver reclaims already-freed blocks without copying them back to host—but it cannot break the eviction–re-fault cycle once the live working set itself no longer fits on the GPU. We therefore explore two complementary mechanisms. *Direct CPU-memory access* leaves cold pages on host and lets the GPU read them in place over the interconnect, short-circuiting that cycle entirely. *Access-counter–based migration* defers fault-in until a page has been touched above a threshold, filtering one-shot accesses that would otherwise pay full migration cost only to be evicted shortly after. Together, they reveal that the right policy depends on the interconnect and active working-set ratio, not on the workload alone.

We implement PyTorch-UVM on PyTorch 2.8 with CUDA 13 and evaluate Hugging Face OPT and Qwen inference on A100-PCIe, H100-PCIe, and GH200

NVLink-C2C. When memory is not oversubscribed, PyTorch-UVM matches PyTorch's native `cudaMalloc`-based allocator within 20%, while already outperforming default UVM by 52×–58× and HF KV-cache offloading by 30×–35×. Under low oversubscription (ratio < 1.10), where the native allocator crashes with OOM, PyTorch-UVM remains 45× faster than default UVM and 27× faster than HF offloading. Under high oversubscription PyTorch-UVM still avoids OOM, but narrows to 1.6×–1.7× over default UVM and falls behind HF offloading by $\sim$3×—a gap we are actively closing via the thrashing-mitigation mechanisms described above.

4.1 Background

4.1.1 The GPU Memory Wall for LLM Inference

Modern Large Language Model (LLM) inference workloads are increasingly constrained by GPU memory capacity. The size of flagship LLMs has been growing at roughly 410× every two years, while GPU memory capacity only doubles in the same period [165]. As a result, even high-end accelerators (e.g., A100 with 80 GiB or H100 with 80 GiB) struggle to host large models together with their working state.

LLM serving has two primary on-GPU memory consumers:

1. **Model weights**, whose size is fixed by the model architecture (e.g., tens to hundreds of GiB for 13B–70B-parameter models);

2. **KV cache**, whose size grows linearly with sequence length and batch size [15].

To improve serving throughput, modern frameworks such as vLLM [15] batch concurrent requests. Larger batches, however, inflate the KV cache and quickly exhaust GPU memory.

To cope, modern serving frameworks use CPU memory as a larger but slower memory pool and explicitly offload either part of the model weights [15], [17], [166] or part of the KV cache [166], [167], [169] when the working set exceeds GPU memory. Each

framework, however, re-implements its own offloading logic tailored to its own model and execution structure, leading to substantial engineering cost and limited reusability across the ecosystem.

4.1.2 Unified Virtual Memory (UVM)

Unified Virtual Memory (UVM) is a CUDA driver-level feature that lets GPU kernels access memory of the CPU and of peer GPUs transparently, without explicit programmer-managed copies. On allocation through the `cudaMallocManaged` API,[11] pages are owned by the CUDA runtime and are migrated on demand between the CPU and GPU.

UVM is a natural systems-level answer to the GPU memory wall because it offers three properties that are otherwise laboriously re-implemented by every framework:

1. **Memory oversubscription.** UVM transparently spills data to CPU memory when GPU memory is full, enabling models larger than the physical GPU memory to run without CUDA out-of-memory (OOM) errors.

2. **Transparent data movement.** The driver migrates hot pages onto the GPU and can prefetch pages based on hints, avoiding hand-written `cudaMemcpy` pipelines.

3. **Unified memory access.** Both CPU and GPU can dereference the same virtual addresses, opening opportunities for hybrid CPU/GPU execution.

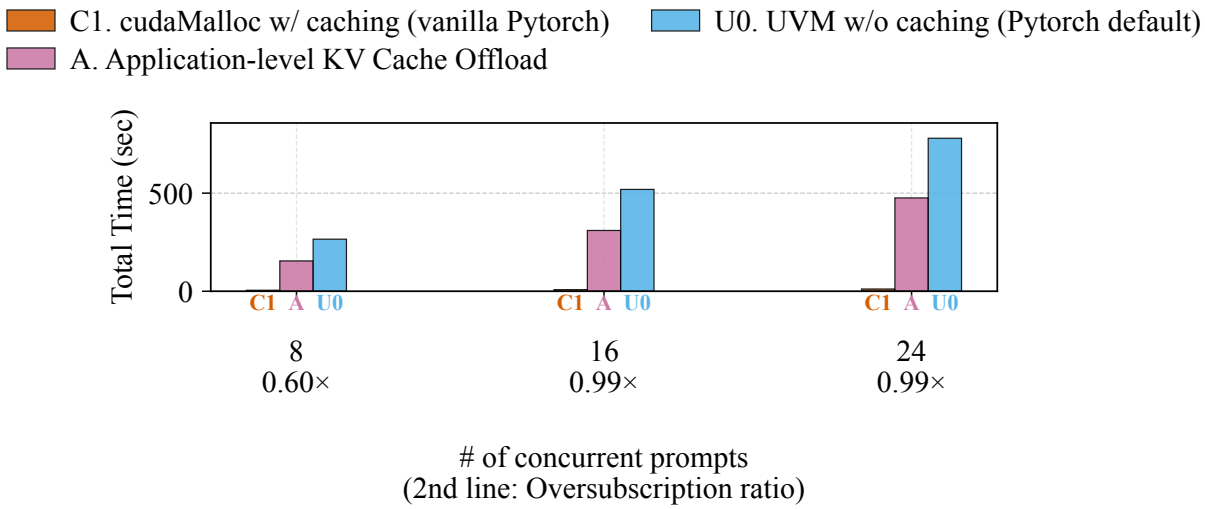
On modern hardware with coherent CPU–GPU interconnects (e.g., NVLink-C2C on GH200, ∼450 GB/s/dir), these properties become even more attractive: the high interconnect bandwidth makes CPU-resident accesses far cheaper.

[11] Conventional GPU allocations use `cudaMalloc`, which allocates device-resident memory and does not support oversubscription.

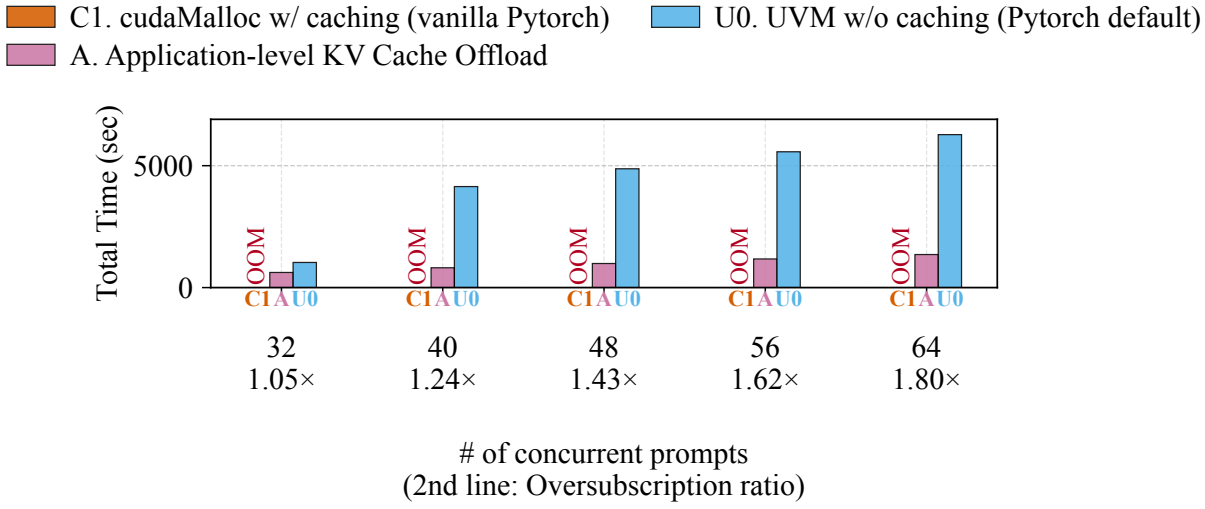
4.1.3 Performance Gap of UVM

Despite its promise, UVM is widely regarded as too slow for production AI workloads. PyTorch's documentation [170] explicitly advises against using `cudaMallocManaged` for performance reasons, and prior work reports that UVM-based inference is over 5× slower per request than application-level KV-cache offloading in FlexGen [17], [166].

We quantify the performance gap of UVM in Figure 34. We evaluate Hugging Face OPT-13B inference on an H100 PCIe GPU across a range of batch sizes. When the working set fits in GPU memory (Figure 34a), vanilla PyTorch with its native `cudaMalloc`-based caching allocator (C1) is highly efficient. In contrast, UVM (U0), implemented with PyTorch's `CUDAPluggableAllocator` interface, is 53×–69× slower than C1, and application-level KV-cache offloading (A) is 31×–42× slower due to the overhead of moving data over PCIe. When we increase the batch size to oversubscribe the GPU memory (Figure 34b), the native allocator (C1) crashes with an out-of-memory (OOM) error. Application-level offloading (A) survives by explicitly moving the KV cache to host memory. Naive UVM (U0) also survives, but its performance collapses: at batch size 40, where the active working set exceeds GPU memory by 1.14×, UVM suffers from severe thrashing and becomes 5× slower than application-level offloading.



(a) Without oversubscription



(b) With oversubscription

Figure 34: Performance gap of UVM for Hugging Face OPT-13B inference on H100 PCIe. (a) When the working set fits in GPU memory (batch sizes 8–24), default UVM (U0) runs over 50× slower than vanilla PyTorch (C1). (b) When the GPU is oversubscribed (batch sizes 32–64), UVM avoids OOM but thrashes, running up to 5× slower than application-level KV-cache offloading (A).

We attribute this gap to two fundamental issues:

Lack of application context. The CUDA driver sees only a stream of memory accesses and has no insight into the structure of an LLM workload—which tensors are

weights versus activations, which blocks are about to be reused, or which blocks have been logically freed by the framework's own allocator. As a result, UVM suffers long GPU page-fault handling latencies, redundant migrations, and pathological eviction decisions when the working set exceeds GPU memory.

Inadequate framework support. Modern AI frameworks such as PyTorch are built around their own caching allocators (e.g., the `CUDACachingAllocator` on top of `cudaMalloc`). Using UVM with PyTorch today requires routing allocations through the `CUDAPluggableAllocator` interface, which bypasses this caching layer and allocates directly from the driver. Every allocation and deallocation therefore reaches the driver, and each one must synchronize against all in-flight GPU kernels to avoid use-after-free errors, incurring substantial per-operation overhead.

Naively replacing `cudaMalloc` with `cudaMallocManaged` inside the caching allocator is not viable either. The caching allocator relies on an allocation failure to trigger garbage collection and return memory to the driver, but `cudaMallocManaged` never fails—it simply spills to host memory. A direct substitution therefore breaks this assumption, causing either host OOM kills or an unbounded UVM working set that destroys performance.

PyTorch has never built first-class UVM integration because UVM's degraded performance has left it impractical for production workloads. To the best of our knowledge, our work is the first attempt to build a framework-aware UVM substrate for a production AI framework.

4.2 Design

4.2.1 User-Space Caching

The first piece of PyTorch-UVM is a UVM-aware, user-space caching allocator that removes the `cudaMallocManaged/cudaFree` traffic off the critical path. Without it, the most natural way to plug UVM into PyTorch—a `CUDAPluggableAllocator` that forwards

every allocation to `cudaMallocManaged` and every deallocation to `cudaFree`—runs 54× slower than vanilla PyTorch and even 1.6× slower than Hugging Face's application-level KV-cache offloading on OPT-13B (§4.1.3). The reason is structural: PyTorch's native `CUDACachingAllocator` retains freed blocks in user space and amortizes `cudaMalloc` cost across many tensor lifetimes, whereas the pluggable wrapper bypasses that cache and exposes every PyTorch `malloc`/`free` to the CUDA driver. PyTorch-UVM closes this gap by porting the caching discipline of the native allocator to managed memory, while adding the extra bookkeeping that UVM requires to remain safe under oversubscription.

CUDACachingAllocator-UVM. PyTorch-UVM introduces `CUDACachingAllocator-UVM`, a drop-in caching allocator that sits between PyTorch tensors and the CUDA driver, and that allocates its backing storage with `cudaMallocManaged` instead of `cudaMalloc`. Its block-management policy mirrors the native allocator: blocks obtained from CUDA are split, coalesced, and reused across tensor lifetimes, so a PyTorch `free` merely returns a block to the cache rather than calling `cudaFree`. As a result, only a small fraction of PyTorch-level allocations translates into a `cudaMallocManaged` call, and freed blocks remain available for fast reuse without triggering additional GPU driver operations. Because the underlying memory is managed by UVM, these cached blocks are still eligible for transparent CPU spilling when the GPU is oversubscribed, so PyTorch-UVM retains UVM's oversubscription guarantees.

UVM-Specific Active Garbage Collection. Naively reusing the native caching policy on top of UVM is, however, unsustainable. The native allocator relies on `cudaMalloc` failing an allocation as a back-pressure signal: when CUDA refuses a new allocation, the caching allocator interprets this as memory pressure, invokes its garbage collector to release cached blocks via `cudaFree`, and retries. The retry then draws on the just-freed memory, which CUDA can coalesce into a region large enough to satisfy the request. With `cudaMallocManaged`, no such signal ever arrives—the call simply succeeds by spilling pages to host memory. Consequently, a UVM-backed caching allocator keeps

growing its reservation indefinitely: GPU-resident pages overflow into CPU memory, cached but unused blocks are never returned. This produces an unbounded UVM working set that destroys application performance, and in extreme cases the host kernel OOM-kills the process once CPU memory is exhausted.

To eliminate this failure mode, PyTorch-UVM replaces the missing OOM signal with an explicit, GPU-side capacity check inside the allocator's slow path. Before satisfying an allocation that would require a fresh `cudaMallocManaged` of size s, the allocator tests whether $s + $ `gpu_reserved` would exceed the physical GPU memory, where `gpu_reserved` is the total bytes the allocator has already obtained from CUDA. If so, it triggers active garbage collection (GC): it walks its free-block list, releases reclaimable blocks back to CUDA via `cudaFree`, and retries the allocation. This keeps the allocator's reservation bounded by the physical GPU capacity, so UVM is only ever asked to oversubscribe on *live* working-set growth, not on stale cached blocks—avoiding the unbounded host-memory growth that makes naive UVM caching unsustainable.

Outcome. Together, user-space caching and UVM-specific active GC turn UVM from an unusable substrate into a competitive one. On the non-oversubscribed regime, PyTorch-UVM's allocator is within a small overhead of vanilla PyTorch, delivering 51× speedup over the pluggable-allocator UVM baseline and 30× speedup over Hugging Face's KV-cache offloading, while transparently enabling oversubscription without any application-level changes. What remains, and what motivates the rest of this section, is that caching on top of UVM also *couples* the allocator's cache to UVM's eviction decisions, creating fragmentation patterns that pure GC cannot resolve (§4.2.2).

4.2.2 Framework-Guided Eviction with UVMDiscard

User-space caching plus active GC stabilizes PyTorch-UVM's working set only when freed blocks can be returned to CUDA whole. In real PyTorch workloads, however, cached blocks are routinely *split* to satisfy smaller allocations, and the resulting

fragments cannot be released independently—a single `cudaMallocManaged` region can only be returned to CUDA via a single `cudaFree` of the entire region. This couples caching with fragmentation in a way that pure GC cannot resolve, and pushes UVM into precisely the eviction traffic that PyTorch-level information would otherwise let us avoid.

Consider a sequence that allocates two 40 GiB blocks, frees them, then allocates two 25 GiB tensors. It produces two sets of 25 GiB-used / 15 GiB-free split out of a cached block. A subsequent 30 GiB allocation cannot be served from the cache. The two 15 GiB free fragments are non-contiguous virtually, so it can not satisfy the request. What'sm more, they live inside still-live 40 GiB `cudaMallocManaged` regions, so GC cannot return them to CUDA either. The allocator must therefore obtain a fresh 30 GiB region from CUDA, which on a now-reserved-full GPU forces UVM to evict roughly 30 GiB of pages to host memory. Worse, the driver picks evictees from *all* resident pages—including the 25 GiB *live* tensors that the application is actively using—incurring both non-trivial eviction traffic and the risk of evicting the wrong data.

Discarding Partially-Freed Blocks. PyTorch-UVM addresses this fragmentation–eviction coupling by extending its slow path with a second tier of reclamation that operates at sub-block granularity. Before issuing a new `cudaMallocManaged` when `new_size` + `gpu_reserved` > `gpu_total`, the allocator (i) `cudaFree`s independent (fully unused) cached blocks, as in active GC, and then (ii) for any cached block that is split into used and free fragments, marks them discardable to the UVM driver with UVMDiscard [171]. Discard requires no application change: PyTorch-UVM already tracks the exact byte-range layout of every cached block as part of its allocator metadata, so identifying the partially-freed sub-regions is a cheap lookup with no involvement from the GPU. Crucially, the live fragments of the same block are left untouched, so all existing tensors retain their virtual-address mappings and continue to fault and migrate as usual.

Higher-Priority, Copy-Free Eviction. Once a region is marked discarded, the UVM

driver treats it differently from a normal resident page in two ways. First, discarded pages are eligible to be evicted *without* a device-to-host copy: the driver simply unmaps them and reclaims the physical frames, skipping the PCIe (or NVLink-C2C) transfer that dominates eviction cost on PCIe platforms. Second, discarded pages are evicted at a *higher priority* than normal resident pages, so when the driver needs to free GPU memory for a new allocation it draws from the discarded pool first. For the fragmentation example above, discarding the two 15 GiB free fragments lets the driver satisfy the 30 GiB allocation by reclaiming exactly those 30 GiB—with no copy and without touching live tensor data.

Guiding UVM Away from Useless Data. The combined effect is that PyTorch-UVM converts PyTorch-level knowledge of which bytes are no longer needed into a UVM-level eviction hint that the driver can act on at the next GPU page fault or allocation, without copying or evicting live data. This redirects the dominant source of eviction traffic from data the application still needs to data it has already discarded, cutting both eviction volume and end-to-end latency. Empirically, discard makes UVM up to 1.6× faster than the PyTorch-UVM configuration without it and 27× faster than Hugging Face's KV-cache offloading on OPT-13B.

4.2.3 Thrashing Mitigation Exploration

By default, PyTorch-UVM faults in every GPU-touched page so that subsequent accesses hit local HBM. This maximizes GPU residency at low oversubscription, but at high oversubscription it thrashes: every fault-in forces an eviction of a page that may itself be touched again moments later. UVMDiscard (§4.2.2) cuts the cost of each eviction but not the eviction–re-fault cycle itself, which dominates wall time for LLM inference on a discrete-PCIe GPU once oversubscription is significant. We explore two mechanisms that attack the cycle by changing *when* a page migrates.

Direct CPU-Memory Access. Rather than migrate on every fault, PyTorch-UVM

can advise the driver to keep pages on host (with `cudaMemAdvise` call) and let the GPU access them in place over the interconnect, trading higher per-access latency for the elimination of page-fault traffic. This wins when the alternative would have been a near-immediate eviction and the CPU–GPU link has bandwidth to absorb in-place accesses. We evaluate it on H100 PCIe in §4.3.3.

Access-Counter–Based Migration. NVIDIA GPUs maintain per-page access counters, and the driver can defer migration until a host page is touched above a threshold from the device, filtering out one-shot accesses while still promoting genuinely hot pages. Access counters are currently exposed only on Grace-based systems with 64 KiB UVM pages, so we evaluate this on GH200 in §4.3.3; extending it to discrete-PCIe H100 (4 KiB pages) awaits driver support.

4.2.4 Extension to System-Allocated Memory (SAM)

`cudaMallocManaged` is not the only way to back UVM. *System-Allocated Memory* (SAM) delivers the same UVM properties — transparent data movement and unified CPU/GPU addressing — by letting the GPU access ordinary host pages from `malloc`/`free`, which the OS exposes to the GPU via ATS on GH200 or Linux's HMM path on x86 / H100. We port PyTorch-UVM to both; two differences matter.

Allocation alignment. `cudaMallocManaged` returns coarsely aligned pointers (e.g. 512 B on GH200) that downstream libraries such as cuBLAS rely on for tensor-core and TMA-style accesses, whereas `malloc` guarantees only the C-ABI minimum. PyTorch-UVM's SAM allocator therefore uses `posix_memalign` to reproduce the `cudaMallocManaged` alignment, so existing kernels work unchanged.

Free-time synchronization. `cudaFree` internally synchronizes against in-flight kernels and copies on the allocation's stream(s), so freeing a buffer while a kernel still references it is safe. Plain `free` offers no such guarantee, so on the SAM path it can yield a

use-after-free. PyTorch-UVM's SAM allocator therefore wraps `free` with an explicit CUDA-side synchronization of in-flight work on the relevant streams, matching `cudaFree`.

CUDA-level hints (`cudaMemAdvise`, `cudaMemPrefetchAsync`) are unchanged for SAM, but UVMDiscard is not yet supported on the SAM path. Evaluation results appear in §4.3.4.

4.3 Evaluation

4.3.1 Experimental Setup

Hardware. We evaluate PyTorch-UVM on two platforms that span the two CPU–GPU interconnects of interest for UVM today: a discrete PCIe-attached system and a coherent NVLink-C2C system.

1. **H100-PCIe.** An NVIDIA H100-PCIe with 80 GiB of HBM, paired with an AMD EPYC 7413 (24 cores) host and 252 GiB of DDR4 system memory. The CPU and GPU communicate over PCIe Gen-4 (∼32 GB/s/dir). The default GPU page size is 4 KiB.

2. **GH200 (NVLink-C2C).** An NVIDIA GH200 Grace Hopper Superchip that integrates a 72-core Grace CPU (Arm Neoverse-V2, single socket) with a Hopper GPU on a single coherent fabric. The Hopper GPU exposes 96 GiB of HBM3 and the Grace CPU is attached to 480 GiB of LPDDR5X, all addressable from the GPU through the NVLink-C2C interconnect (∼450 GB/s/dir). The host runs an Arm64 Linux kernel built with 64 KiB base pages, so UVM uses 64 KiB managed pages on this platform—a configuration we explicitly call out because page granularity changes the cost of UVM faults and discard.

Software. Both platforms run the same software stack: PyTorch 2.8, Hugging Face Transformers v4.55, CUDA 13.0, and NVIDIA driver 580.95. PyTorch-UVM is

implemented as a modified `CUDACachingAllocator` inside this PyTorch build, so all baselines and PyTorch-UVM configurations share an identical Python and framework surface and differ only in the underlying allocator.

Models and workloads. Our primary workload is Hugging Face causal-LM inference on the OPT family (OPT-6.7B through OPT-175B), with Qwen-2.5 (7B–72B) and Qwen-1.5 (110B) used to confirm that results generalize across model architectures. Each request uses a context length of 2048 tokens (1920 prompt tokens + 128 generated tokens). We sweep two oversubscription axes: (i) *batch-size sweep*—fix the model (OPT-13B as the canonical configuration) and grow the batch size from 4 to 64, inflating the KV-cache footprint until the working set no longer fits on the GPU; and (ii) *model-size sweep*—fix the batch size at 1 and grow the model footprint until the weights alone exceed device memory.

Baselines. We compare PyTorch-UVM against the four classes of allocators described in §4.1:

1. **C1, vanilla PyTorch.** The stock `CUDACachingAllocator` on top of `cudaMalloc`. Fast when the working set fits; OOMs the moment it does not.

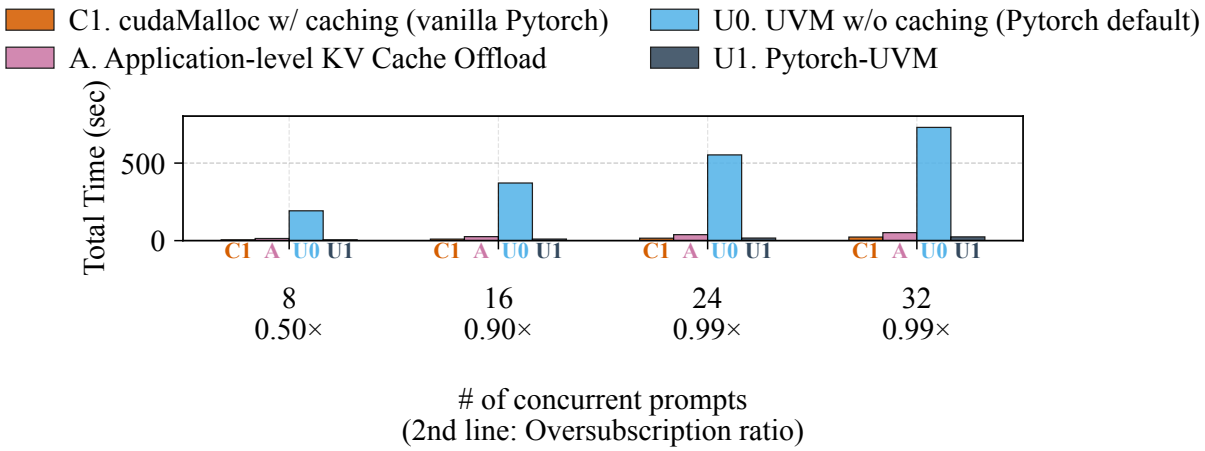
2. **A2, application-level weight offload.** Hugging Face's built-in CPU offloading of model weights, which manually streams parameters between host and device around each forward pass.

3. **U0, default UVM.** A `CUDAPluggableAllocator` that forwards every allocation to `cudaMallocManaged` (first-touch placement), representing the naive way to use UVM in PyTorch today.

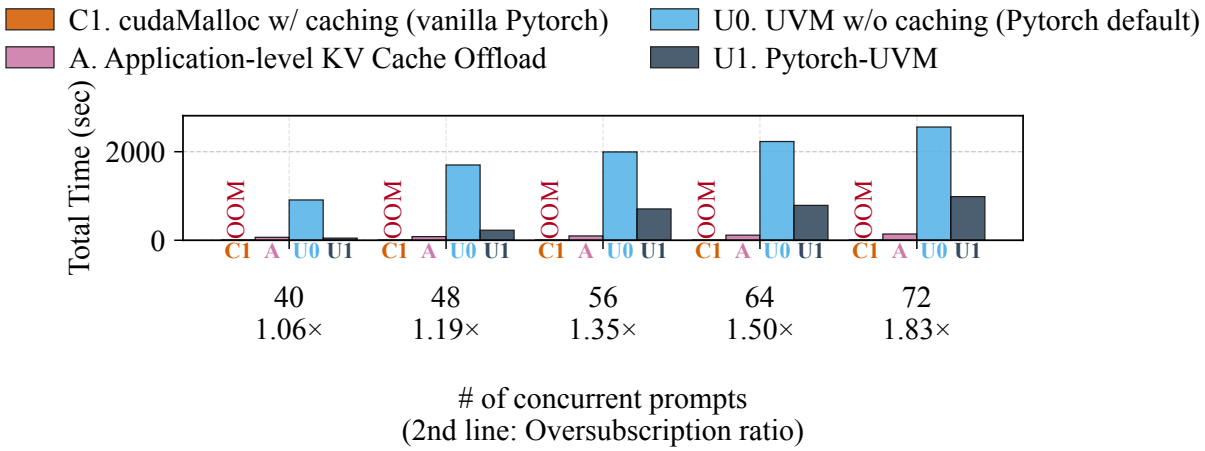
4. **U1, PyTorch-UVM.** Our full design, combining the UVM-aware caching allocator, UVM-specific active GC, and framework-guided eviction (§4.2.1, §4.2.2).

Metrics. We report end-to-end request latency, decomposed where relevant into Time-to-First-Token (TTFT) and Inter-Token Latency (ITL).

4.3.2 Pytorch-UVM Performance



(a) Performance before oversubscription (batch size 8–32)



(b) Performance after oversubscription (batch size 40–72)

Figure 35: Total run time on GH200 NVLink-C2C for varying batch sizes on OPT-13B.

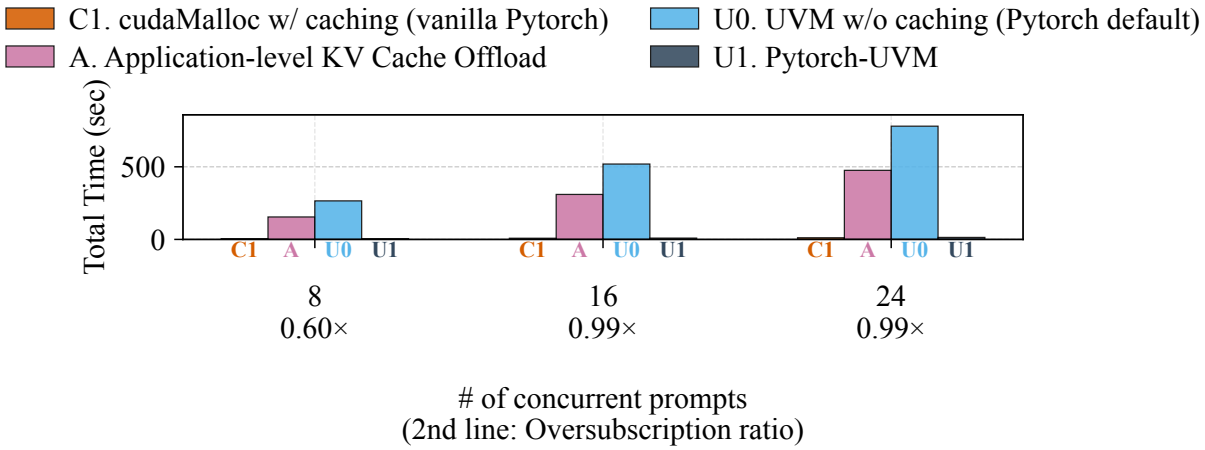
Concurrent requests GH200 NVLink-C2C. Figure 35 shows the total run time of OPT-13B on the GH200 platform as we scale the number of concurrent prompts. We split the results into two regimes: before oversubscription (batch size 8 to 32) and after

oversubscription (batch size 40 to 72).

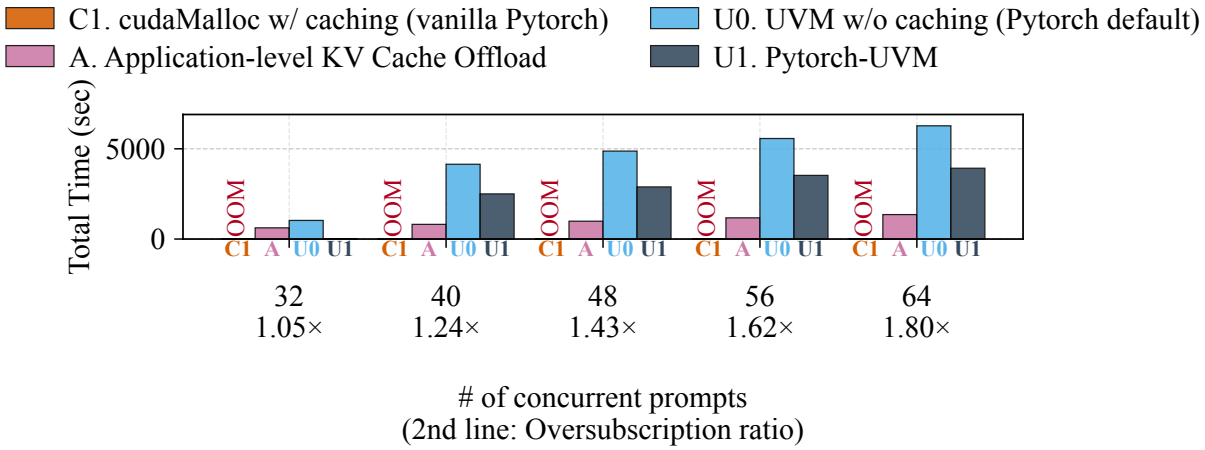
Before oversubscription (Figure 35a), the total working set comfortably fits in the physical GPU memory. Vanilla PyTorch (C1) and PyTorch-UVM (U1) achieve nearly identical performance (within 5% at batch size 32). In contrast, the application-level KV cache offload (A) incurs a significant overhead (2.2× slower) due to manually moving tensors between the CPU and GPU. The PyTorch default UVM (U0) performs worse by orders of magnitude (30× slower) due to the lack of user-space caching, exposing every allocation to the driver.

After oversubscription (Figure 35b), the total working set grows to exceed the 96 GiB HBM limit (from 1.06× oversubscribed at batch size 40 up to 1.83× at batch size 72). Vanilla PyTorch (C1) consistently fails with an Out-Of-Memory (OOM) error in this regime. PyTorch-UVM (U1), however, runs seamlessly, and provides a massive speedup over default UVM (U0)—for example, 19× faster at batch size 40 and 2.6× faster at batch size 72.

Compared to the application-level KV cache offload (A) in the oversubscribed regime, PyTorch-UVM (U1) is initially faster at light oversubscription (1.38× faster at batch size 40). However, at heavy oversubscription (batch size 48 to 72), the application-level offload outperforms PyTorch-UVM (U1), because the former refactors application code to explicitly move KV cache tensors to GPU before they are needed, whereas PyTorch-UVM relies on transparent page faults and UVM-level eviction heuristics. Nonetheless, PyTorch-UVM achieves functional oversubscription completely transparently without the engineering complexity of explicit KV cache management.



(a) Performance before oversubscription (batch size 8–24)



(b) Performance after oversubscription (batch size 32–64)

Figure 36: Total run time on H100 PCIe (4 KiB pages) for varying batch sizes on OPT-13B.

H100 PCIe. Figure 36 shows the total run time of OPT-13B on the H100 PCIe platform as we scale the number of concurrent prompts. We split the results into two regimes: before oversubscription (batch size 8 to 24) and after oversubscription (batch size 32 to 64).

Before oversubscription (Figure 36a), the total working set fits in the 80 GiB HBM (oversubscription ratio of 0.60× at batch size 8 to 0.99× at batch size 24). Vanilla PyTorch (C1) and PyTorch-UVM (U1) achieve nearly identical performance. In contrast,

application-level KV cache offload (A) is roughly 35× slower than PyTorch-UVM, because every forward pass must stream the KV tensors over the comparatively narrow PCIe Gen-4 link. PyTorch default UVM (U0) is even worse, roughly 57× slower than PyTorch-UVM, as it pays a UVM page-fault round-trip for every cold access without any user-space caching.

After oversubscription (Figure 36b), the total working set exceeds the 80 GiB HBM limit (from 1.05× oversubscribed at batch size 32 up to 1.80× at batch size 64). Vanilla PyTorch (C1) fails with OOM across the entire range. PyTorch-UVM (U1) runs to completion in every configuration and outperforms default UVM (U0) throughout (e.g., a 45× speedup at batch size 32 and a 1.6× speedup at batch size 64). At very light oversubscription (batch size 32, 1.05×), PyTorch-UVM stays close to the in-memory baseline and is also more than 27× faster than application-level offload.

Beyond batch size 32, however, the comparison with application-level offload reverses on H100 PCIe: A is ∼3× faster than PyTorch-UVM across batch sizes 40 to 64. Unlike on GH200, where NVLink-C2C makes transparent UVM page faults relatively cheap, PCIe Gen-4's much lower bandwidth amplifies the per-fault cost, so the perfectly synchronous CPU–GPU streaming of application-level offload pays off as soon as the working set materially exceeds device memory. PyTorch-UVM still delivers oversubscription transparently and significantly improves over default UVM, but on the discrete-PCIe interconnect there is a clear performance gap to hand-tuned application-level offload at heavy oversubscription.

Large models GH200 NVLink-C2C. Figure 37 shows the total run time of OPT and Qwen models on a GH200 system with 64 KiB pages. As the model size increases, the memory footprint eventually exceeds the physical GPU memory capacity.

Vanilla PyTorch (C1) fails with an Out-Of-Memory (OOM) error on the largest models (OPT-66B, OPT-175B, Qwen2.5-72B, and Qwen1.5-110B), because its native

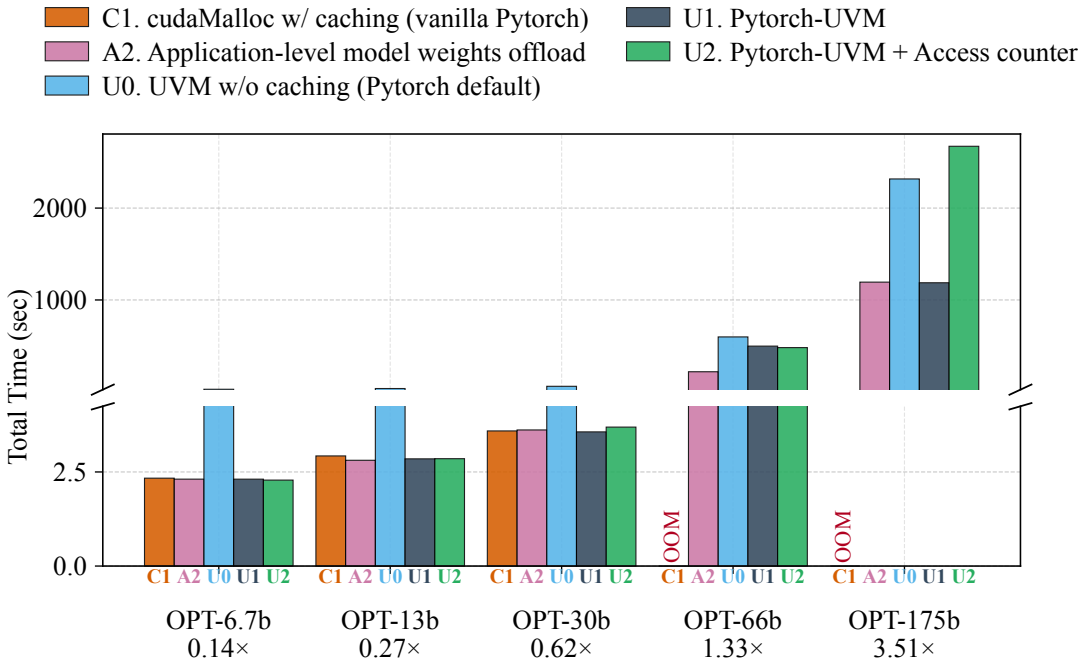
allocator expects memory to be strictly device-resident. Application-level weight offload (A2) runs all models by manually streaming weights between the CPU and GPU, avoiding OOM at the cost of application complexity; default UVM (U0) likewise avoids OOM by transparently spilling pages to host, but degrades severely under oversubscription due to unrestrained allocation growth and eviction thrashing.

PyTorch-UVM (U1) runs all models transparently. When memory is not oversubscribed (e.g., OPT-6.7B–OPT-30B, Qwen2.5-7B–Qwen2.5-32B), it matches vanilla PyTorch (C1) and A2, showing that our user-space caching and UVM-specific active garbage collection (§4.2.1) eliminate the overhead of managed memory in the common case. When the GPU is oversubscribed, PyTorch-UVM outperforms default UVM (U0) by a large margin (e.g., 1.95× on OPT-175B and 1.87× on Qwen1.5-110B), while still matching the application-level weight-offload baseline (A2) without any manual changes to application code.

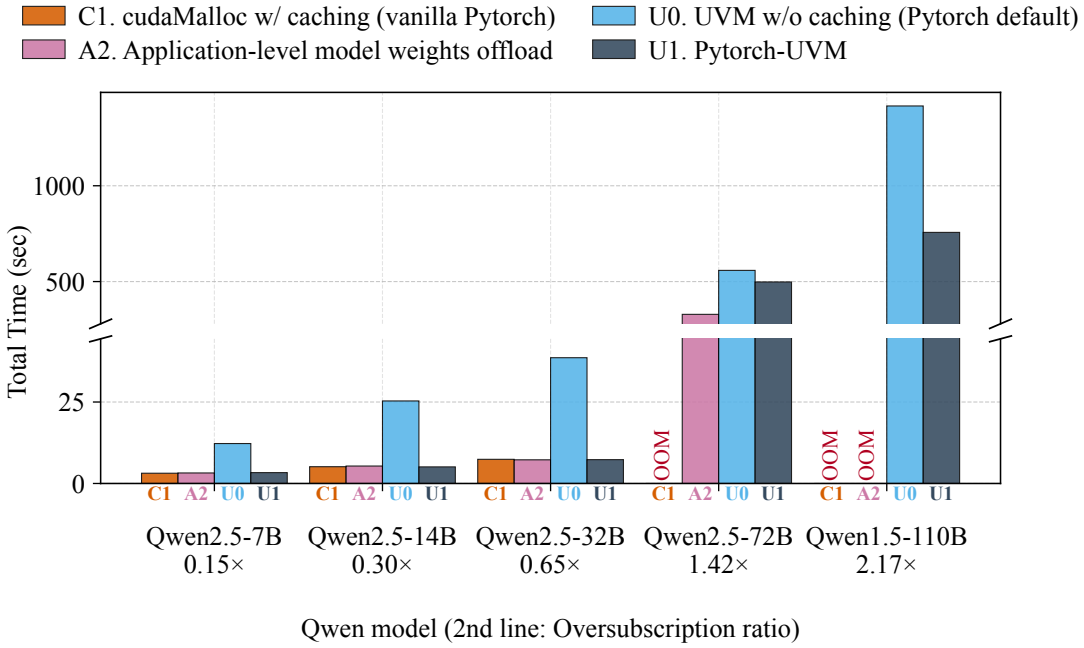
H100 PCIe. Figure 38 repeats the model-size sweep on the H100 PCIe platform. We omit OPT-175B from this sweep because the model's roughly 350 GB of fp16 weights exceed the host's 252 GiB of system memory: every UVM-based baseline (U0, U1, and even A2) fails with a host-side OOM before it can stream the first page to the GPU, so there is no meaningful number to report.

Below the device-memory limit. For models whose weights fit in the 80 GiB of HBM (OPT-6.7B–OPT-30B and Qwen2.5-7B–Qwen2.5-32B), PyTorch-UVM (U1), vanilla PyTorch (C1), and application-level offload (A2) are essentially indistinguishable, while default UVM (U0) is roughly 10–20× slower. This mirrors the GH200 result and confirms that PyTorch-UVM's caching and active GC eliminate UVM's per-allocation overhead on H100 PCIe just as effectively as on NVLink-C2C.

Above the device-memory limit. On the largest models that still fit in host memory (OPT-66B, Qwen2.5-72B, Qwen1.5-110B), Vanilla PyTorch (C1) OOMs as expected, and the gap between U1 and U0 narrows or even reverses on H100: on



(a) OPT models

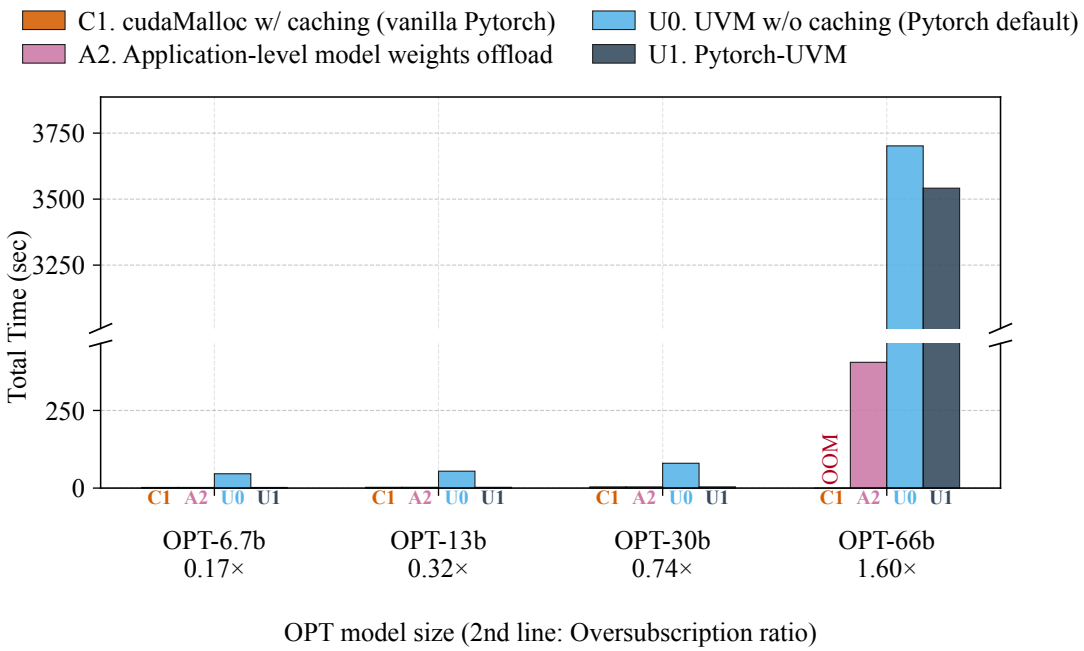


(b) Qwen models

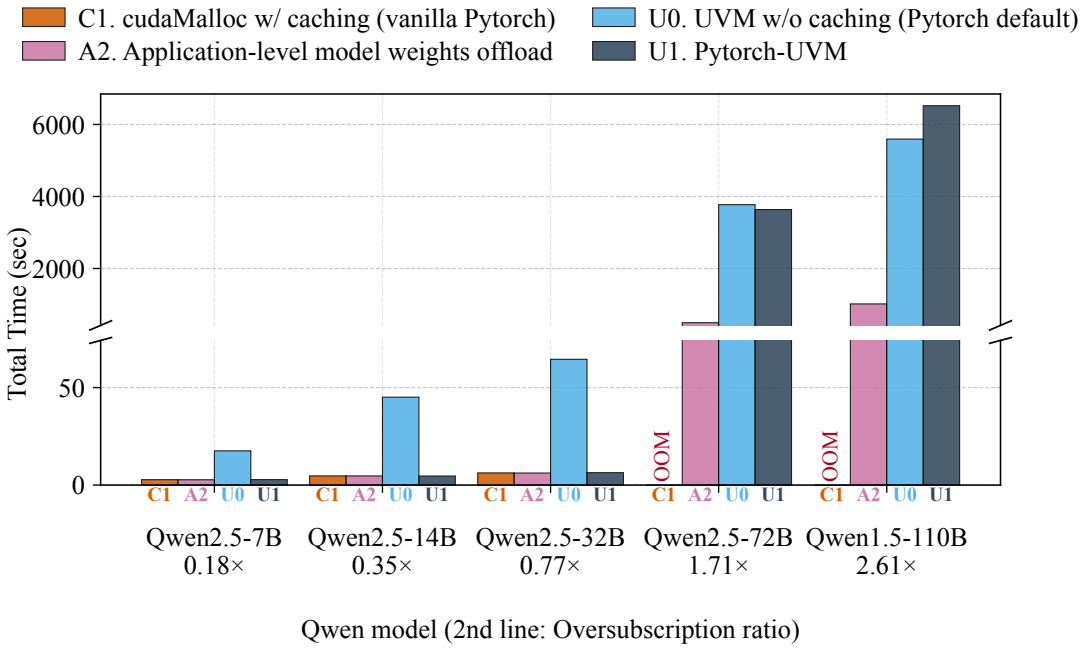
Figure 37: Total run time on GH200 NVLink-C2C for varying model sizes.

OPT-66B U1 is only ∼5% faster than U0; on Qwen2.5-72B ∼4% faster; and on Qwen1.5-110B PyTorch-UVM is actually ∼1.17× slower than U0. Application-level offload (A2) wins decisively across the board on these models, roughly 7–9× faster than PyTorch-UVM on H100 PCIe.

This is in sharp contrast to GH200 NVLink-C2C, where PyTorch-UVM achieved 1.95× and 1.87× speedups over U0 on OPT-175B and Qwen1.5-110B respectively (§4.3.2, GH200 paragraph). The first-order explanation is interconnect bandwidth. At batch size 1 the decode loop reads the model's weights once per token, so the achievable throughput is set by how fast pages can move from host to GPU. NVLink-C2C provides ∼450 GB/s/dir, leaving the interconnect with enough headroom that eliminating spurious eviction and re-fault traffic (the work that framework-guided eviction and the active GC do) translates directly into end-to-end speedup. PCIe Gen-4 provides only ∼32 GB/s/dir — more than 10× less — so on H100 the link itself is the bottleneck for both U0 and U1 in this regime, and the savings from smarter eviction are largely absorbed by the link being saturated either way. A2's hand-tuned synchronous streaming keeps the PCIe link continuously busy without the per-page-fault overheads, which is why it dominates here. We expect this gap to shrink on PCIe Gen-5 / NVLink-attached H100 variants where the bandwidth-to-compute ratio more closely resembles GH200, but on the discrete-PCIe-Gen-4 platform PyTorch-UVM primarily delivers oversubscription transparency rather than a meaningful performance edge over U0 in the model-weights-dominated regime.



(a) OPT models



(b) Qwen models

Figure 38: Run time on H100 PCIe for varying model sizes (1920 prompt + 128 decode tokens, batch size 1). OPT-175B is omitted because its weights exceed the host's 252 GiB of system memory, so all UVM-based baselines fail with a host OOM before they can begin streaming pages to the GPU.

4.3.3 Analysis of Design Choices

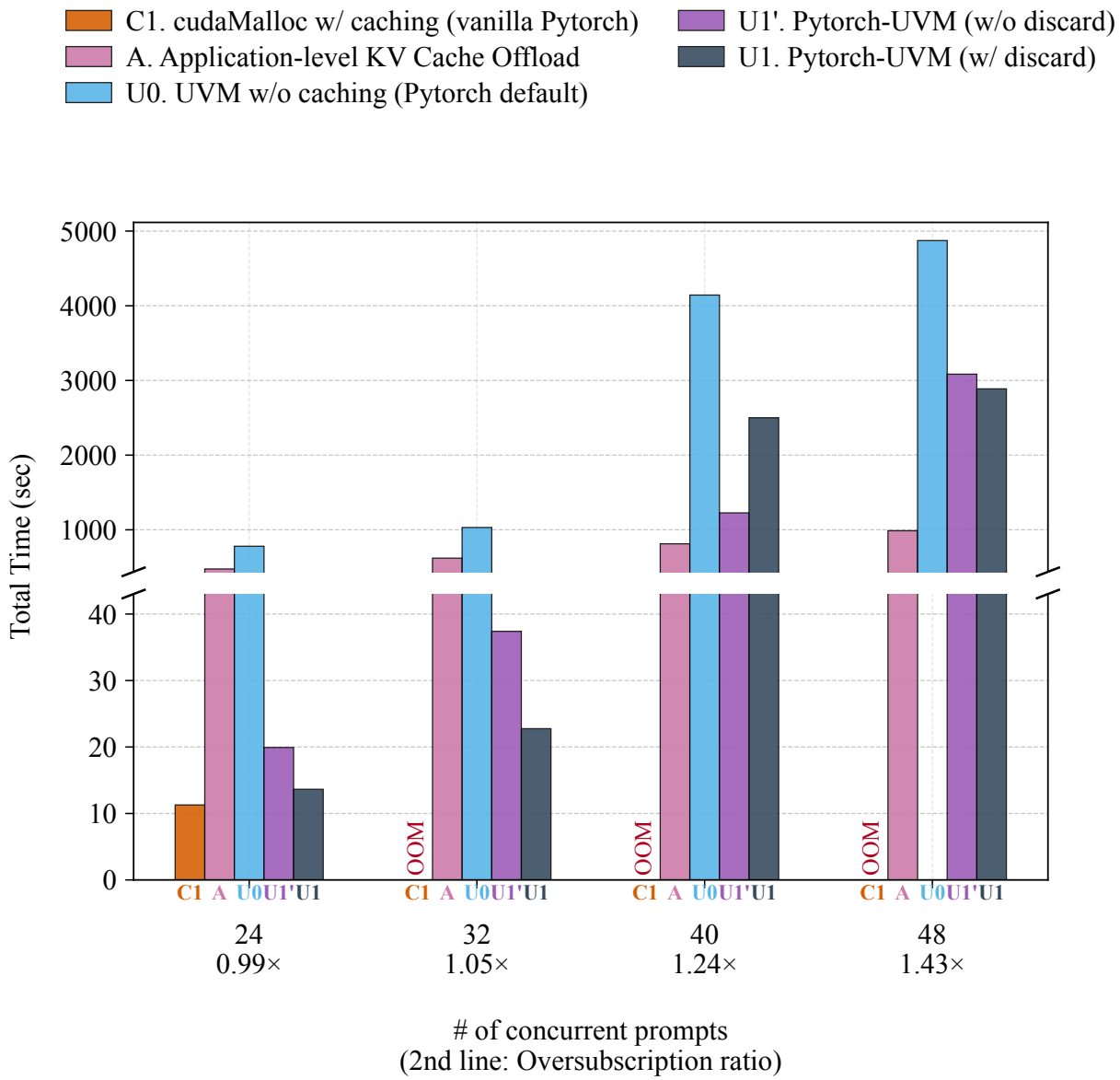


Figure 39: Effect of framework-guided eviction on H100 PCIe (4 KiB pages): total run time of OPT-13B for batch sizes spanning the oversubscription transition. U1 enables framework-guided eviction (implemented via UVMDiscard); U1' keeps the rest of PyTorch-UVM but evicts via copy-back to host. Reserved / active oversubscription ratios are bsz=24: 0.99× / 0.81×, bsz=32: 1.05× / 0.97×, bsz=40: 1.24× / 1.14×, bsz=48: 1.43× / 1.31×.

Trade-off of Framework-guided Eviction with UVMDiscard H100 PCIe.

Figure 39 isolates the contribution of framework-guided eviction (§4.2.2) by comparing

PyTorch-UVM with framework-guided eviction enabled (U1) against an otherwise-identical configuration that disables it and instead evicts pages by copying them back to host (U1'). The sweep covers the oversubscription transition on H100 PCIe. We distinguish two oversubscription ratios: *reserved* oversubscription is the size of the allocator's reserved memory pool relative to HBM, while *active* oversubscription is the size of the live working set (pages that PyTorch is actively using at any one time) relative to HBM. At bsz 24–32 the workload is reserved-oversubscribed (0.99× and 1.05×) but *active*-undersubscribed (0.81× and 0.97×), while at bsz 40–48 even the active working set exceeds HBM (1.14× and 1.31×). Vanilla PyTorch (C1), default UVM (U0), and application-level offload (A) are shown for reference.

Active oversub < 1 (bsz 24, 32). When the actively-used pages still fit on the GPU, our framework-guided eviction is a decisive win: U1 is 1.46× faster than U1' at bsz 24 and 1.64× faster at bsz 32. The pages that get evicted in this regime are exactly the ones that the allocator has marked as cached/freed (KV blocks for completed steps, scratch buffers, etc.), and PyTorch-UVM's allocator can tell UVM that those pages are safe to drop. This is the design intent of PyTorch-UVM: information that lives in the framework—which buffers are still semantically live—is normally invisible to the GPU MMU, and exposing it lets us avoid pure-overhead copy-backs to host. U1' loses this information, so even when active oversub is below 1 every eviction still pays the full PCIe round-trip, and the resulting end-to-end gap is roughly the volume of those wasted copies.

Active oversub > 1 (bsz 40, 48). Once the live working set itself exceeds HBM, the trend reverses or flattens: U1 (framework-guided eviction) is 2.04× slower than U1' at bsz 40, and the two variants are within 7% at bsz 48. The pages that PyTorch-UVM now evicts are no longer dominated by safely-discardable cached blocks; they include genuinely live data that is re-touched within the same forward pass. In this regime, framework-guided eviction provides no benefit over plain copy-back: the same thrashing occurs either way. Worse, at bsz 40 discard is not merely neutral but adds overhead of its

own. When a discarded block is later reused, PyTorch-UVM prepopulates it on the GPU to avoid page faults at kernel-execution time. At an active oversubscription ratio just above 1 (e.g., bsz 40), this produces a discard-reuse ping-pong: a page is discarded to reclaim space and then almost immediately back-populated onto the GPU for its next use, so the bandwidth discard was meant to save is instead spent shuttling the same page back and forth. Note that both variants nonetheless remain $\geq$1.6$\times$ faster than U0, and both are substantially slower than application-level offload (A) for the same reasons discussed in §4.3.2.

The takeaway is that framework-guided eviction pays off precisely when PyTorch holds framework-level information that UVM otherwise cannot see: namely, that some currently-resident pages are cached but inactive allocations rather than live data. This is the regime where PyTorch-UVM stays close to the in-memory baseline, and our design of passing framework-level information is what lets it close the last gap to C1. Once the active working set itself exceeds device memory that signal is no longer there to exploit.

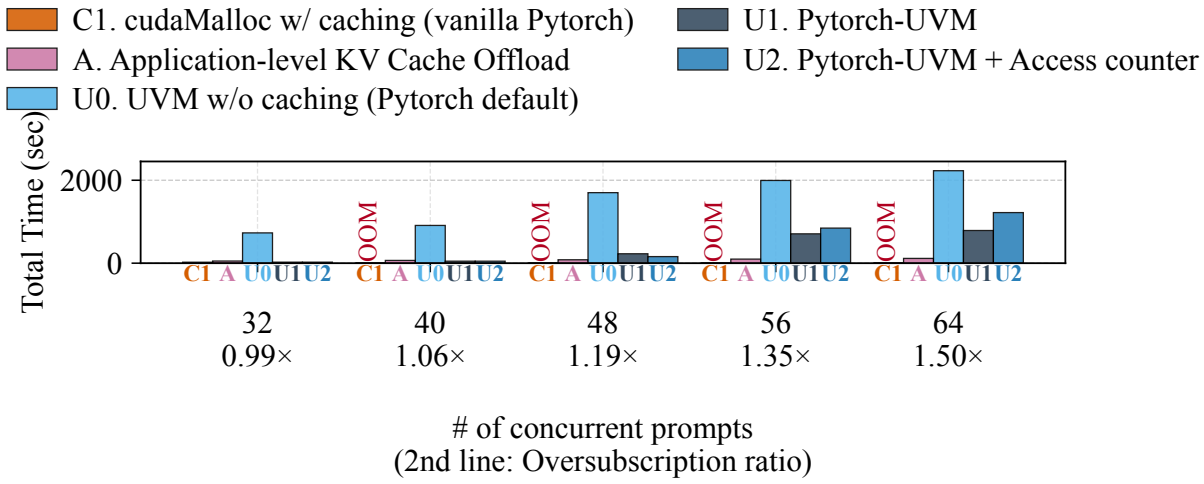


Figure 40: Effect of access-counter–driven placement on GH200 NVLink-C2C: total run time of OPT-13B for batch sizes 32–64 (0.99$\times$–1.50$\times$ oversubscribed). U1 is PyTorch-UVM without access counters; U2 enables access-counter–driven migration on top of U1.

Trade-off of Access Counters GH200 NVLink-C2C. We restrict this ablation to GH200 because access-counter–driven migration in the current NVIDIA driver is only supported for 64 KiB UVM-managed and system-allocated memory; the H100 PCIe path with 4 KiB pages does not expose access counters today, so an apples-to-apples comparison is not yet possible there.

Figure 40 compares PyTorch-UVM (U1) against an otherwise-identical configuration that additionally enables access-counter–driven migration (U2), over batch sizes 32–64 (0.99×–1.50× reserved oversubscription). The effect of access counters is non-monotonic in oversubscription pressure and breaks into three regimes.

Before oversubscription (bsz 32). Access counters add no measurable overhead in the unoversubscribed regime: U1 and U2 are within noise of each other at bsz 32, matching the in-memory baseline (C1). Counter-driven migration is only triggered when pages cross the migration threshold, so when no pages are being faulted in cold from host the mechanism is effectively dormant.

Onset of thrashing (bsz 40, 48; 1.06–1.19×). At light active oversubscription access counters mitigate thrashing: U2 is 1.43× faster than U1. At this point the working set just exceeds HBM, so a small set of hot pages is being faulted in repeatedly; counters quickly identify those pages and pin them on the GPU instead of letting them be evicted again, which is exactly the access pattern that the mechanism is designed for.

Heavier thrashing (bsz 56, 64). As oversubscription grows further the trend reverses: U2 is up to 1.55× *slower* than U1 at bsz 56 and 64. Once a large fraction of the working set is hot, the counters cannot promote pages in time — by the construction of the driver's static threshold, a page must be touched several times from the GPU before it is migrated — so accesses still pay the full GPU-to-CPU round-trip, and the additional accounting and migration traffic becomes net overhead rather than net benefit.

The takeaway is that the absolute benefit of access counters depends sharply on the working-set profile, even within a single workload (OPT-13B inference) on a single

platform. The current UVM driver uses a fixed migration threshold, which is well-tuned for the bsz-48 regime but is too conservative under heavier thrashing and pays for itself only narrowly. A dynamic threshold that adapts to the observed fault rate and reuse distance would let the mechanism remain helpful at higher oversubscription ratio.

Trade-off of No migration H100 PCIe. The thrashing we observed for U1 in §4.3.3 is a direct consequence of the default UVM migration policy: whenever the GPU touches a non-resident page, the driver eagerly faults it in and, if necessary, evicts another page to make room. Once the active working set exceeds HBM, every fault-in forces a follow-up eviction, with too little reuse in between to amortize the PCIe round-trip. Configuration U3 keeps the rest of PyTorch-UVM unchanged but disables on-demand migration, so any GPU reference to a non-resident page is served *in place* over PCIe (zero-copy). Figure 41 compares U1 and U3 across the oversubscription transition. The effect breaks into three regimes.

Active oversub < 1 (bsz 32). When the live working set still fits on the GPU, disabling migration is strictly detrimental: U3 is 1.54× slower than U1. There is no thrashing to avoid: framework-guided eviction already lets the allocator keep only the live pages resident and silently drop cached/freed ones, so U1's placement is essentially optimal. U3 just replaces each one-time page-in with a per-access PCIe round-trip, most visibly in prefill, where U3's throughput is ∼2.5× lower than U1's.

Moderate oversub (bsz 40, 48). Once the active working set just exceeds HBM, disabling migration is a decisive win: U3 is 10.5× faster than U1 at bsz 40 and 6.94× faster at bsz 48. Here U1's eager fault-in becomes counterproductive — a page is migrated in, displaces another page that is touched again shortly after, and the displaced page is faulted back in — so most of the wall time is spent on PCIe traffic that produces no useful residency. U3 short-circuits the cycle by leaving those pages on host, trading higher per-access latency for the elimination of eviction churn.

Heavier thrashing (bsz 56, 64). As oversubscription grows further the working set itself grows, and accessing all of it over PCIe stops being viable: U3 fails to complete within the 3-hour timeout at bsz 56 and 64, while U1 still completes. With 1.48×–1.65× active oversubscription, every layer streams through hundreds of MiB of weights and KV per step; if none of that is allowed to migrate, each step is bottlenecked end-to-end on a ∼25 GB/s PCIe Gen-4 link. U1 can at least keep some hot pages resident and amortize their fault-in across many accesses, so even with heavy fault traffic it beats raw zero-copy by more than an order of magnitude.

The takeaway is that whether migration helps or hurts depends on how much of the working set is hot and how fast the host link is. On PCIe Gen-4 H100 the eager-migration default is right at low and very high oversubscription, but is exactly wrong in the moderate middle (bsz 40, 48), where in-place access avoids the eviction-and-refault cycle and recovers up to 10.5×. We expect this picture to shift on GH200, whose ∼450 GB/s/dir NVLink-C2C link makes in-place access far cheaper and should extend the regime where U3 beats U1; a full GH200 sweep is left to future work.

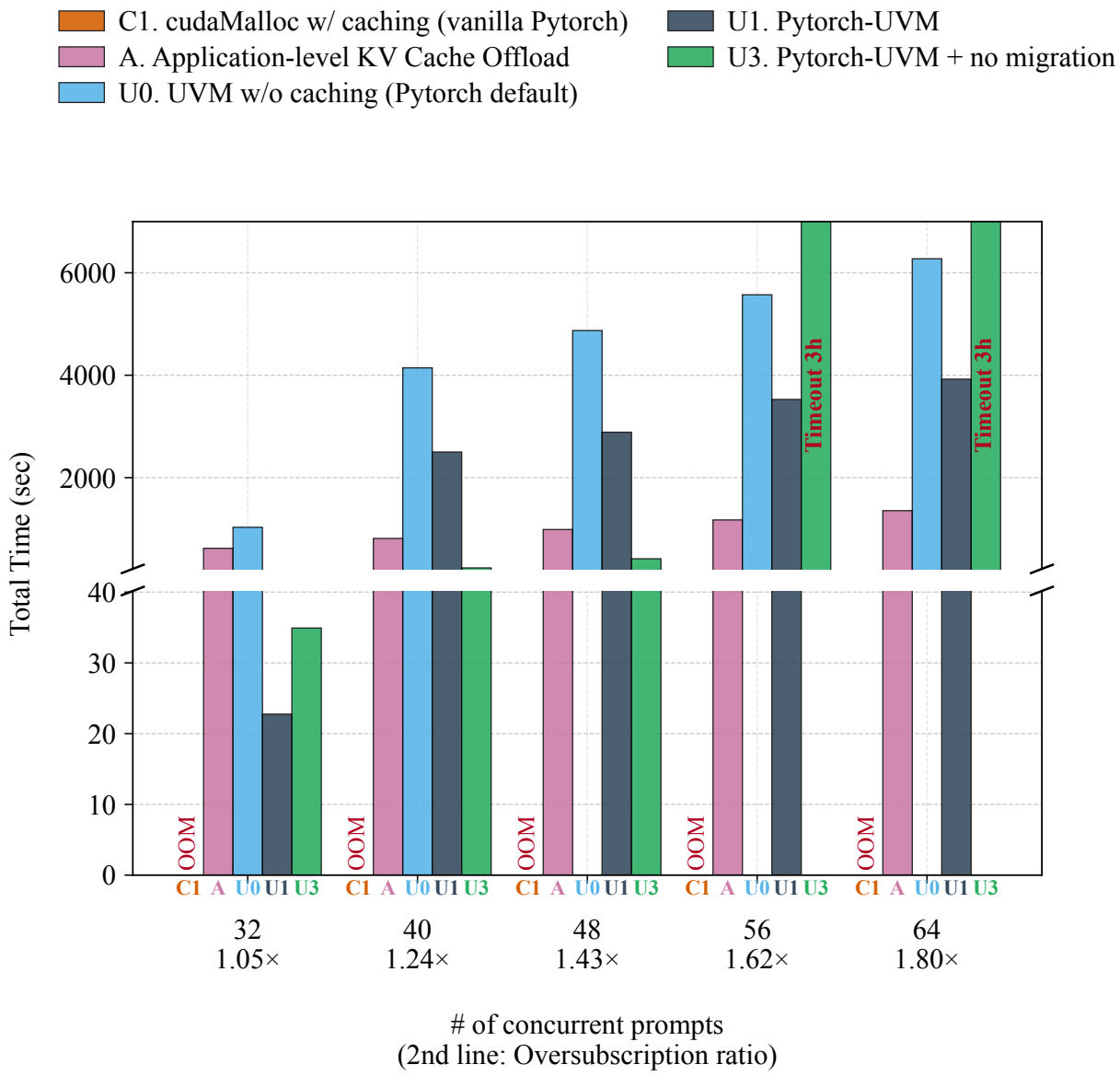


Figure 41: Effect of disabling migration on H100 PCIe (4 KiB pages): total run time of OPT-13B for batch sizes 32–64. U1 is PyTorch-UVM with on-demand migration; U3 is the same configuration with migration disabled, so that pages are accessed in place over PCIe (zero-copy). Reserved / active oversubscription ratios are bsz=32: 1.05× / 0.97×, bsz=40: 1.24× / 1.14×, bsz=48: 1.43× / 1.31×, bsz=56: 1.62× / 1.48×, bsz=64: 1.80× / 1.65×. Bars at 10,800 s indicate runs that hit the 3-hour timeout.

4.3.4 System-Allocated Memory (SAM)

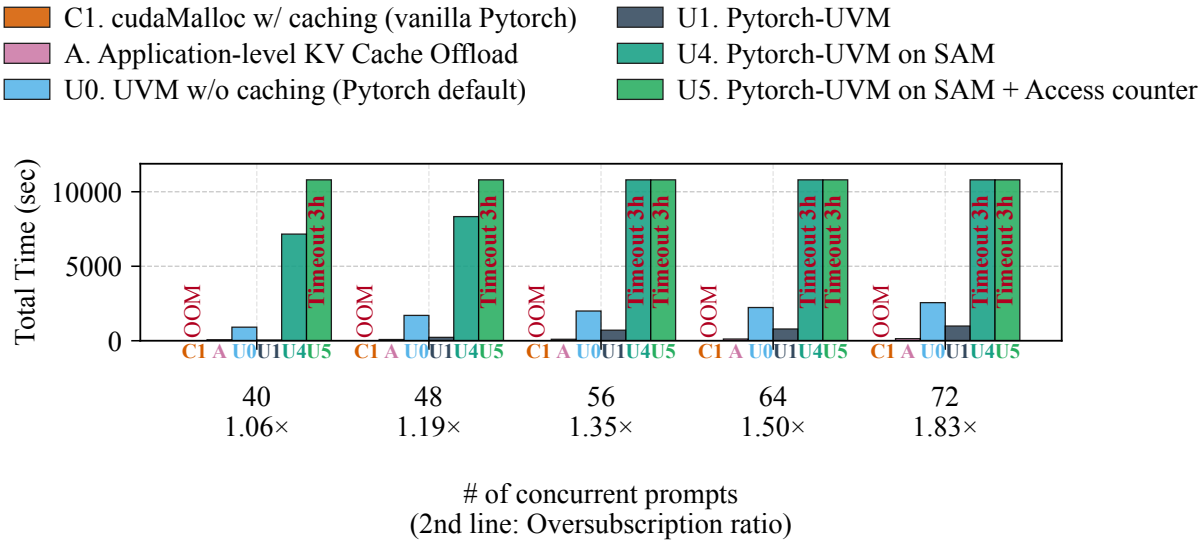


Figure 42: Performance of PyTorch-UVM on System-Allocated Memory (SAM) on GH200 NVLink-C2C, OPT-13B, batch sizes 8–72. U4 uses ordinary `malloc` memory accessed via ATS. U5 adds access-counter–based migration to the SAM path. Baselines C1, A, U0, and U1 (managed memory) are shown for reference.

ATS-Based SAM (GH200, 64 KiB) We evaluate PyTorch-UVM on System-Allocated Memory (SAM) on the GH200 NVLink-C2C system, where the GPU accesses ordinary `malloc`'d host pages via Address Translation Services (ATS). Figure 42 compares the SAM-backed PyTorch-UVM (U4) against the `cudaMallocManaged`-backed version (U1) and other baselines. When memory is not oversubscribed (batch sizes 8–32), the SAM path (U4) performs identically to the managed-memory path (U1), matching vanilla PyTorch (C1) and delivering ∼2500–3000 e2e tokens/s. This confirms that ATS-based SAM introduces no overhead for resident working sets.

However, as soon as the active working set exceeds GPU memory (batch size 40 and beyond), the SAM path's performance collapses. At batch size 40 (active oversubscription 1.06×), U4 is 149× slower than U1. At batch sizes 56 and above, U4 times out after 3 hours (>10,800 s). Adding access-counter–based migration to the SAM path (U5) fails to rescue performance; it times out at batch size 40 and beyond.

The severe degradation under oversubscription stems from the lack of UVM-aware optimizations on the SAM path. Unlike `cudaMallocManaged`, SAM allocations currently do not support UVMDiscard, and migration bandwidth is also limited on ATS-based SAM.

HMM-Based SAM (H100) We port PyTorch-UVM to the HMM-based SAM path on H100 PCIe, which lets the GPU access ordinary `malloc`'d host pages via Linux's Heterogeneous Memory Management rather than going through `cudaMallocManaged`. On the same OPT-13B / 16-concurrent-prompt configuration used in §4.3.2, the HMM-backed run takes *over 3 hours* to complete a single end-to-end inference — more than 1200× slower than PyTorch-UVM on top of `cudaMallocManaged`.

The order-of-magnitude gap reflects the cost of HMM's per-fault path on H100 PCIe. Every device access to a host page is resolved through a Linux MMU-notifier callback into the NVIDIA driver, which migrates the page over PCIe one fault at a time and offers no equivalent of the `cudaMemAdvise` / prefetch / discard hints that PyTorch-UVM's allocator uses to amortize and avoid faults on the `cudaMallocManaged` path. As a result, the HMM path on H100 today is functional but not yet competitive with managed-memory UVM, and we report this configuration primarily to characterize the gap between the two SAM mechanisms rather than as a performance recommendation.

4.3.5 Comparison with cudaMempool

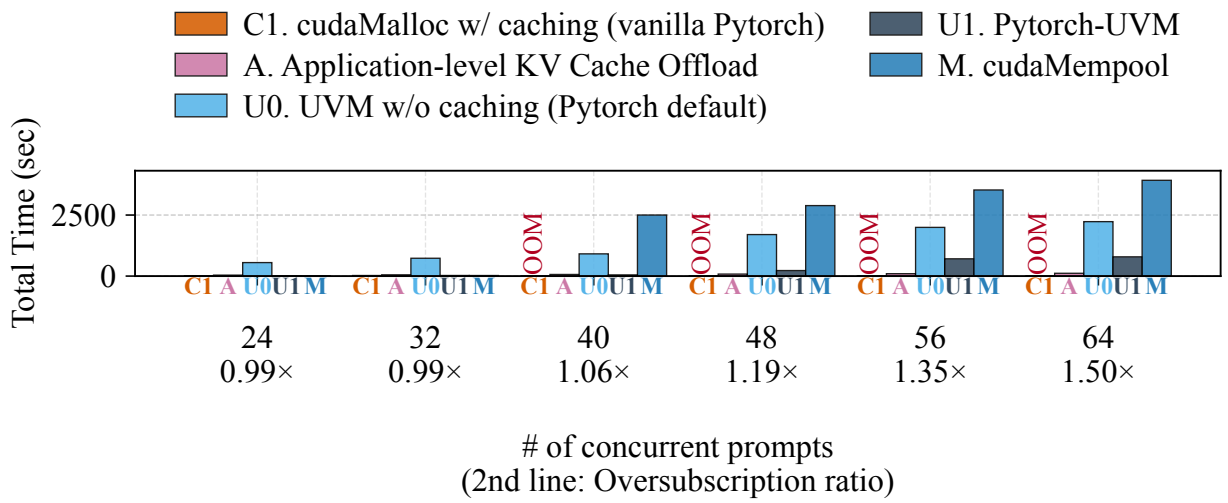


Figure 43: Comparison of PyTorch-UVM (U1) and a cudaMempool-based allocator (M) on GH200 NVLink-C2C, OPT-13B, batch sizes 24–64 (0.99×–1.50× reserved oversubscription). Both allocators sit on top of `cudaMallocManaged`; baselines C1, A, and U0 are shown for reference. At bsz 72 (1.83×) cudaMempool fails with a host-side OOM and is therefore omitted from the rightmost point.

CUDA's mempool API is the closest in-tree alternative to PyTorch-UVM's caching layer — runtime-level allocator caching, applicable to any CUDA application, and backable by either `cudaMalloc` or `cudaMallocManaged`. We compare against the most-favorable configuration (a `cudaMallocManaged`-backed mempool) on GH200 NVLink-C2C using the same OPT-13B sweep as in §4.3.2.

Before oversubscription (bsz 24, 32), PyTorch-UVM (U1) and cudaMempool (M) are essentially equivalent (within noise at bsz 24 and 32). Once the workload becomes oversubscribed, however, the two diverge sharply: at bsz 40 (1.06× oversubscribed) cudaMempool is 52× slower than PyTorch-UVM, the gap stays large through bsz 64 (5.0×), and at bsz 72 (1.83×) cudaMempool exhausts host memory and OOMs entirely while PyTorch-UVM still completes. The cause is that cudaMempool retains pooled allocations indefinitely and never returns the underlying UVM-managed pages to the driver, so its reserved working set grows monotonically and the driver is forced to handle

a much larger working set than the real active footprint. The host-OOM at bsz 72 is the same failure mode pushed to its limit.

This is not a fundamental limitation of the mempool design. We believe that a mempool implementation with PyTorch-UVM-style active garbage collection could recover most of PyTorch-UVM's performance while still being a general-purpose allocator for any CUDA application.

4.4 Discussion

4.4.1 Thrashing Mitigation: App-Aware UVM Eviction

As demonstrated in our evaluation, thrashing remains a primary bottleneck for UVM under heavy oversubscription. A root cause of this thrashing is the semantic gap between the UVM driver's eviction policy and the framework's user-space caching allocator. The driver typically evicts pages based on hardware-level access history or allocation order, which frequently conflicts with logical reuse: a block allocated via `cudaMallocManaged` long ago might be logically hot because the caching allocator just reassigned it to a new tensor. To fully mitigate thrashing, the UVM driver needs eviction policies that are informed by application-level usage patterns and logical block lifetimes.

4.4.2 Thrashing Mitigation: Utilization of App Hints

Existing application-level offloading frameworks (e.g., for KV caches and model weights) already compute detailed schedules for when data will be needed next. Rather than using these schedules to manually orchestrate data movement, frameworks could pass them down as hints to improve the UVM driver's eviction and prefetching policies. While relying on application hints may seem contrary to UVM's goal of transparent memory management, it strikes a pragmatic balance: the application provides high-level guidance on data lifetimes, and the driver retains the responsibility of executing the physical migrations and managing the hardware page tables.

4.4.3 Multiple GPU and Cross-Node Support

Existing PyTorch-UVMis based on single GPU attached to single CPU node. In practice, multi-GPU system is a hard requirement for AI workloads in production. We need to test UVM and extend PyTorch-UVMto support multi-GPU system. Existing Pytorch manages memory for each GPU separately; in other words, each GPU has its own caching allocator that is unaware of other GPUs.

For UVM, the pattern will change since every GPU can access the memory.

4.4.4 Using CUDA Graphs to Hint UVM Migration

UVM suffers from a lack of application-level information: the driver reacts to page faults after they occur, with no advance knowledge of which pages the next kernel will touch. CUDA Graphs offer a natural way to close this gap. Rather than launching kernels one at a time, a CUDA Graph captures an entire sequence of kernels ahead of time and replays the whole graph on each invocation; since DNN training and inference are highly repetitive, the access pattern of one replay closely predicts the next. We can exploit this with a record-and-replay approach to migration: during the first execution of a captured graph, the runtime records the page faults and resulting migrations to build a per-graph access trace, and on subsequent replays the driver consults this trace to prefetch pages before they fault and to evict pages that are dead rather than those reused later in the graph. This supplies the driver with the logical-lifetime information called for in Sections 4.4.1 and 4.4.2, but derived automatically from graph capture rather than explicit application annotations.

4.4.5 Concluding Remarks

LLM inference is increasingly bound by GPU memory, and today each framework answers this with its own hand-tuned offloading code. UVM promises to retire that machinery with transparent oversubscription, but has long been dismissed as too slow for

production. PyTorch-UVM shows that this gap is not fundamental: by propagating application-level intent down to the driver through a UVM-aware caching allocator and framework-guided eviction, it turns UVM into a practical substrate for PyTorch, matching the native `cudaMalloc` allocator when memory fits and running to completion—often far faster than default UVM—where the native allocator simply crashes. A performance gap to hand-tuned offloading remains under heavy oversubscription, but our thrashing-mitigation study and the directions above chart a concrete path toward closing it, and we hope PyTorch-UVM encourages frameworks to treat driver-managed memory as a first-class citizen rather than a last resort.

Chapter 5 Conclusion

This dissertation argues that designing memory management systems with deep understanding of hardware capabilities and workload characteristic can eliminate performance bottlenecks without compromising security or extensibility.

We identified three manifestations of the bottlenecks and addressed each with a targeted system. First, address translation has become a major performance bottleneck on terabyte-scale servers, yet commodity OS kernels remain hardwired to radix-tree page tables, blocking the faster translation architectures that would relieve it. EMT gives Linux an extensible, architecture-neutral interface for memory translation, enabling OS support for radically different page table structures without modifying the kernel for each new design; using it, we port ECPT and FPT—two recent hardware schemes for fast translation—to Linux and reveal OS-level challenges invisible to hardware-only evaluation. Second, strict I/O memory protection in virtualized environments incurs 83–95% throughput degradation from per-DMA VM exits, forcing production deployments to disable it entirely. vFree redesigns the Linux IOMMU invalidation protocol around one-for-many invalidations and deferred free page, amortizing VM exit costs while preserving strict security guarantees, and achieves performance within 1–8% of an unprotected system—a 74–147× improvement over the state of the art. Third, GPU memory oversubscription for AI workloads is unaddressed by the CUDA driver, leaving each framework to hand-write its own offloading logic or suffer a more than 5× penalty from default UVM. PyTorch-UVM integrates a UVM-aware caching allocator and framework-guided eviction directly into PyTorch, enabling practical oversubscription for LLM inference without modifying application code; under oversubscription it is 45×–58× faster than default UVM and 27×–35× faster than hand-written KV-cache offloading.

Across all three, a common structure emerges. Each begins with a mature, widely deployed abstraction—Linux's radix-tree paging, the Linux IOMMU subsystem, and CUDA Unified Virtual Memory—that is correct and general but was designed under

assumptions that no longer hold at scale. Closing the resulting semantic gap does not require replacing the existing stack: in each case the fix is surgical, identifying the specific invariant that the legacy design enforces unnecessarily and relaxing it while retaining the surrounding system. Realizing this required cross-layer evaluation, because each bottleneck is created not by hardware but by a software layer—the OS overhead that simulation models assume constant, the IOMMU datapath that serializes VM exits triggered by invalidation, and lack of framework support for UVM.

Today's computing stacks pair increasingly heterogeneous hardware with increasingly demanding workloads, and memory management systems must evolve to bridge them—serving the needs of modern workloads while fully exploiting the capabilities of modern hardware. This dissertation demonstrates that redesigning the memory management systems to be hardware- and workload-aware is a high-leverage path to unlocking the full potential of modern hardware.

References

[1] Redis Ltd., *Redis*, `https://redis.io`, Accessed: 2026-03-19, 2026.

[2] NVIDIA Corporation, *NVIDIA ConnectX-7 400G Adapters*, `https://www.nvidia.com/en-us/networking/ethernet/connectx-7/`, Product datasheet; accessed 2026-05-15, 2024.

[3] A. Grattafiori, A. Dubey, A. Jauhri, et al., "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024. [Online]. Available: `https://arxiv.org/abs/2407.21783`

[4] A. Basu, J. Gandhi, J. Chang, M. D. Hill, and M. M. Swift, "Efficient Virtual Memory for Big Memory Servers," in *Proceedings of the 40th Annual International Symposium on Computer Architecture (ISCA-40)*, Jun. 2013.

[5] S. Gupta, A. Bhattacharyya, Y. Oh, A. Bhattacharjee, B. Falsafi, and M. Payer, "Rebooting Virtual Memory with Midgard," in *Proceedings of the 48th Annual International Symposium on Computer Architecture (ISCA-48)*, Jun. 2021.

[6] Y. Kwon, H. Yu, S. Peter, C. J. Rossbach, and E. Witchel, "Coordinated and Efficient Huge Page Management with Ingens," in *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI '16)*, Nov. 2016.

[7] A. Margaritov, D. Ustiugov, E. Bugnion, and B. Grot, "Prefetched Address Translation," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-52)*, Oct. 2019.

[8] D. Skarlatos, A. Kokolis, T. Xu, and J. Torrellas, "Elastic Cuckoo Page Tables: Rethinking Virtual Memory Translation for Parallelism," in *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '20)*, Mar. 2020.

[9] I. Yaniv and D. Tsafrir, "Hash, Don't Cache (the Page Table)," in *Proceedings of the 2016 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems (SIGMETRICS '16)*, Jun. 2016.

[10] J. Stojkovic, D. Skarlatos, A. Kokolis, T. Xu, and J. Torrellas, "Parallel Virtualized Memory Translation with Nested Elastic Cuckoo Page Tables," in *Proceedings of the 27th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '22)*, Feb. 2022.

[11] C. H. Park, I. Vougioukas, A. Sandberg, and D. Black-Schaffer, "Every Walk's a Hit: Making Page Walks Single-Access Cache Hits," in *Proceedings of the 27th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '22)*, Feb. 2022.

[12] H. Alam, T. Zhang, M. Erez, and Y. Etsion, "Do-It-Yourself Virtual Memory Translation," in *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA-44)*, Jun. 2017.

[13] K. Kanellopoulos, R. Bera, K. Stojiljkovic, F. N. Bostanci, C. Firtina, R. Ausavarungnirun, R. Kumar, N. Hajinazar, M. Sadrosadati, N. Vijaykumar, and O. Mutlu, "Utopia: Fast and Efficient Address Translation via Hybrid Restrictive & Flexible Virtual-to-Physical Address Mappings," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-56)*, Oct. 2023.

[14] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32 (NeurIPS '19)*, 2019.

[15] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*, 2023.

[16] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He, "DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters," in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '20)*, 2020.

[17] W. Lee, J. Lee, J. Seo, and J. Sim, "InfiniGen: Efficient generative inference of large language models with dynamic KV cache management," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, Santa Clara, CA: USENIX Association, 2024, pp. 155–172.

[18] L. Torvalds, *Re: [Lse-tech] Re: 10.31 second kernel compile*, `https://yarchive.net/comp/linux/page_tables.html`, Mar. 2002.

[19] S. Chai, J. Zhang, J. Kim, A. Wang, F. Chung, J. Stojkovic, W. Jia, D. Skarlatos, J. Torrellas, and T. Xu, "EMT: An OS Framework for New Memory Translation Architectures," in *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI'25)*, Jul. 2025.

[20] J. Barr, *EC2 High Memory Update - New 18 TB and 24 TB Instances*, `https://aws.amazon.com/blogs/aws/ec2-high-memory-update-new-18-tb-and-24-tb-instances/`, Oct. 2019.

[21] Google Cloud, *Memory-optimized machine family for Compute Engine*, `https://cloud.google.com/compute/docs/memory-optimized-machines`, Apr. 2024.

[22] H. Li, D. S. Berger, L. Hsu, D. Ernst, P. Zardoshti, S. Novakovic, M. Shah, S. Rajadnya, S. Lee, I. Agarwal, M. D. Hill, M. Fontoura, and R. Bianchini,

"Pond: CXL-Based Memory Pooling Systems for Cloud Platforms," in *Proceedings of the 28th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '23)*, Mar. 2023.

[23] D. D. Sharma, "Compute Express Link (CXL): Enabling Heterogeneous Data-Centric Computing With Heterogeneous Memory Hierarchy," *IEEE Micro*, vol. 43, no. 2, pp. 99–109, Mar. 2023.

[24] *Intel® 64 and IA-32 Architectures Developer's Manual*, `https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html`, Mar. 2024.

[25] *AMD64 Architecture Programmer's Manual*, `https://www.amd.com/content/dam/amd/en/documents/processor-tech-docs/programmer-references/40332.pdf`, Jun. 2023.

[26] *Arm® Architecture Reference Manual for A-profile architecture*, `https://developer.arm.com/documentation/ddi0487/latest/`, Mar. 2024.

[27] *5-Level Paging and 5-Level EPT White Paper*, `https://software.intel.com/content/www/us/en/develop/download/5-level-paging-and-5-level-ept-white-paper.html`, May 2017.

[28] H. Ye, *Introduction to 5-Level Paging in 3rd Gen Intel Xeon Scalable Processors with Linux*, `https://lenovopress.lenovo.com/lp1468.pdf`, May 2021.

[29] R. Bhargava, B. Serebrin, F. Spadini, and S. Manne, "Accelerating Two-Dimensional Page Walks for Virtualized Systems," in *Proceedings of the 13th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-XIII)*, Mar. 2008.

[30] T. Merrifield and H. R. Taheri, "Performance Implications of Extended Page Tables on Virtualized x86 Processors," in *Proceedings of the 12th ACM*

SIGPLAN/SIGOPS International Conference on Virtual Execution Environments (VEE '16), Apr. 2016.

[31] K. Gosakan, J. Han, W. Kuszmaul, I. N. Mubarek, N. Mukherjee, K. Sriram, G. Tagliavini, E. West, M. A. Bender, A. Bhattacharjee, A. Conway, M. Farach-Colton, J. Gandhi, R. Johnson, S. Kannan, and D. E. Porter, "Mosaic Pages: Big TLB Reach with Small Pages," in *Proceedings of the 28th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '23)*, Mar. 2023.

[32] J. Stojkovic, N. Mantri, D. Skarlatos, T. Xu, and J. Torrellas, "Memory-Efficient Hashed Page Tables," in *Proceedings of the 29th IEEE International Symposium on High-Performance Computer Architecture (HPCA-29)*, Feb. 2023.

[33] M. A. Bender, A. Bhattacharjee, A. Conway, M. Farach-Colton, R. Johnson, S. Kannan, W. Kuszmaul, N. Mukherjee, D. Porter, G. Tagliavini, J. Vorobyeva, and E. West, "Paging and the Address-Translation Problem," in *Proceedings of the 33rd ACM Symposium on Parallelism in Algorithms and Architectures (SPAA '21)*, Jul. 2021.

[34] O. Kwon, Y. Lee, J. Park, S. Jang, B. Tak, and S. Hong, "Distributed Page Table: Harnessing Physical Memory as an Unbounded Hashed Page Table," in *Proceedings of the 57th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-57)*, Oct. 2024.

[35] V. Karakostas, J. Gandhi, F. Ayar, A. Cristal, M. D. Hill, K. S. McKinley, M. Nemirovsky, M. M. Swift, and O. Ünsal, "Redundant Memory Mappings for Fast Access to Large Memories," in *Proceedings of the 42rd Annual International Symposium on Computer Architecture (ISCA-42)*, Jun. 2015.

[36] J. Ahn, S. Jin, and J. Huh, "Revisiting Hardware-Assisted Page Walks for Virtualized Systems," in *Proceedings of the 39th Annual International Symposium on Computer Architecture (ISCA-39)*, Jun. 2012.

[37] G. Hoang, C. Bae, J. Lange, L. Zhang, P. Dinda, and R. Joseph, "A Case for Alternative Nested Paging Models for Virtualized Systems," *IEEE Computer Architecture Letters*, vol. 9, no. 1, pp. 17–20, Jan. 2010.

[38] S. Ghose, A. Boroumand, J. S. Kim, J. Gómez-Luna, and O. Mutlu, "Processing-in-memory: A workload-driven perspective," *IBM Journal of Research and Development*, vol. 63, no. 6, 3:1–3:19, Aug. 2019.

[39] J. Zhang, W. Jia, S. Chai, P. Liu, J. Kim, and T. Xu, "Direct Memory Translation for Virtualized Clouds," in *Proceedings of the 29th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '24)*, Apr. 2024.

[40] C. Alverti, S. Psomadakis, V. Karakostas, J. Gandhi, K. Nikas, G. Goumas, and N. Koziris, "Enhancing and Exploiting Contiguity for Fast Memory Virtualization," in *Proceedings of the 47th Annual International Symposium on Computer Architecture (ISCA-47)*, Jun. 2020.

[41] J. H. Ryoo, N. Gulur, S. Song, and L. K. John, "Rethinking TLB Designs in Virtualized Environments: A Very Large Part-of-Memory TLB," in *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA-44)*, Jun. 2017.

[42] R. Gooch and P. Enberg, *Overview of the Linux Virtual File System*, `https://www.kernel.org/doc/html/next/filesystems/vfs.html`, Feb. 2024.

[43] J. Corbet, *Five-level page tables*, `https://lwn.net/Articles/717293/`, Mar. 2017.

[44] T. W. Barr, A. L. Cox, and S. Rixner, "Translation Caching: Skip, Don't Walk (the Page Table)," in *Proceedings of the 37th Annual International Symposium on Computer Architecture (ISCA-37)*, Jun. 2010.

[45] A. Bhattacharjee, "Large-reach Memory Management Unit Caches," in *Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-46)*, Dec. 2013.

[46] J. Navarro, S. Iyer, P. Druschel, and A. Cox, "Practical, Transparent Operating System Support for Superpages," in *Proceedings of the 5th Symposium on Operating Systems Design and Implementation (OSDI '02)*, Dec. 2002.

[47] J. Gandhi, M. D. Hill, and M. M. Swift, "Agile Paging: Exceeding the Best of Nested and Shadow Paging," in *Proceedings of the 43nd Annual International Symposium on Computer Architecture (ISCA-43)*, Jun. 2016.

[48] I. Vougioukas, *How about a short walk?* `https://community.arm.com/arm-research/b/articles/posts/how-about-a-short-walk`, ARM Blogs, Mar. 2022.

[49] C. Dougan, P. Mackerras, and V. Yodaiken, "Optimizing the Idle Task and Other MMU Tricks," in *Proceedings of the 3rd Symposium on Operating Systems Design and Implementation (OSDI '99)*, Feb. 1999.

[50] J. Huck and J. Hays, "Architectural Support For Translation Table Management In Large Address Space Machines," in *Proceedings of the 20th Annual International Symposium on Computer Architecture (ISCA-20)*, May 1993.

[51] M. Talluri, M. D. Hill, and Y. A. Khalid, "A New Page Table for 64-bit Address Spaces," in *Proceedings of the 15th ACM Symposium on Operating Systems Principles (SOSP '95)*, Dec. 1995.

[52] C. Gray, M. Chapman, P. Chubb, D. Mosberger-Tang, and G. Heiser, "Itanium – A System Implementor's Tale," in *Proceedings of the 2005 USENIX Annual Technical Conference (USENIX '05)*, Apr. 2005.

[53] R. Pagh and F. F. Rodler, "Cuckoo Hashing," *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, May 2004.

[54] Z. Yan, D. Lustig, D. Nellans, and A. Bhattacharjee, "Nimble Page Management for Tiered Memory Systems," in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '19)*, Apr. 2019.

[55] M. Accetta, R. Baron, W. Bolosky, D. Golub, R. Rashid, A. Tevanian, and M. Young, "Mach: A New Kernel Foundation For UNIX Development," in *Proceedings of the 1986 Summer USENIX Technical Conference (USENIX Summer '86)*, Jul. 1986.

[56] R. Rashid, A. Tevanian, M. Young, D. Golub, R. Baron, D. Black, W. Bolosky, and J. Chew, "Machine-Independent Virtual Memory Management for Paged Uniprocessor and Multiprocessor Architectures," in *Proceedings of the Second International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-II)*, Oct. 1987.

[57] C. D. Cranor and G. M. Parulkar, "The UVM Virtual Memory System," in *Proceedings of the 1999 USENIX Annual Technical Conference (USENIX '99)*, Jun. 1999.

[58] *Intel® IA-64 Architecture Software Developer's Manual*, `http://refspecs.linux-foundation.org/IA64-softdevman-vol2.pdf`, Jul. 2000.

[59] *Power ISA™ Version 3.1B*, `https://openpowerfoundation.org/specifications/isa/`, Sep. 2021.

[60] T. E. Anderson, H. M. Levy, B. N. Bershad, and E. D. Lazowska, "The Interaction of Architecture and Operating System Design," in *Proceedings of the*

Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-IV), Apr. 1991.

[61] K. A. Shutemov, *mm: convert generic code to 5-level paging*, `https://github.com/torvalds/linux/commit/c2febafc67734a62196c1b9dfba926412d4077ba`, Mar. 2017.

[62] *Machine address mapping definitions – machine-independent section*, `https://github.com/freebsd/freebsd-src/blob/release/14.0.0/sys/vm/pmap.h`, Aug. 2023.

[63] H. M. Levy and P. H. Lipman, "Virtual Memory Management in the VAX/VMS Operating System," *IEEE Computer*, vol. 15, no. 3, pp. 35–41, Mar. 1982.

[64] J. Huang, M. K. Qureshi, and K. Schwan, "An Evolutionary Study of Linux Memory Management for Fun and Profit," in *Proceedings of the 2016 USENIX Annual Technical Conference (USENIX ATC '16)*, Jun. 2016.

[65] B. Tabatabai, J. Sorenson, and M. M. Swift, "FBMM: Making Memory Management Extensible With Filesystems," in *Proceedings of the 2024 USENIX Annual Technical Conference (USENIX ATC '24)*, Jul. 2024.

[66] A. Raybuck, T. Stamler, W. Zhang, M. Erez, and S. Peter, "HeMem: Scalable Tiered Memory Management for Big Data Applications and Real NVM," in *Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP '21)*, Oct. 2021.

[67] S. Jalalian, S. Patel, M. R. Hajidehi, M. Seltzer, and A. Fedorova, "ExtMem: Enabling Application-Aware Virtual Memory Management for Data-Intensive Applications," in *Proceedings of the 2024 USENIX Annual Technical Conference (USENIX ATC '24)*, Jul. 2024.

[68] D. R. Engler, S. K. Gupta, and M. F. Kaashoek, "AVM: Application-Level Virtual Memory," in *Proceedings of the 5th Workshop on Hot Topics in Operating Systems (HotOS-V)*, May 1995.

[69] R. A. Gingell, J. P. Moran, and W. A. Shannon, "Virtual Memory Architecture in SunOS," in *Proceedings of the 1987 Summer USENIX Technical Conference (USENIX Summer '87)*, Jun. 1987.

[70] J. Woodruff, R. N. M. Watson, D. Chisnall, S. W. Moore, J. Anderson, B. Davis, B. Laurie, P. G. Neumann, R. Norton, and M. Roe, "The CHERI capability model: Revisiting RISC in an age of risk," in *Proceedings of the 41st Annual International Symposium on Computer Architecture (ISCA-41)*, Jun. 2014.

[71] *Memory Protection Keys*, `https://docs.kernel.org/core-api/protection-keys.html`, Oct. 2024.

[72] *Page migration*, `https://www.kernel.org/doc/html/next/mm/page_migration.html`, Aug. 2023.

[73] *Transparent Hugepage Support*, `https://docs.kernel.org/admin-guide/mm/transhuge.html`, Dec. 2023.

[74] N. Piggin, *page table iterators*, `https://lwn.net/Articles/124037/`, Feb. 2005.

[75] N. P. Carter, S. W. Keckler, and W. J. Dally, "Hardware Support for Fast Capability-based Addressing," Nov. 1994.

[76] *Page Attribute Table in Linux Kernel*, `https://docs.kernel.org/next/x86/pat.html`.

[77] *Memory Type Range Registers in Linux Kernel*, `https://docs.kernel.org/arch/x86/mtrr.html`.

[78] *Paging in OSDev Wiki*, `https://wiki.osdev.org/Paging`.

[79] B. Suchy, S. Ghosh, D. Kersnar, S. Chai, Z. Huang, A. Nelson, M. Cuevas, A. Bernat, G. Chaudhary, N. Hardavellas, S. Campanoni, and P. Dinda, "CARAT CAKE: Replacing Paging via Compiler/Kernel Cooperation," in *Proceedings of the 27th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '22)*, Feb. 2022.

[80] B. Suchy, S. Campanoni, N. Hardavellas, and P. Dinda, "CARAT: A Case for Virtual Memory through Compiler- and Runtime-Based Address Translation," in *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '20)*, Jun. 2020.

[81] *Paravirtualization in Linux Kernel*, `https://docs.kernel.org/virt/paravirt_ops.html`.

[82] *Pv_ops.mmu in linux kernel*, `https://elixir.bootlin.com/linux/v5.15/source/arch/x86/kernel/paravirt.c#L290`.

[83] *Linux kernel coding style - The Linux Kernel documentation*, `https://www.kernel.org/doc/html/v6.12/process/coding-style.html#the-inline-disease`, Sep. 2024.

[84] J. Corbet, *The final step for huge-page swapping*, `https://lwn.net/Articles/758677/`, Jul. 2018.

[85] *Split page table lock*, `https://www.kernel.org/doc/html/next/mm/split_page_table_lock.html`, Aug. 2023.

[86] P. Magnusson, M. Christensson, J. Eskilson, D. Forsgren, G. Hallberg, J. Hogberg, F. Larsson, A. Moestedt, and B. Werner, "Simics: A Full System Simulation Platform," *IEEE Computer*, vol. 35, no. 2, pp. 50–58, Feb. 2002.

[87] A. Rodrigues, K. S. Hemmert, B. W. Barrett, C. Kersey, R. Oldfield, M. Weston, R. Riesen, J. Cook, P. Rosenfeld, E. Cooper-Balis, and B. Jacob, "The Structural Simulation Toolkit," *ACM SIGMETRICS Performance Evaluation Review*, vol. 38, no. 4, pp. 37–42, Mar. 2011.

[88] *QEMU: A generic and open source machine emulator and virtualizer*, `http://www.qemu.org`, Jun. 2020.

[89] *Features/SoftMMU*, `https://wiki.qemu.org/Features/SoftMMU`, Jan. 2018.

[90] *QEMU TCG Plugins*, `https://www.qemu.org/docs/master/devel/tcg-plugins.html`, Feb. 2024.

[91] *Memory Management*, `https://www.kernel.org/doc/html/v5.15/x86/x86_64/mm.html`, Aug. 2021.

[92] *Page Table Isolation (PTI)*, `https://www.kernel.org/doc/html/next/x86/pti.html`, Jan. 2024.

[93] R. Balasubramonian, A. B. Kahng, N. Muralimanohar, A. Shafiee, and V. Srinivas, "CACTI 7: New Tools for Interconnect Exploration in Innovative Off-Chip Memories," *ACM Transactions on Architecture and Code Optimization (TACO)*, 2017.

[94] M. A. Bender, A. Conway, M. Farach-Colton, W. Kuszmaul, and G. Tagliavini, "Iceberg Hashing: Optimizing Many Hash-Table Criteria at Once," *Journal of the ACM*, vol. 70, no. 6, pp. 1–55, Nov. 2023.

[95] P. Zuo, Y. Hua, and J. Wu, "Write-Optimized and High-Performance Hashing Index Scheme for Persistent Memory," in *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI '18)*, Oct. 2018.

[96] D. Zhou, B. Fan, H. Lim, M. Kaminsky, and D. G. Andersen, "Scalable, High Performance Ethernet Forwarding with CuckooSwitch," in *Proceedings of the 9th ACM Conference on Emerging Networking Experiments and Technologies (CoNEXT '13)*, Dec. 2013.

[97] X. (Ren, K. Rodrigues, L. Chen, C. Vega, M. Stumm, and D. Yuan, "An Analysis of Performance Evolution of Linux's Core Operations," in *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP '19)*, Oct. 2019.

[98] L. Nai, Y. Xia, I. G. Tanase, H. Kim, and C.-Y. Lin, "GraphBIG: Understanding Graph Computing in the Context of Industrial Solutions," in *Proceedings of SC15: The International Conference for High Performance Computing, Networking, Storage and Analysis (SC '15)*, Nov. 2015.

[99] *GraphBIG Dataset*, `https://github.com/graphbig/graphBIG/wiki/GraphBIG-Dataset`, Mar. 2015.

[100] HPC Challenge Benchmark, *RandomAccess: GUPS (Giga Updates Per Second)*, `https://hpcchallenge.org/projectsfiles/hpcc/RandomAccess.html`, Aug. 2022.

[101] A. Kopytov, *SysBench: Scriptable database and system performance benchmark*, `https://github.com/akopytov/sysbench/tree/1.0.20`, Apr. 2020.

[102] *Linux Test Project*, `https://github.com/linux-test-project/ltp/tree/20230929`, Sep. 2023.

[103] D. L. Bruening, "Efficient, Transparent, and Comprehensive Runtime Code Manipulation," Ph.D. dissertation, Massachusetts Institute of Technology, Sep. 2004.

[104] B. Gregg, *CPU Flame Graphs*, `https://www.brendangregg.com/FlameGraphs/cpuflamegraphs.html`, Aug. 2021.

[105] *clang-query tool*, https://github.com/llvm/llvm-project/tree/main/clang-tools-extra/clang-query, Jan. 2024.

[106] *arch/x86/boot/compressed/head_64.S*, https://github.com/torvalds/linux/blob/v5.15/arch/x86/boot/compressed/head_64.S, Aug. 2021.

[107] *arch/x86/kernel/head64.c*, https://github.com/torvalds/linux/blob/v5.15/arch/x86/kernel/head64.c, May 2021.

[108] *arch/x86/kernel/cpu/common.c*, https://github.com/torvalds/linux/blob/v5.15/arch/x86/kernel/cpu/common.c, Oct. 2021.

[109] B. Pham, V. Vaidyanathan, A. Jaleel, and A. Bhattacharjee, "CoLT: Coalesced Large-Reach TLBs," in *Proceedings of the 45th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-45)*, Dec. 2012.

[110] B. Pham, A. Bhattacharjee, Y. Eckert, and G. H. Loh, "Increasing TLB Reach by Exploiting Clustering in Page Translations," in *Proceedings of the 20th IEEE International Symposium on High-Performance Computer Architecture (HPCA-20)*, Feb. 2014.

[111] C. H. Park, T. Heo, J. Jeong, and J. Huh, "Hybrid TLB Coalescing: Improving TLB Translation Coverage under Diverse Fragmented Memory Allocations," in *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA-44)*, Jun. 2017.

[112] T. W. Barr, A. L. Cox, and S. Rixner, "SpecTLB: A Mechanism for Speculative Address Translation," in *Proceedings of the 38th Annual International Symposium on Computer Architecture (ISCA-38)*, Sep. 2011.

[113] A. Panwar, A. Prasad, and K. Gopinath, "Making Huge Pages Actually Useful," in *Proceedings of the 23rd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '18)*, Mar. 2018.

[114] A. Panwar, S. Bansal, and K. Gopinath, "HawkEye: Efficient Fine-grained OS Support for Huge Pages," in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '19)*, Apr. 2019.

[115] W. Jia, J. Zhang, J. Shan, Y. Du, X. Ding, and T. Xu, "HugeGPT: Storing Guest Page Tables on Host Huge Pages to Accelerate Address Translation," in *Proceedings of the 32nd International Conference on Parallel Architectures and Compilation Techniques (PACT'23)*, Oct. 2023.

[116] K. Krueger, D. Loftesness, A. Vahdat, and T. Anderson, "Tools for the Development of Application Specific Virtual Memory Management," in *Proceedings of the Eighth Annual Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA '93)*, Sep. 1993.

[117] E. Abrossimov, M. Rozier, and M. Shapiro, "Generic Virtual Memory Management for Operating System Kernels," in *Proceedings of the 12th ACM Symposium on Operating Systems Principles (SOSP '89)*, Nov. 1989.

[118] B. N. Bershad, S. Savage, P. Pardyak, E. G. Sirer, M. E. Fiuczynski, D. Becker, C. Chambers, and S. Eggers, "Extensibility, Safety and Performance in the SPIN Operating System," in *Proceedings of the 15th ACM Symposium on Operating Systems Principles (SOSP '95)*, Dec. 1995.

[119] H. A. Maruf, H. Wang, A. Dhanotia, J. Weiner, N. Agarwal, P. Bhattacharya, C. Petersen, M. Chowdhury, S. Kanaujia, and P. Chauhan, "TPP: Transparent Page Placement for CXL-Enabled Tiered-Memory," in *Proceedings of the 28th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '23)*, Mar. 2023.

[120] S. Bergman, P. Faldu, B. Grot, L. Vilanova, and M. Silberstein, "Reconsidering OS Memory Optimizations in the Presence of Disaggregated Memory," in

Proceedings of the 2022 ACM SIGPLAN International Symposium on Memory Management (ISMM '22), Jun. 2022.

[121] K. Kanellopoulos, K. Sgouras, F. N. Bostanci, A. K. Kakolyris, B. K. Konar, R. Bera, M. Sadrosadati, R. Kumar, N. Vijaykumar, and O. Mutlu, "Virtuoso: Enabling Fast and Accurate Virtual Memory Research via an Imitation-based Operating System Simulation Methodology," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'25)*, Mar. 2025.

[122] N. Amit, M. Ben-Yehuda, D. Tsafrir, and A. Schuster, "vIOMMU: Efficient IOMMU Emulation," in *Proceedings of the 2011 USENIX Conference on USENIX Annual Technical Conference (USENIX ATC'11)*, Jun. 2011.

[123] B. Rubin, S. Agarwal, Q. Cai, and R. Agarwal, "Fast & Safe IO Memory Protection," in *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP'24)*, Nov. 2024.

[124] S. Agarwal, R. Agarwal, B. Montazeri, M. Moshref, K. Elmeleegy, L. Rizzo, M. A. De Kruijf, G. Kumar, S. Ratnasamy, D. Culler, and A. Vahdat, "Understanding Host Interconnect Congestion," in *Proceedings of the 21st ACM Workshop on Hot Topics in Networks (HotNets'22)*, Nov. 2022.

[125] M. Ben-Yehuda, J. Xenidis, M. Ostrowski, K. Rister, A. Bruemmer, and L. Van Doorn, "The Price of Safety: Evaluating IOMMU Performance," in *The Ottawa Linux Symposium*, 2007.

[126] M. Malka, N. Amit, M. Ben-Yehuda, and D. Tsafrir, "rIOMMU: Efficient IOMMU for I/O Devices that Employ Ring Buffers," in *Proceedings of the 20th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'15)*, Mar. 2015.

[127] M. Malka, N. Amit, and D. Tsafrir, "Efficient Intra-Operating System Protection Against Harmful DMAs," in *Proceedings of the 13th USENIX Conference on File and Storage Technologies (FAST'15)*, Feb. 2015.

[128] O. Peleg, A. Morrison, B. Serebrin, and D. Tsafrir, "Utilizing the IOMMU Scalably," in *Proceedings of the 2015 USENIX Annual Technical Conference (USENIX ATC'15)*, Jul. 2015.

[129] A. Markuze, A. Morrison, and D. Tsafrir, "True IOMMU Protection from DMA Attacks: When Copy is Faster than Zero Copy," in *Proceedings of the 21st International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'16)*, Apr. 2016.

[130] A. Markuze, I. Smolyar, A. Morrison, and D. Tsafrir, "DAMN: Overhead-Free IOMMU Protection for Networking," in *Proceedings of the 23rd International Conference on Architecture Support for Programming Languages and Operating Systems (ASPLOS'18)*, Mar. 2018.

[131] C. Rajapaksha, L. Delshadtehrani, R. Muri, M. Egele, and A. Joshi, "IOMMU Deferred Invalidation Vulnerability: Exploit and Defense," in *Proceedings of 2024 Design, Automation & Test in Europe Conference & Exhibition (DATE'24)*, Feb. 2024.

[132] M. Alex, S. Vargaftik, G. Kupfer, B. Pismeny, N. Amit, A. Morrison, and D. Tsafrir, "Characterizing, Exploiting, and Detecting DMA Code Injection Vulnerabilities in the Presence of an IOMMU," in *Proceedings of the 16th European Conference on Computer Systems (EuroSys'21)*, Apr. 2021.

[133] *Intel® Virtualization Technology for Directed I/O Architecture Specification*, `https://www.intel.com/content/www/us/en/content-details/774206/intel-virtualization-technology-for-directed-i-o-architecture-specification.html`, Version 5.1, 2023.

[134] *AMD I/O Virtualization Technology (IOMMU) Specification*, https://docs.amd.com/v/u/en-US/48882_3.10_PUB, Publication No. 48882, Rev 3.10, Feb 2025, 2025.

[135] Arm Limited, *ARM system memory management unit architecture specification (SMMUv3)*, https://developer.arm.com/documentation/ihi0070/latest/, Document IHI0070.

[136] Y. Wang, L. Chen, J. Ji, X. Tian, B. Luo, Z. Wei, Z. Huang, K. Xu, K. Peng, K. Guo, N. Luo, G. Wang, S. Dai, Y. Shen, J. Wu, and Z. Qi, "To pri or not to pri, that's the question," in *Proceedings of the 19th USENIX Conference on Operating Systems Design and Implementation*, Jul. 2025.

[137] K. Guo, D. Li, B. Luo, Y. Shen, K. Peng, N. Luo, S. Dai, C. Liang, J. Song, H. Yang, X. Zhang, and Z. Mi, "Vpri: Efficient i/o page fault handling via software-hardware co-design for iaas clouds," in *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, Nov. 2024.

[138] *`Dmar_domain.cache_lock`* — *linux iommu domain cache tag spinlock*, https://elixir.bootlin.com/linux/v6.12.9/source/drivers/iommu/intel/iommu.h#L622, Linux v6.12.9, Accessed: 2026-04-02.

[139] *`Q_inval.q_lock`* — *linux iommu invalidation queue raw spinlock*, https://elixir.bootlin.com/linux/v6.12.9/source/drivers/iommu/intel/iommu.h#L485, Linux v6.12.9, Accessed: 2026-04-02.

[140] *Intel® Virtualization Technology for Directed I/O Architecture Specification, Chapter 6: Caching Translation Information*, https://www.intel.com/content/www/us/en/content-details/774206/intel-virtualization-technology-for-directed-i-o-architecture-specification.html, Version 5.1, 2023.

[141] *Page_pool: Convert to use netmem*, https://github.com/torvalds/linux/commit/4dec64c52e24c2c9a15f81c115f1be5ea35121cb, Linux kernel commit 4dec64c, 2024.

[142] Y. Liu and Z. Duan, *QEMU with iommufd support and intel scalable IOV*, https://github.com/yiliu1765/qemu, Accessed: 2026-02-24, 2024.

[143] *iPerf3 – The TCP, UDP and SCTP Network Bandwidth Measurement Tool*, https://iperf.fr, Accessed: 2026-02-24.

[144] J. Shan, X. Ding, and N. H. Gehani, "APPLES: Efficiently Handling Spin-lock Synchronization on Virtualized Platforms," *IEEE Transactions on Parallel and Distributed Systems*, vol. 28, no. 7, pp. 1811–1824, Jul. 2017.

[145] S. Kashyap, C. Min, and T. Kim, "Opportunistic Spinlocks: Achieving Virtual Machine Scalability in the Clouds," in *Proceedings of the 6th ACM SIGOPS Asia-Pacific Workshop on Systems (APSys'15)*, Jul. 2015.

[146] X. Ding, P. B. Gibbons, M. A. Kozuch, and J. Shan, "Gleaner: Mitigating the Blocked-Waiter Wakeup Problem for Virtualized Multicore Applications," in *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC'14)*, Jun. 2014.

[147] S. Kashyap, C. Min, and T. Kim, "Scaling Guest OS Critical Sections with eCS," in *Proceedings of the 2018 USENIX Annual Technical Conference (USENIX ATC'18)*, Jul. 2018.

[148] *Intel® Scalable I/O Virtualization Technical Specification*, https://cdrdv2-public.intel.com/671403/intel-scalable-io-virtualization-technical-specification.pdf, Accessed: 2026-03-03, 2023.

[149] Z. Zhang, J. Chen, B. Ying, Y. Cao, L. Liu, J. Li, X. Zeng, J. Wang, W. Li, and H. Guan, "HD-IOV: SW-HW Co-designed I/O Virtualization with Scalability and

Flexibility for Hyper-Density Cloud," in *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys'24)*, 2024.

[150] *What is RCU? — Linux Kernel Documentation*, `https://www.kernel.org/doc/Documentation/RCU/whatisRCU.txt`, Accessed: 2026-03-31.

[151] *`ma_free_rcu` identifier - linux source code v6.12.9*, `https://elixir.bootlin.com/linux/v6.12.9/A/ident/ma_free_rcu`, Accessed: 2026-04-01.

[152] *`__Kfence_free` identifier - linux source code v6.12.9*, `https://elixir.bootlin.com/linux/v6.12.9/A/ident/__kfence_free`, Accessed: 2026-04-01.

[153] *`tlb_remove_table_free` identifier - linux source code v6.12.9*, `https://elixir.bootlin.com/linux/v6.12.9/A/ident/tlb_remove_table_free`, Accessed: 2026-04-01.

[154] *sar(1) – Linux Manual Page*, `https://man7.org/linux/man-pages/man1/sar.1.html`, Accessed: 2026-03-29.

[155] *perf: Linux profiling with performance counters*, `https://perf.wiki.kernel.org/index.php/Main_Page`, Dec. 2024.

[156] *wrk - a HTTP benchmarking tool*, `https://github.com/wg/wrk`, Accessed: 2026-03-19.

[157] *Redis Benchmark*, `https://redis.io/docs/latest/operate/oss_and_stack/management/optimization/benchmarks/`, Accessed: 2026-03-30, 2026.

[158] J. Axboe, "Fio-flexible io tester," *URL http://freemeat.net/projects/fio*, 2014.

[159] *Overview of Single Root I/O Virtualization (SR-IOV)*, https://learn.microsoft.com/en-us/windows-hardware/drivers/network/overview-of-single-root-i-o-virtualization--sr-iov-, Accessed: 2026-03-03, 2024.

[160] K. Tian, Y. Zhang, L. Kang, Y. Zhao, and Y. Dong, "coIOMMU: A Virtual IOMMU with Cooperative DMA Buffer Tracking for Efficient Memory Management in Direct I/O," in *Proceedings of the 2020 USENIX Conference on Usenix Annual Technical Conference (USENIX ATC'20)*, Jul. 2020.

[161] A. Gordon, N. Amit, N. Har'El, M. Ben-Yehuda, A. Landau, A. Schuster, and D. Tsafrir, "ELI: Bare-Metal Performance for I/O Virtualization," in *Proceedings of the 17th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'12)*, Mar. 2012.

[162] N. Har'El, A. Gordon, A. Landau, M. Ben-Yehuda, A. Traeger, and R. Ladelsky, "Efficient and Scalable Paravirtual I/O System," in *Proceedings of the 2013 USENIX Annual Technical Conference (USENIX ATC'13)*, Jun. 2013.

[163] *QEMU 10.0*, https://github.com/qemu/qemu/tree/stable-10.0, Accessed: 2026-03-10, 2026.

[164] N. Amit, M. Ben-Yehuda, and B.-A. Yassour, "IOMMU: Strategies for Mitigating the IOTLB Bottleneck," in *Proceedings of the 2010 International Symposium on Computer Architecture (ISCA'10)*, Jun. 2010.

[165] RISELab, *AI and the memory wall*, https://medium.com/riselab/ai-and-memory-wall-2cb4265cb0b8, Accessed: 2023-10-01.

[166] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, "FlexGen: High-throughput generative inference of large

language models with a single GPU," in *Proceedings of the 40th International Conference on Machine Learning (ICML'23)*, 2023.

[167] C. Luo, Z. Cai, H. Sun, J. Xiao, B. Yuan, W. Xiao, J. Hu, J. Zhao, B. Chen, and A. Anandkumar, *HeadInfer: Memory-efficient LLM inference by head-wise offloading*, 2025. arXiv: 2502.12574 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2502.12574

[168] Meta Platforms, Inc., *Table batched embedding (TBE) training module—FBGEMM 1.4.0 documentation*, https://docs.pytorch.org/FBGEMM/fbgemm_gpu/python-api/tbe_ops_training.html, Accessed: 2026-04-15, 2025.

[169] Hugging Face, *KV cache offloading in transformers*, https://huggingface.co/docs/transformers/en/kv_cache, Accessed: 2025-04-01.

[170] PyTorch, *Mixing different CUDA system allocators in the same program*, https://docs.pytorch.org/docs/stable/notes/cuda.html#mixing-different-cuda-system-allocators-in-the-same-program, Accessed: 2023-10-01.

[171] W. Zhu, G. Cox, J. Veselý, M. Hairgrove, A. L. Cox, and S. Rixner, "UVM discard: Eliminating redundant memory transfers for accelerators," in *2022 IEEE International Symposium on Workload Characterization (IISWC)*, 2022, pp. 27–38. DOI: 10.1109/IISWC55918.2022.00013